

ĐẠI HỌC QUỐC GIA TP. HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA

NGUYỄN TẤN KHÔI

ỨNG DỤNG HỌC CƯỜNG CỐ ĐIỀU KHIỂN PHÂN TÁN HỆ
ĐA ROBOT TRÁNH VA CHẠM

Chuyên ngành: Kỹ thuật Điều khiển - Tự động hóa
Mã số: 8520216

LUẬN VĂN THẠC SĨ

TP. HỒ CHÍ MINH, tháng 12 năm 2025

ĐẠI HỌC QUỐC GIA TP. HCM
TRƯỜNG ĐẠI HỌC BÁCH KHOA

NGUYỄN TẤN KHÔI

ỨNG DỤNG HỌC CƯỜNG CỐ ĐIỀU KHIỂN PHÂN TÁN HỆ
ĐA ROBOT TRÁNH VA CHẠM

Chuyên ngành: Kỹ thuật Điều khiển - Tự động hóa
Mã số: 8520216

LUẬN VĂN THẠC SĨ

TP. HỒ CHÍ MINH, tháng 12 năm 2025

**CÔNG TRÌNH ĐƯỢC HOÀN THÀNH TẠI
TRƯỜNG ĐẠI HỌC BÁCH KHOA – ĐHQG – HCM**

Cán bộ hướng dẫn khoa học: **TS. Phạm Việt Cường**

(Ghi rõ họ, tên, học hàm, học vị và chữ ký)

Cán bộ chấm nhận xét 1:

(Ghi rõ họ, tên, học hàm, học vị và chữ ký)

Cán bộ chấm nhận xét 2:

(Ghi rõ họ, tên, học hàm, học vị và chữ ký)

Luận văn thạc sĩ được bảo vệ tại Trường Đại học Bách Khoa, ĐHQG Tp. HCM
ngày tháng năm

Thành phần Hội đồng đánh giá luận văn thạc sĩ gồm:

(Ghi rõ họ, tên, học hàm, học vị của Hội đồng chấm bảo vệ luận văn thạc sĩ)

1.

2.

3.

4.

5.

Xác nhận của Chủ tịch Hội đồng đánh giá LV và Trưởng Khoa quản lý chuyên ngành sau khi luận văn đã được sửa chữa (nếu có).

CHỦ TỊCH HỘI ĐỒNG

TRƯỞNG KHOA ĐIỆN – ĐIỆN TỬ

NHIỆM VỤ LUẬN VĂN THẠC SĨ

Họ tên học viên: NGUYỄN TẤN KHÔI

MSHV: 2171017

Ngày, tháng, năm sinh: 08/02/1999

Nơi sinh: TP.HCM

Chuyên ngành: Kỹ thuật Điều khiển - Tự động hóa

Mã số: 8520216

I. TÊN ĐỀ TÀI: Ứng Dụng Học Cường Cố Điều Khiển Phân Tán Hệ Đa Robot Tránh Va Chạm
- Application of Reinforcement Learning in Decentralized Multi-Robot Collision Avoidance Control System

II. NHIỆM VỤ VÀ NỘI DUNG:

- Nghiên cứu tổng quan về các phương pháp tránh va chạm trong hệ đa robot.
- Tìm hiểu thuật toán Proximal Policy Optimization (PPO) và các kỹ thuật huấn luyện multi-agent.
- Đề xuất cải tiến về learning rate scheduling, value clipping và chiến lược huấn luyện hai giai đoạn.
- Thiết kế và chế tạo robot di động sử dụng cảm biến LiDAR và Jetson Nano.
- Triển khai thuật toán lên robot thực tế và đánh giá.

III. NGÀY GIAO NHIỆM VỤ: 25/08/2025

IV. NGÀY HOÀN THÀNH NHIỆM VỤ: 14/12/2025

V. CÁN BỘ HƯỚNG DẪN: TS. Phạm Việt Cường

CÁN BỘ HƯỚNG DẪN

(Họ tên và chữ ký)

Tp. HCM, ngày tháng năm 2025

CHỦ NHIỆM BỘ MÔN ĐÀO TẠO

(Họ tên và chữ ký)

TRƯỞNG KHOA ĐIỆN – ĐIỆN TỬ

(Họ tên và chữ ký)

LỜI CẢM ƠN

Tôi xin chân thành cảm ơn TS. Phạm Việt Cường, người đã tận tình hướng dẫn và định hướng cho tôi trong suốt quá trình thực hiện luận văn này. Sự góp ý và hỗ trợ chân thành của thầy đã giúp tôi có thêm kiến thức, hoàn thiện nghiên cứu và phát triển kỹ năng nghiên cứu khoa học.

Tôi cũng xin gửi lời cảm ơn đến các thầy cô trong Bộ môn Tự động hóa, Khoa Điện - Điện tử, Trường Đại học Bách Khoa TP.HCM đã truyền đạt kiến thức nền tảng vững chắc trong suốt quá trình học tập.

Cuối cùng, tôi xin cảm ơn gia đình, bạn bè đã luôn động viên và hỗ trợ tôi trong suốt thời gian thực hiện luận văn.

Nguyễn Tấn Khôi

TÓM TẮT LUẬN VĂN THẠC SĨ

Luận văn này nghiên cứu ứng dụng học tăng cường sâu (Deep Reinforcement Learning) vào bài toán điều khiển phân tán hệ đa robot tránh va chạm. Dựa trên thuật toán Proximal Policy Optimization (PPO), nghiên cứu đề xuất các cải tiến về adaptive learning rate scheduling, value clipping và chiến lược huấn luyện hai giai đoạn để cải thiện hiệu suất và khả năng mở rộng của hệ thống.

Các cải tiến đề xuất bao gồm Adaptive Learning Rate Scheduler, Value Clipping và kỹ thuật chuẩn hóa observation đã giúp cải thiện độ ổn định và tốc độ hội tụ của quá trình huấn luyện.

Nghiên cứu cũng trình bày thiết kế phần cứng robot sử dụng cảm biến LiDAR 360 độ và động cơ encoder điều khiển bằng PID, chuẩn bị cho việc triển khai thuật toán trên robot thực tế. Kết quả từ thực nghiệm cho thấy mô hình hoàn toàn có khả năng triển khai và hoạt động tốt không chỉ trên mô phỏng mà còn ở thực tế.

ABSTRACT

This thesis investigates the application of Deep Reinforcement Learning to decentralized multi-robot collision avoidance control. Based on the Proximal Policy Optimization (PPO) algorithm, the research proposes improvements in adaptive learning rate scheduling, value clipping, and a two-stage training strategy to enhance system performance and scalability.

The proposed improvements include Adaptive Learning Rate Scheduler, Value Clipping, and observation normalization techniques, which help improve training stability and convergence speed.

The research also presents hardware design for a mobile robot using 360-degree LiDAR sensor and PID-controlled encoder motors, preparing for real-world algorithm deployment. Experimental results demonstrate that the model is fully capable of deployment and performs well not only in simulation but also in real-world environments.

LỜI CAM ĐOAN

Tôi xin cam đoan rằng luận văn “*Ứng Dụng Học Cửng Cố Điều Khiển Phân Tán Hệ Đa Robot Tránh Va Chạm*” là công trình nghiên cứu của riêng tôi dưới sự hướng dẫn của TS. Phạm Việt Cường.

Các kết quả nghiên cứu và số liệu trong luận văn là trung thực, chưa từng được ai công bố trong bất kỳ công trình nào khác. Tôi xin hoàn toàn chịu trách nhiệm về tính chính xác và trung thực của nội dung luận văn này.

TP. Hồ Chí Minh, Tháng 12 năm 2025

Nguyễn Tấn Khôi

MỤC LỤC

LỜI CẢM ƠN

TÓM TẮT LUẬN VĂN THẠC SĨ

LỜI CAM ĐOAN

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

1	MỞ ĐẦU	1
1.1	Lý do chọn đề tài	1
1.2	Mục tiêu nghiên cứu	1
1.3	Đối tượng và phạm vi nghiên cứu	2
1.3.1	Đối tượng nghiên cứu	2
1.3.2	Phạm vi nghiên cứu	2
1.4	Ý nghĩa khoa học và thực tiễn	3
1.4.1	Ý nghĩa khoa học	3
1.4.2	Ý nghĩa thực tiễn	3
2	TỔNG QUAN	4
2.1	Giới thiệu về hệ đa robot	4
2.2	Các phương pháp tránh va chạm truyền thống	4
2.3	Các phương pháp dựa trên học sâu	5
2.4	Phương pháp huấn luyện với PPO được đề xuất	6
2.4.1	Mô hình bài toán	6
2.4.2	Hàm reward	7
2.4.3	Kiến trúc mạng nơ-ron	7
2.4.4	Thuật toán huấn luyện	8
2.4.5	Môi trường và kịch bản huấn luyện	8
2.5	Cơ sở lý thuyết	9
2.5.1	Các thuật toán tối ưu	9
2.5.1.1	Gradient Descent	9
2.5.1.2	Adam Optimizer (Adaptive Moment Estimation) . .	10

2.5.2	Reinforcement Learning	13
2.5.3	Multi-Agent Reinforcement Learning (MARL)	14
2.5.3.1	Centralized Training with Decentralized Execution (CTDE)	15
2.5.4	Proximal Policy Optimization (PPO)	16
2.5.4.1	Policy Gradient cơ bản	16
2.5.4.2	Proximal Policy Optimization (PPO)	17
2.5.4.3	Value Function Clipping	18
2.5.4.4	Complete PPO Objective	18
2.5.4.5	PPO-Penalty: Alternative Formulation	19
2.5.4.6	Lý do chọn PPO cho bài toán đa robot	20
2.5.4.7	PPO Hyperparameters	21
2.5.5	Kiến trúc mạng nơ-ron	22
2.5.5.1	Fully Connected Neural Networks	22
2.5.5.2	Convolutional Neural Networks (CNN)	23
2.5.5.3	Loss Functions	25
2.5.5.4	Backpropagation - Lan truyền ngược	25
2.5.5.5	Kỹ thuật Regularization	26
2.5.6	Điều khiển PID	27
2.5.7	UKF Sensor Fusion	28
2.5.8	Cartographer và SLAM	30
2.5.8.1	Bài toán SLAM	30
2.5.8.2	Google Cartographer	31
2.5.8.3	Nguyên lý hoạt động	31
2.5.8.4	Kiến trúc hệ thống	32
2.5.8.5	Quy trình mapping	32
2.5.8.6	Các tham số quan trọng	33
3	PHƯƠNG PHÁP NGHIÊN CỨU	35
3.1	Sơ đồ khối tổng thể	35
3.2	Huấn luyện trên môi trường mô phỏng	36
3.2.1	Môi trường huấn luyện	36
3.2.1.1	Không gian quan sát	36
3.2.1.2	Không gian hành động	36
3.2.2	Thiết kế hàm reward	36
3.2.2.1	Terminal rewards	37
3.2.2.2	Step rewards - Stage 1	37
3.2.2.3	Điều chỉnh reward cho Stage 2	38
3.2.3	Chiến lược huấn luyện 2 giai đoạn	38

3.2.4	Kiến trúc mạng nơ-ron	40
3.2.4.1	Encoder - Xử lý LiDAR	40
3.2.4.2	Mạng Actor - Sinh policy	41
3.2.4.3	Mạng Critic - Ước lượng giá trị	41
3.2.4.4	Entropy và Exploration	42
3.2.5	Thuật toán PPO với GAE	43
3.2.6	Hyperparameters chi tiết	44
3.2.7	Các kỹ thuật cải tiến	45
3.2.7.1	Orthogonal Initialization	45
3.2.7.2	Chuẩn hóa Observation	46
3.2.7.3	Log-std Clamping - Giới hạn log-std	46
3.2.7.4	Adaptive Learning Rate Scheduler	47
3.2.7.5	Aggressive Exploration Strategy (Chiến lược khám phá)	47
3.3	Triển khai model và robot thực tế	48
3.3.1	Xây dựng robot thực tế	48
3.3.1.1	Thông số kỹ thuật	48
3.3.1.2	Kiến trúc phần mềm	48
3.3.2	Thiết kế PID controller	49
3.3.2.1	Mô hình động học differential drive robot	49
3.3.2.2	Thiết kế PID controller	49
3.3.2.3	Phân tích ổn định	50
3.3.2.4	Kết quả thực nghiệm	52
3.3.3	Thiết kế sensor fusion với UKF	53
3.3.3.1	Sensor fusion - Đầu vào và đầu ra	53
3.3.3.2	Ý tưởng chính của UKF	54
3.3.3.3	Mô hình state và measurements	54
3.3.3.4	Thuật toán UKF - Các bước chính	55
3.3.4	Triển khai Cartographer SLAM	55
3.3.4.1	Quy trình Mapping	55
3.3.4.2	Quá trình Tinh chỉnh Tham số	56
3.3.4.3	Quy trình Navigation/Localization	57
3.3.4.4	Công cụ Debug và Tinh chỉnh	58
3.3.5	Thiết kế an toàn trong thực nghiệm	58
3.3.5.1	Giới hạn tốc độ tối đa	59
3.3.5.2	Velocity ramping trên Arduino	59
3.3.6	Triển khai hệ robot hoàn chỉnh	60
3.3.6.1	Kiến trúc tổng thể	60
3.3.6.2	Cấu hình phần cứng	60

3.3.6.3	Cấu hình mạng ROS	61
3.3.6.4	Phân bổ các ROS nodes	61
3.3.6.5	Thiết kế phi tập trung với inference tập trung	62
3.3.6.6	Tóm tắt deployment	63
4	KẾT QUẢ VÀ THẢO LUẬN	64
4.1	Kết quả mô phỏng	64
4.1.1	Kết quả Stage 1 - Môi trường đơn giản	64
4.1.2	Kết quả Stage 2 - Môi trường phức tạp	66
4.1.3	So sánh và phân tích	69
4.2	Kết quả thực nghiệm	71
4.2.1	Thiết lập thực nghiệm	71
4.2.2	Kết quả thực nghiệm	72
4.2.2.1	Kết quả không có vật cản	72
4.2.2.2	Kết quả với vật cản tĩnh	73
4.2.2.3	Kết quả với vật cản động (người)	74
4.2.2.4	So sánh với kết quả mô phỏng	75
4.3	Thảo luận	76
4.3.1	Hiệu quả của curriculum learning	76
4.3.2	Khả năng mở rộng số lượng robots	76
4.3.3	Từ mô phỏng đến thực tế (Sim-to-real)	76
4.3.4	Hạn chế của nghiên cứu	77
4.3.5	Hướng cải tiến	78
5	KẾT LUẬN VÀ KIẾN NGHỊ	80
5.1	Kết luận	80
5.1.1	Về thuật toán và huấn luyện	80
5.1.2	Về thiết kế và triển khai phần cứng	80
5.1.3	Về triển khai từ mô phỏng vào thực tế	81
5.1.4	Đánh giá tổng thể	81
5.2	Kiến nghị	81
5.2.1	Cải tiến phần cứng	81
5.2.2	Cải tiến thuật toán	81
5.2.3	Mở rộng quy mô thực nghiệm	82
5.3	Hướng phát triển	82
5.4	Lời kết	82
	PHỤ LỤC	86
	A. Dữ liệu chi tiết kết quả thực nghiệm	86

DANH MỤC HÌNH

2.1	Minh họa Gradient Descent: Tham số di chuyển theo hướng ngược gradient để giảm loss	10
2.2	Kiến trúc Actor-Critic: Actor quyết định hành động, Critic đánh giá giá trị trạng thái	14
2.3	PPO Clipping: Objective bị chặn ngoài khoảng $[1 - \epsilon, 1 + \epsilon]$ để ngăn policy thay đổi quá nhanh	18
2.4	Kiến trúc thuật toán Cartographer SLAM với hai hệ thống con: Local SLAM (frontend) xử lý real-time và Global SLAM (backend) tối ưu hóa toàn cục	32
3.1	Sơ đồ khối tổng thể hệ thống: Huấn luyện MARL trong mô phỏng (trái) và triển khai trên robot thực (phải)	35
3.2	Stage 1: môi trường đơn giản cho 24 robots	39
3.3	Stage 2: 7 tình huống huấn luyện đa dạng từ đơn giản đến phức tạp giúp policy generalize tốt (nguồn: Long et al. 2018)	40
3.4	Kiến trúc mạng Actor-Critic.	41
3.5	Xác định tham số động cơ từ thực nghiệm. (a) Đáp ứng step để xác định time constant $\tau = 0.05s$: các điểm đo (xanh) có nhiễu nhẹ nhưng fit tốt với mô hình lý thuyết (đường xanh đậm); tại $t = \tau$, vận tốc đạt 63.2% giá trị ổn định. (b) Quan hệ tuyến tính PWM-vận tốc để xác định motor gain: 9 điểm đo (đỏ) cho fitting $K_m \approx 0.81$, làm tròn thành $K_m = 0.8$ cho mô hình.	51
3.6	Đáp ứng step response của bộ PID. Hàng trên: $v = 0.4$ m/s (88 units), hàng dưới: $v = -0.5$ m/s (-110 units). Cột trái: step response 0-1s, cột phải: zoom vào vùng steady-state (0.3-2s) để thấy dao động quanh setpoint.	53
3.7	Kiến trúc hệ thống robot hoàn chỉnh	60
4.1	Stage 1: (a) Success rate tăng từ 2.57% lên 83.03%, (b) Collision rate giảm từ 76.93% xuống 14.11%, (c) Average reward tăng từ -13.97 lên 46.87.	65
4.2	Dữ liệu metrics huấn luyện Stage 1 chưa qua xử lý	65

4.3	Stage 2: (a) Success rate tăng từ 34.66% lên 89.18%, (b) Collision rate giảm từ 65.28% xuống 9.04%, (c) Average reward tăng từ -10.36 lên 62.46.	67
4.4	Dữ liệu metrics huấn luyện Stage 2 chưa qua xử lý	67
4.5	Circle Test với 5 robots: Các robots (chấm màu) di chuyển từ vị trí ban đầu đến goal đối diện. Đường màu thể hiện quỹ đạo di chuyển, vùng tròn màu xanh là phạm vi quan sát của mỗi robot.	69
4.6	Circle Test với 24 robots: (a) Robots bắt đầu di chuyển từ vòng tròn bán kính 10m, tạo thành pattern xoắn để tránh va chạm tại tâm; (b) Robots hoàn thành di chuyển đến vị trí đối diện với quỹ đạo xoắn ốc đặc trưng của thuật toán tránh va chạm.	69
4.7	Môi trường thực nghiệm trong nhà (4m x 3m)	72
4.8	Robot tránh vật cản tĩnh: trajectory màu xanh lá cho thấy robot đi vòng qua vật cản để đến goal	74
4.9	Robot tránh vật cản động (người): đường xanh dương là quỹ đạo khi dựng map, đường xanh lá là quỹ đạo hiện tại. Hai hình chữ V là chân người được LiDAR phát hiện.	75

DANH MỤC BẢNG

2.1	So sánh các phương pháp tránh va chạm truyền thống	5
2.2	So sánh PPO-Clip và PPO-Penalty	20
2.3	So sánh PPO với các thuật toán RL khác cho bài toán robot navigation	21
2.4	So sánh UKF và EKF	30
3.1	Hyperparameters Stage 1 (24 robots - 83% success)	45
3.2	Hyperparameters Stage 2 (44 robots - 89% success)	45
3.3	Bảng Routh-Hurwitz cho hệ PID bậc 3	51
3.4	Tham số Cartographer sau tinh chỉnh cho Jetson Nano + LiDAR LD19	57
3.5	Cấu hình phần cứng hệ thống	61
3.6	Phân bổ ROS nodes trên các thiết bị	62
4.1	Metrics huấn luyện Stage 1 qua các mốc chính	66
4.2	Metrics huấn luyện Stage 2 qua các mốc chính	68
4.3	Kết quả Circle Test theo số lượng robots	70
4.4	Mối quan hệ giữa khoảng cách inter-robot và success rate	70
4.5	So sánh kết quả Circle Test với bài báo gốc CADRL và NH-ORCA . .	70
4.6	Tổng hợp kết quả thực nghiệm theo nhóm các trường hợp	72
4.7	Chi tiết kết quả test không có vật cản	73
4.8	Chi tiết kết quả test với vật cản tĩnh	73
4.9	Chi tiết kết quả test với vật cản động (người)	74
4.10	So sánh kết quả thực nghiệm với mô phỏng	75
5.1	Dữ liệu chi tiết toàn bộ kết quả thực nghiệm	86

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

CNN	Convolutional Neural Network - Mạng nơ-ron tích chập
CTDE	Centralized Training with Decentralized Execution - Huấn luyện tập trung, thực thi phân tán
DRL	Deep Reinforcement Learning - Học tăng cường sâu
MARL	Multi-Agent Reinforcement Learning - Học tăng cường đa tác tử
GAE	Generalized Advantage Estimation - Ước lượng ưu thế tổng quát
HCMUT	Ho Chi Minh City University of Technology - Trường Đại học Bách Khoa TP.HCM
LiDAR	Light Detection and Ranging - Công nghệ quét laser để đo khoảng cách
POMDP	Partially Observable Markov Decision Process - Quá trình quyết định Markov quan sát từng phần
PPO	Proximal Policy Optimization - Tối ưu hóa chính sách gần kề
RL	Reinforcement Learning - Học tăng cường
a_t	Action at time step t - Hành động tại bước thời gian t
o_t	Observation at time step t - Quan sát tại bước thời gian t
r_t	Reward at time step t - Phần thưởng tại bước thời gian t
v	Linear velocity - Vận tốc tuyến tính
ω	Angular velocity - Vận tốc góc (omega)
π	Policy function - Hàm chính sách (pi)
θ	Neural network parameters - Tham số mạng nơ-ron (theta)

Ghi chú: Danh sách này sẽ được cập nhật thêm các ký hiệu khác khi xuất hiện trong các chương của luận văn.

CHƯƠNG 1

MỞ ĐẦU

1.1 Lý do chọn đề tài

Trong những năm gần đây, hệ thống đa robot đã và đang được ứng dụng rộng rãi trong nhiều lĩnh vực như kho hàng tự động, nhà máy thông minh, và logistics. Việc điều khiển nhiều robot hoạt động đồng thời trong cùng một môi trường đặt ra thách thức lớn về tránh va chạm, đặc biệt khi số lượng robot tăng lên.

Các phương pháp điều khiển tập trung truyền thống gặp khó khăn về khả năng mở rộng (scalability) khi số lượng robot lớn, do yêu cầu về tính toán và truyền thông tăng theo cấp số nhân. Ngược lại, phương pháp điều khiển phân tán, trong đó mỗi robot tự quyết định dựa trên quan sát cục bộ, cho phép hệ thống mở rộng tốt hơn.

Học tăng cường sâu (Deep Reinforcement Learning - DRL) đã chứng minh hiệu quả trong các bài toán điều khiển phức tạp. Thuật toán Proximal Policy Optimization (PPO) nổi bật nhờ tính ổn định trong quá trình huấn luyện, được ứng dụng rộng rãi từ điều khiển robot đến tinh chỉnh mô hình ngôn ngữ lớn (LLM). Tuy nhiên, việc mở rộng DRL cho hệ thống đa robot quy mô lớn vẫn là hướng nghiên cứu còn nhiều thách thức.

1.2 Mục tiêu nghiên cứu

Mục tiêu chính của luận văn là nghiên cứu và phát triển thuật toán điều khiển phân tán cho hệ đa robot tránh va chạm sử dụng học tăng cường sâu. Cụ thể:

1. Nghiên cứu thuật toán PPO và các phương pháp học tăng cường cho bài toán đa robot.
2. Huấn luyện mô hình neural network có khả năng điều khiển robots tránh va chạm trong môi trường mô phỏng.
3. Đề xuất các cải tiến về learning rate scheduling, value clipping và policy để đáp ứng với thực nghiệm.
4. Thiết kế phần cứng robot prototype với cảm biến LiDAR, Jetson và Arduino đơn giản và chi phí thấp.
5. Triển khai và kiểm nghiệm thực tế mô hình trên robot trong thực nghiệm.

1.3 Đối tượng và phạm vi nghiên cứu

1.3.1 Đối tượng nghiên cứu

Đối tượng nghiên cứu là hệ thống đa robot di động (mobile robots) với các đặc điểm:

- **Kiểu robot:** Nonholonomic robots (robot không toàn hướng) di chuyển trên mặt phẳng 2D
- **Cảm biến:** LiDAR 360° với độ phân giải 0.8°
- **Hành động:** Điều khiển vận tốc tuyến tính v và vận tốc góc ω
- **Số lượng:** 44 robots hoạt động đồng thời trong mô phỏng; 1 robot trong thực nghiệm

1.3.2 Phạm vi nghiên cứu

Luận văn tập trung vào các khía cạnh sau:

- Thuật toán học tăng cường PPO cho bài toán điều khiển phân tán
- Môi trường mô phỏng 2D với chướng ngại vật tĩnh
- Quan sát cục bộ (local observation) từ cảm biến LiDAR
- Không sử dụng truyền thông giữa các robot (fully decentralized)
- Tuning PID ổn định điều khiển robot thực nghiệm
- Triển khai được model huấn luyện từ mô phỏng vào robot thực nghiệm
- Sensor fusing để robot có thể hoạt động và xác định chính xác vị trí của nó trong môi trường với các cảm biến giá rẻ
- Mục tiêu: Đạt tỉ lệ thành công robot di chuyển từ vị trí xuất phát đến đích mà không va chạm trên 80%

Ngoài phạm vi: Môi trường 3D, multi-task learning, xác định vị trí chính xác robot.

1.4 Ý nghĩa khoa học và thực tiễn

1.4.1 Ý nghĩa khoa học

Luận văn đóng góp vào lĩnh vực nghiên cứu đa robot và học tăng cường thông qua:

- Đánh giá tính ứng dụng của mô hình học tăng cường trong điều khiển hệ robot phân tán
- Đề xuất các cải tiến về thuật toán huấn luyện PPO:
 - *Adaptive Learning Rate Scheduler*: Điều chỉnh learning rate động dựa trên performance để thoát khỏi plateau và duy trì momentum khi đang học tốt
 - *Dense Rewards*: Cải tiến hàm reward với các thành phần chi tiết (timeout penalty, heading guidance, safety margin, smoothness) để hướng dẫn robot học hiệu quả hơn
 - *Observation Normalization*: Chuẩn hóa input sử dụng running statistics để ổn định quá trình học
 - *Entropy Decay*: Giảm dần entropy coefficient từ 8×10^{-3} xuống 2×10^{-3} để cân bằng exploration và exploitation
 - *Value Clipping*: Giảm instability trong quá trình huấn luyện
 - *Two-stage Training*: Chiến lược huấn luyện hai giai đoạn để cải thiện khả năng mở rộng
- Nghiên cứu khả năng thực tiễn của mô hình với robot Nonholonomic

1.4.2 Ý nghĩa thực tiễn

Kết quả nghiên cứu có thể ứng dụng vào:

- **Kho hàng tự động (Automated Warehouses)**: Điều khiển nhiều robot AGV (Automated Guided Vehicle) di chuyển hàng hóa hiệu quả.
- **Nhà máy thông minh (Smart Factories)**: Phối hợp nhiều robot công nghiệp trong dây chuyền sản xuất.
- **Logistics và vận tải**: Quản lý đội xe tự hành trong khu vực hạn chế.

Kết quả đạt được trên robot thực nghiệm chứng minh tính khả thi của phương pháp trong điều kiện thực tế với mô hình phần cứng đơn giản và dễ dàng mở rộng với số lượng robot lớn.

CHƯƠNG 2

TỔNG QUAN

2.1 Giới thiệu về hệ đa robot

Hệ đa robot (Multi-Robot Systems - MRS) là hệ thống bao gồm nhiều robot hoạt động đồng thời trong cùng một môi trường để thực hiện một hoặc nhiều nhiệm vụ chung. Hệ đa robot có thể được phân loại dựa trên mức độ phối hợp giữa các robot: từ hoàn toàn độc lập (independent) đến phối hợp chặt chẽ (tightly coordinated), và dựa trên kiến trúc điều khiển: tập trung (centralized) hoặc phân tán (decentralized).

Trong những năm gần đây, hệ đa robot đã được ứng dụng rộng rãi trong nhiều lĩnh vực thực tế. Tại các kho hàng tự động như Amazon Centers, hàng trăm robot di chuyển đồng thời để vận chuyển hàng hóa, giúp tăng hiệu suất và giảm chi phí nhân công. Trong sản xuất công nghiệp, các robot di động (Automated Guided Vehicles - AGV) phối hợp vận chuyển nguyên liệu và sản phẩm giữa các trạm làm việc. Hệ đa robot cũng được ứng dụng trong logistics, nông nghiệp thông minh, cứu hộ thảm họa, và thăm dò không gian.

Một trong những thách thức chính trong điều khiển hệ đa robot là vấn đề tránh va chạm (collision avoidance). Khi số lượng robot tăng lên, không gian hoạt động trở nên chật hẹp và xác suất va chạm giữa các robot hoặc với chướng ngại vật tăng cao. Điều này đòi hỏi mỗi robot phải có khả năng cảm nhận môi trường xung quanh, dự đoán chuyển động của các robot khác, và điều chỉnh quỹ đạo của mình để tránh va chạm trong khi vẫn đạt được mục tiêu đề ra. Vấn đề trở nên phức tạp hơn khi môi trường có chướng ngại vật động, không gian hạn chế, hoặc yêu cầu thời gian phản ứng nhanh.

2.2 Các phương pháp tránh va chạm truyền thống

Các phương pháp tránh va chạm truyền thống thường dựa trên mô hình toán học và quy tắc xác định (rule-based) để đảm bảo an toàn cho robot. Các phương pháp này có ưu điểm về tính minh bạch và khả năng chứng minh an toàn, nhưng thường gặp khó khăn khi áp dụng cho hệ đa robot với số lượng lớn.

Artificial Potential Field (APF) [1] là một trong những phương pháp sớm nhất và đơn giản nhất. Phương pháp này mô hình hóa môi trường như một trường thế năng, trong đó mục tiêu tạo ra lực hút (attractive force) và các chướng ngại vật tạo ra lực đẩy (repulsive force). Robot di chuyển theo hướng của gradient âm của trường thế năng. Tuy nhiên, APF thường gặp vấn đề local minima, trong đó robot có thể bị mắc kẹt tại

điểm cân bằng không phải là mục tiêu.

Rapidly-exploring Random Tree (RRT) [2] và biến thể tối ưu RRT* [3] là các thuật toán lấy mẫu ngẫu nhiên (sampling-based) để tìm đường đi trong không gian cấu hình. RRT xây dựng một cây tìm kiếm bằng cách mở rộng ngẫu nhiên từ điểm khởi đầu đến mục tiêu. RRT* cải thiện RRT bằng cách tối ưu hóa chi phí đường đi. Các phương pháp này hiệu quả cho bài toán tìm đường trong không gian phức tạp, nhưng không phù hợp cho điều khiển thời gian thực với nhiều robot do chi phí tính toán cao.

Optimal Reciprocal Collision Avoidance (ORCA) [4] là phương pháp phổ biến cho tránh va chạm đa tác tử. ORCA tính toán vận tốc an toàn cho mỗi robot bằng cách xác định một tập hợp các vận tốc không gây va chạm trong một khoảng thời gian nhất định. Mỗi robot chọn vận tốc gần nhất với vận tốc mong muốn trong tập hợp này. ORCA đảm bảo không va chạm nếu tất cả robot tuân theo quy tắc, nhưng phương pháp này yêu cầu robot có mô hình động học holonomic (có thể di chuyển theo mọi hướng), trong khi nhiều robot thực tế là nonholonomic (ví dụ: differential drive robot).

Bảng 2.1 tổng hợp so sánh các phương pháp truyền thống. Có thể thấy, các phương pháp này có ưu điểm về tính toán nhanh và khả năng chứng minh an toàn, nhưng hạn chế chính là khó khăn khi mở rộng cho hệ đa robot với số lượng lớn (>50 robots) và môi trường động phức tạp. Điều này tạo động lực cho việc nghiên cứu các phương pháp dựa trên học máy có khả năng học từ dữ liệu và thích nghi với môi trường.

Bảng 2.1. So sánh các phương pháp tránh va chạm truyền thống

Phương pháp	Ưu điểm	Nhược điểm	Khả năng mở rộng
Artificial Potential Field [1]	Đơn giản, tính toán nhanh, dễ triển khai	Local minima, khó điều chỉnh tham số, không đảm bảo tối ưu	Khó với >20 robots
RRT/RRT* [2], [3]	Tìm đường trong không gian phức tạp, RRT* đảm bảo tối ưu tiệm cận	Chi phí tính toán cao, không phù hợp real-time, cần re-plan thường xuyên	Không phù hợp cho đa robot
ORCA [4]	Đảm bảo không va chạm, phân tán, phù hợp real-time	Yêu cầu holonomic robot, không tối ưu đường đi, cần giao tiếp	Tốt đến 50 robots

2.3 Các phương pháp dựa trên học sâu

Trong những năm gần đây, học sâu (Deep Learning - DL) [5] đã đạt được thành công vượt bậc trong nhiều lĩnh vực như thị giác máy tính, xử lý ngôn ngữ tự nhiên, và điều

khiến robot. Kết hợp học sâu với học tăng cường (Reinforcement Learning - RL) tạo ra phương pháp học tăng cường sâu (Deep Reinforcement Learning - DRL) có khả năng học các chính sách điều khiển phức tạp trực tiếp từ dữ liệu cảm biến.

Deep Q-Network (DQN) [6] là một trong những bước đột phá đầu tiên của DRL, cho phép agent học chơi các trò chơi Atari ở mức độ con người chỉ từ pixels. DQN sử dụng mạng nơ-ron sâu để xấp xỉ hàm giá trị Q và kết hợp experience replay để ổn định quá trình học. Tuy nhiên, DQN được thiết kế cho không gian hành động rời rạc, không phù hợp cho điều khiển robot với hành động liên tục.

Để giải quyết vấn đề hành động liên tục, các thuật toán policy gradient như **Asynchronous Advantage Actor-Critic (A3C)** [7] được phát triển. A3C sử dụng nhiều agent song song để thu thập dữ liệu và cập nhật policy, giúp tăng tốc độ học và ổn định training. Kiến trúc Actor-Critic bao gồm hai mạng: Actor đưa ra hành động và Critic đánh giá hành động đó. Tuy nhiên, A3C vẫn gặp vấn đề về độ ổn định khi learning rate quá lớn.

Ứng dụng DRL cho điều khiển robot di động đã được nghiên cứu rộng rãi. **Tai et al.** [8] đề xuất phương pháp học navigation không cần bản đồ (mapless navigation) cho robot di động sử dụng DRL với đầu vào là dữ liệu LiDAR. Phương pháp này cho phép robot học tránh chướng ngại vật và di chuyển đến mục tiêu trong môi trường chưa biết trước. Điểm mạnh là không cần xây dựng bản đồ chi tiết, nhưng hạn chế là chỉ xét robot đơn lẻ.

Đối với bài toán đa robot, có hai cách tiếp cận chính: tập trung (centralized) và phân tán (decentralized). Cách tiếp cận tập trung sử dụng một agent trung tâm điều khiển tất cả robots, có ưu điểm là dễ tối ưu hóa toàn cục nhưng gặp khó khăn về khả năng mở rộng và đòi hỏi giao tiếp đáng tin cậy. Ngược lại, cách tiếp cận phân tán [9] cho phép mỗi robot có policy riêng và ra quyết định độc lập dựa trên quan sát cục bộ. Cách tiếp cận này mở rộng tốt hơn cho số lượng robot lớn và không yêu cầu giao tiếp, phù hợp cho ứng dụng thực tế.

2.4 Phương pháp huấn luyện với PPO được đề xuất

Bài báo của Long et al. [10] đề xuất phương pháp điều khiển phân tán cho hệ đa robot tránh va chạm sử dụng deep reinforcement learning. Đây là công trình nền tảng cho luận văn này.

2.4.1 Mô hình bài toán

Bài toán được công thức hóa như một Partially Observable Markov Decision Process (POMDP) cho mỗi robot. Điểm khác biệt quan trọng so với các phương pháp trước (ORCA, GA3C-CADRL) là không giả định "perfect sensing" mỗi robot chỉ quan sát

được môi trường xung quanh từ sensor của chính nó, không biết chính xác vị trí và ý định của robots khác.

Không gian quan sát: Mỗi robot quan sát $\mathbf{o}^t = [\mathbf{o}_z^t, \mathbf{o}_g^t, \mathbf{o}_v^t]$ gồm:

- Laser scan 180° với 512 beams (tia) từ 3 frames liên tiếp
- Vị trí tương đối của goal (khoảng cách và góc)
- Vận tốc hiện tại (linear và angular).

Action space: Hành động là vận tốc $\mathbf{a}^t = [v^t, w^t]$ với $v^t \in (0, 1.0)$ m/s (không cho phép lùi) và $w^t \in (-1.0, 1.0)$ rad/s, được sample từ stochastic policy π_θ chia sẻ bởi tất cả robot.

Mục tiêu: Cực tiểu thời gian đến đích trung bình của tất cả robot trong khi đảm bảo không va chạm với nhau ($\|\mathbf{p}_t^i - \mathbf{p}_t^j\| > 2R$) và với vật cản.

2.4.2 Hàm reward

Hàm reward gồm ba thành phần: $r_t^i = (g_r)_t^i + (c_r)_t^i + (w_r)_t^i$

- **Goal reward:** $r_{\text{arrival}} = +15$ khi đến đích ($\|\mathbf{p}_t^i - \mathbf{g}_i\| < 0.1$); trong khi di chuyển, reward $\omega_g(\|\mathbf{p}_{t-1}^i - \mathbf{g}_i\| - \|\mathbf{p}_t^i - \mathbf{g}_i\|)$ với $\omega_g = 2.5$ tỷ lệ với khoảng cách tiến gần đích.
- **Collision penalty:** $r_{\text{collision}} = -15$ khi va chạm với robot khác ($\|\mathbf{p}_t^i - \mathbf{p}_t^j\| < 2R$) hoặc vật cản. Giá trị tuyệt đối bằng arrival reward để cân bằng giữa đến đích nhanh và tránh va chạm.
- **Smoothness penalty - Phạt tăng giảm tốc đột ngột:** Phạt nhẹ $\omega_w|w_t^i| = -0.1|w_t^i|$ khi vận tốc góc lớn ($|w| > 0.7$) để khuyến khích chuyển động mượt mà.

2.4.3 Kiến trúc mạng nơ-ron

Policy network π_θ ánh xạ trực tiếp từ dữ liệu quan sát gốc của cảm biến (raw sensor observation) sang tín hiệu điều khiển (steering command), gồm 4 hidden layers:

Layers 1-2 (Conv1D): Xử lý laser scan với 2 lớp tích chập 1D:

- Conv1D(32 filters, kernel=5, stride=2) + ReLU
- Conv1D(32 filters, kernel=3, stride=2) + ReLU

Hai lớp này trích xuất đặc trưng không gian từ 1536 điểm laser (3×512) thành 4032 đặc trưng. Việc dùng 3 khung hình liên tiếp giúp mạng học thông tin về chuyển động tương đối.

Lớp 3 (FC): 256 nút, flatten output từ Conv1D và tổng hợp đặc trưng không gian.

Lớp 4 (FC): 128 nút. Lớp này concatenate với goal position (2D) và vận tốc hiện tại (2D) để tích hợp thông tin từ LiDAR, nhiệm vụ, và trạng thái động học.

Lớp đầu ra: 2 nút với Sigmoid (cho $v \in (0, 1)$) và Tanh (cho $w \in (-1, 1)$). Output là mean của phân phối Gaussian $\mathcal{N}(\mathbf{v}_{\text{mean}}, \mathbf{v}_{\text{logstd}})$ để sample action. Stochastic policy cho phép exploration trong quá trình huấn luyện.

Value network: Cùng kiến trúc nhưng output là 1 nút (linear activation) để ước lượng phần thưởng kỳ vọng. Không share parameters với policy network.

2.4.4 Thuật toán huấn luyện

Centralized learning, decentralized execution: Một policy duy nhất π_θ được huấn luyện từ experience của tất cả N robot đồng thời (sample efficiency cao, experience đa dạng). Khi thực thi, mỗi robot sample action từ policy:

$$\mathbf{a}^t \sim \pi_\theta(\mathbf{a}^t | \mathbf{o}^t) \quad (2.1)$$

chỉ dựa vào observation riêng, không cần giao tiếp giữa các robot.

Hàm mục tiêu PPO: Thuật toán PPO sử dụng hàm mục tiêu có thêm một thành phần phạt KL (adaptive KL penalty). Hàm mục tiêu được viết như sau:

$$L^{PPO}(\theta) = \mathbb{E}_t \left[\frac{\pi_\theta(\mathbf{a}_t | \mathbf{o}_t)}{\pi_{\text{old}}(\mathbf{a}_t | \mathbf{o}_t)} \hat{A}_t \right] - \beta \text{KL}[\pi_{\text{old}} \| \pi_\theta] \quad (2.2)$$

trong đó $\hat{A}_t$ là ước lượng độ lợi (advantage estimate), β được điều chỉnh tự động để cân bằng tốc độ học và độ ổn định.

Advantage estimation: Sử dụng GAE với $\lambda = 0.95$, $\gamma = 0.99$:

$$\hat{A}_t = \sum_{l=0}^T (\gamma \lambda)^l \delta_t, \quad \delta_t = r_t + \gamma V_\phi(\mathbf{s}_{t+1}) - V_\phi(\mathbf{s}_t) \quad (2.3)$$

2.4.5 Môi trường và kịch bản huấn luyện

Môi trường mô phỏng: Sử dụng thư viện mô phỏng Stage với 2 môi trường huấn luyện tương ứng với 2 giai đoạn khác nhau. Giai đoạn 1: môi trường đơn giản, ít vật cản. Giai đoạn 2: 7 kịch bản đa dạng: (1-3, 5-6) Môi trường có tường/vật cản với điểm bắt đầu/kết thúc cố định, (4) Trường hợp vòng tròn với các robot đổi chỗ qua tâm, (7) Kịch bản ngẫu nhiên với vị trí robot và vật cản ngẫu nhiên. Tất cả robot di chuyển đồng thời.

2.5 Cơ sở lý thuyết

2.5.1 Các thuật toán tối ưu

Các thuật toán tối ưu đóng vai trò then chốt trong việc huấn luyện mạng nơ-ron sâu. Mục tiêu của các thuật toán này là tìm tập tham số θ^* cực tiểu hàm loss $J(\theta)$. Trong phần này, chúng ta sẽ phân tích chi tiết các thuật toán từ cơ bản đến hiện đại được sử dụng rộng rãi trong deep learning.

2.5.1.1 Gradient Descent

Gradient Descent (GD) là thuật toán tối ưu cơ bản nhất, trong đó tham số được cập nhật theo hướng ngược với gradient của hàm loss:

$$\theta_{t+1} = \theta_t - \alpha \nabla_{\theta} J(\theta_t) \quad (2.4)$$

trong đó $\alpha > 0$ là learning rate (tốc độ học), và $\nabla_{\theta} J(\theta_t)$ là gradient của loss function tại θ_t .

Batch Gradient Descent: Tính gradient trên toàn bộ dataset $\mathcal{D} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$:

$$\nabla_{\theta} J(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \ell(f_{\theta}(x^{(i)}), y^{(i)}) \quad (2.5)$$

trong đó ℓ là loss function cho một sample (ví dụ: Mean Squared Error, Cross Entropy).

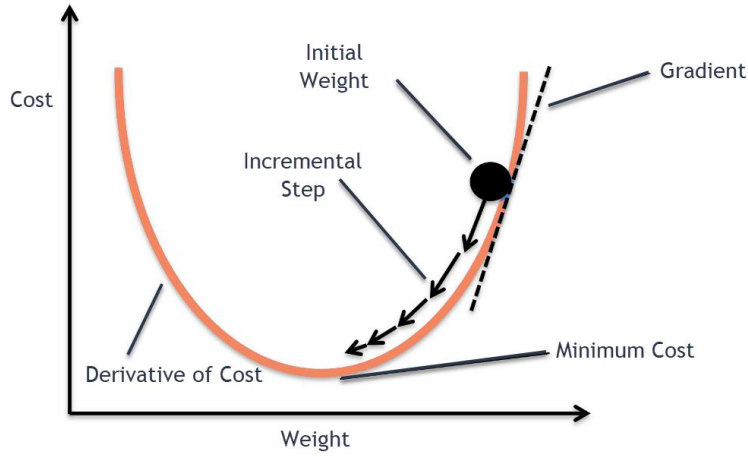
Ưu điểm:

- Cập nhật theo hướng chính xác nhất (true gradient)
- Hội tụ đến cực tiểu cho hàm convex
- Đơn giản, dễ triển khai

Nhược điểm:

- Tính toán gradient trên toàn bộ dataset rất chậm với dữ liệu lớn
- Không thể cập nhật online khi có dữ liệu mới
- Có thể bị mắc kẹt tại local minima hoặc saddle points
- Tốc độ hội tụ phụ thuộc nhiều vào learning rate α

Chọn learning rate: Nếu α quá nhỏ $\rightarrow$ hội tụ chậm. Nếu α quá lớn $\rightarrow$ oscillation hoặc divergence. Phương pháp thử nghiệm: bắt đầu với $\alpha = 0.01$, tăng/giảm theo log scale (0.001, 0.01, 0.1, 1.0) và chọn giá trị cho loss giảm nhanh nhất mà không bị oscillation.



Hình 2.1. Minh họa Gradient Descent: Tham số di chuyển theo hướng ngược gradient để giảm loss

2.5.1.2 Adam Optimizer (Adaptive Moment Estimation)

Adam (Adaptive Moment Estimation) [11] là thuật toán tối ưu hiện đại nhất và được sử dụng rộng rãi nhất trong deep learning và reinforcement learning. Adam kết hợp ưu điểm của hai phương pháp: **momentum** (tích lũy hướng di chuyển) và **adaptive learning rate** (điều chỉnh LR riêng cho từng tham số).

Động lực thiết kế: Các thuật toán gradient descent truyền thống (SGD, Momentum, RMSProp) có những hạn chế:

- SGD: gradient noisy, learning rate global không phù hợp cho tất cả tham số
- Momentum: tăng tốc nhưng vẫn dùng LR global
- RMSProp: adaptive LR nhưng thiếu momentum

Adam giải quyết tất cả các vấn đề trên bằng cách duy trì hai moment estimates:

Công thức Adam:

$$\begin{aligned}
 m_{t+1} &= \beta_1 m_t + (1 - \beta_1) g_t && \text{(first moment: momentum)} \\
 v_{t+1} &= \beta_2 v_t + (1 - \beta_2) g_t^2 && \text{(second moment: adaptive LR)} \\
 \hat{m}_{t+1} &= \frac{m_{t+1}}{1 - \beta_1^{t+1}} && \text{(bias correction cho } m) \\
 \hat{v}_{t+1} &= \frac{v_{t+1}}{1 - \beta_2^{t+1}} && \text{(bias correction cho } v) \\
 \theta_{t+1} &= \theta_t - \alpha \frac{\hat{m}_{t+1}}{\sqrt{\hat{v}_{t+1} + \epsilon}} && \text{(parameter update)}
 \end{aligned} \tag{2.6}$$

trong đó:

- $g_t = \nabla_{\theta} J(\theta_t)$ là gradient tại iteration t
- m_t là exponential moving average của gradient (first moment)
- v_t là exponential moving average của squared gradient (second moment)

- $\beta_1 \in [0, 1)$ là decay rate cho first moment (thường $\beta_1 = 0.9$)
- $\beta_2 \in [0, 1)$ là decay rate cho second moment (thường $\beta_2 = 0.999$)
- α là learning rate (thường $\alpha = 10^{-3}$ hoặc 3×10^{-4} cho RL)
- ϵ là hằng số nhỏ để tránh chia cho 0 (thường $\epsilon = 10^{-8}$)
- Tất cả phép toán trên vector/tensor được thực hiện element-wise

Pseudocode của Adam:

Algorithm 1 Adam Optimizer

Input: Learning rate α , decay rates β_1, β_2 , epsilon ϵ

Input: Khởi tạo parameters θ_0

```

1:  $m_0 \leftarrow 0$  (khởi tạo first moment)
2:  $v_0 \leftarrow 0$  (khởi tạo second moment)
3:  $t \leftarrow 0$  (iteration counter)
4: while  $\theta_t$  chưa hội tụ do
5:    $t \leftarrow t + 1$ 
6:    $g_t \leftarrow \nabla_{\theta} J(\theta_{t-1})$  // Tính gradient
7:    $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$  // Update first moment
8:    $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$  // Update second moment
9:    $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$  // Bias correction first moment
10:   $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$  // Bias correction second moment
11:   $\theta_t \leftarrow \theta_{t-1} - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$  // Update params
12: end while
13: return  $\theta_t$ 

```

Giải thích chi tiết các thành phần:

1. First moment m_t (Momentum):

- m_t là exponentially weighted moving average (EWMA) của gradients
- Với $\beta_1 = 0.9$: $m_t = 0.9m_{t-1} + 0.1g_t$, nghĩa là m_t chứa 90% thông tin từ quá khứ và 10% từ gradient hiện tại
- Công dụng: Tích lũy hướng di chuyển (momentum), giúp tăng tốc trong hướng gradient nhất quán và giảm oscillation khi gradient thay đổi dấu
- Ví dụ: Nếu gradient luôn dương trong 10 iterations, m_t sẽ tích lũy và tăng dần, tạo "quán tính" giúp parameter di chuyển nhanh hơn

2. Second moment v_t (Adaptive Learning Rate):

- v_t là EWMA của squared gradients: $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$
- Với $\beta_2 = 0.999$: v_t có "bộ nhớ" dài hơn m_t (99.9% vs 90%), giúp ước lượng variance ổn định hơn
- Công dụng: Điều chỉnh learning rate riêng cho từng tham số
 - Tham số có v_t lớn (gradient thường lớn) $\rightarrow$ LR giảm (chia cho $\sqrt{v_t}$ lớn) $\rightarrow$ tránh overshoot
 - Tham số có v_t nhỏ (gradient thường nhỏ) $\rightarrow$ LR tăng (chia cho $\sqrt{v_t}$ nhỏ)

→ tăng tốc hội tụ

- Ví dụ: Weight ở layer đầu có gradient lớn, weight ở layer cuối có gradient nhỏ
→ Adam tự động cân bằng tốc độ học

3. Bias Correction:

Do khởi tạo $m_0 = v_0 = 0$, trong các iterations đầu tiên, m_t và v_t bị bias về 0.

Ví dụ cụ thể với $\beta_1 = 0.9$:

- Iteration 1: $m_1 = 0.9 \times 0 + 0.1g_1 = 0.1g_1$ (rất nhỏ so với g_1)
- Iteration 2: $m_2 = 0.9 \times 0.1g_1 + 0.1g_2 = 0.09g_1 + 0.1g_2$
- Expectation: $E[m_t] = E[g](1 - \beta_1^t)$ thay vì $E[g]$

Giải pháp: Chia cho $(1 - \beta_1^t)$ để khử bias:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad (2.7)$$

Với ví dụ trên: $\hat{m}_1 = 0.1g_1 / (1 - 0.9) = 0.1g_1 / 0.1 = g_1$

Khi $t \rightarrow \infty$: $(1 - \beta_1^t) \rightarrow 1$, bias correction không còn ảnh hưởng.

4. Update Rule - Kết hợp tất cả:

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (2.8)$$

Giải thích:

- Tử số $\hat{m}_t$: hướng di chuyển (momentum)
- Mẫu số $\sqrt{\hat{v}_t}$: scaling factor (adaptive LR)
- ϵ : tránh chia cho 0 khi v_t rất nhỏ
- Step size thực tế: $\alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$ khác nhau cho mỗi parameter

Tham số khuyến nghị:

- **Learning rate** α : 10^{-3} cho supervised learning, 3×10^{-4} cho RL/PPO
- β_1 : 0.9 (default cho mọi bài toán)
- β_2 : 0.999 (default cho mọi bài toán)
- ϵ : 10^{-8} (numerical stability)

Ưu điểm của Adam:

- Kết hợp momentum + adaptive LR → hiệu quả cao nhất trong các optimizer
- Hoạt động tốt với default hyperparameters → ít cần tuning
- Hiệu quả với sparse gradients, noisy gradients, và non-stationary objectives
- Computational cost thấp: chỉ cần lưu m_t, v_t (bằng kích thước parameters)
- Phù hợp với high-dimensional parameter spaces (millions of parameters)
- Là default choice cho hầu hết bài toán deep learning và RL

Nhược điểm và cách khắc phục:

- **Vấn đề**: Có thể không hội tụ cho một số bài toán do v_t không decay đủ nhanh

- **Giải pháp - AdamW:** Tách weight decay khỏi gradient update để regularization chính xác hơn
- **Giải pháp - AMSGrad:** Giữ $\max(v_1, \dots, v_t)$ thay vì v_t để đảm bảo LR monotonically giảm

Ứng dụng trong luận văn:

- Adam được sử dụng làm optimizer cho cả Actor và Critic network trong thuật toán PPO
- Learning rate được điều chỉnh động bằng Adaptive LR Scheduler (tăng khi plateau, giữ nguyên khi đang cải thiện)
- Default parameters ($\beta_1 = 0.9$, $\beta_2 = 0.999$) hoạt động tốt, không cần tuning

2.5.2 Reinforcement Learning

Học tăng cường (Reinforcement Learning - RL) là phương pháp học máy trong đó một agent học cách hành động trong môi trường để tối đa hóa tổng reward tích lũy. RL được mô hình hóa bằng Markov Decision Process (MDP), định nghĩa bởi bộ $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$ trong đó:

- $\mathcal{S}$ là tập hợp các trạng thái (states)
- $\mathcal{A}$ là tập hợp các hành động (actions)
- $\mathcal{P}(s'|s, a)$ là xác suất chuyển trạng thái
- $\mathcal{R}(s, a)$ là hàm reward
- $\gamma \in [0, 1]$ là discount factor

Policy $\pi(a|s)$ là phân phối xác suất của hành động cho trước trạng thái. Mục tiêu của RL là tìm policy tối ưu π^* cực đại hóa phần thưởng kỳ vọng $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$.

Value function $V^\pi(s)$ ước lượng kết quả kỳ vọng khi bắt đầu từ trạng thái s và theo policy π :

$$V^\pi(s) = \mathbb{E}_\pi [G_t \mid s_t = s] \quad (2.9)$$

Q-function $Q^\pi(s, a)$ ước lượng kết quả kỳ vọng khi thực hiện hành động a tại trạng thái s rồi theo policy π :

$$Q^\pi(s, a) = \mathbb{E}_\pi [G_t \mid s_t = s, a_t = a] \quad (2.10)$$

Advantage function $A^\pi(s, a)$ đo lường lợi thế của việc chọn hành động a so với policy hiện tại:

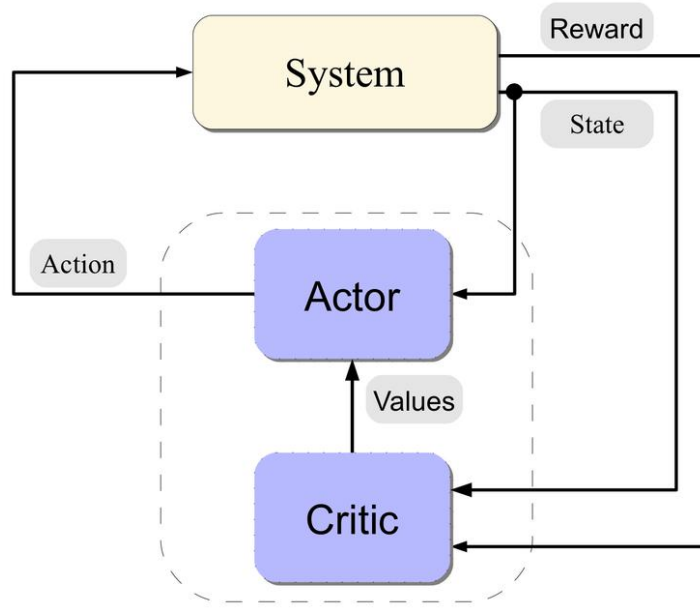
$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s) \quad (2.11)$$

Advantage function giúp giảm variance trong policy gradient. **Generalized Advantage Estimation (GAE)** [12] là phương pháp ước lượng advantage function hiệu quả bằng cách kết hợp nhiều n-step advantage estimates với tham số $\lambda \in [0, 1]$:

$$\hat{A}_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l} \quad (2.12)$$

trong đó $\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$ là temporal difference error.

Kiến trúc **Actor-Critic** kết hợp policy-based và value-based methods. Actor (policy network) $\pi_{\theta}(a|s)$ chọn hành động, Critic (value network) $V_{\phi}(s)$ đánh giá trạng thái. Critic giúp giảm variance cho policy gradient của Actor.



Hình 2.2. Kiến trúc Actor-Critic: Actor quyết định hành động, Critic đánh giá giá trị trạng thái

2.5.3 Multi-Agent Reinforcement Learning (MARL)

Multi-Agent Reinforcement Learning (MARL) mở rộng RL cho hệ thống với nhiều agent tương tác trong cùng môi trường. Trong MARL, mỗi agent i có policy π_i và nhận observation o_i từ môi trường. Thách thức chính của MARL là môi trường trở nên *non-stationary* từ góc nhìn của mỗi agent, vì các agent khác cũng đang học và thay đổi policy.

Phân loại MARL theo mức độ hợp tác:

- **Cooperative:** Các agent có chung mục tiêu, tối đa hóa reward chung. Ví dụ: đội robot cùng hoàn thành nhiệm vụ.

- **Competitive:** Các agent có mục tiêu đối lập (zero-sum game). Ví dụ: game đối kháng.
- **Mixed:** Kết hợp cooperative và competitive. Ví dụ: robot tránh va chạm (cooperative) nhưng cạnh tranh tài nguyên.

Phân loại theo kiến trúc điều khiển:

- **Centralized:** Một controller trung tâm điều khiển tất cả agents. Ưu điểm: tối ưu toàn cục. Nhược điểm: không scale, single point of failure.
- **Decentralized:** Mỗi agent quyết định độc lập dựa trên observation cục bộ. Ưu điểm: scale tốt, robust. Nhược điểm: khó đạt tối ưu toàn cục.
- **Centralized Training with Decentralized Execution (CTDE):** Kết hợp ưu điểm cả hai - huấn luyện tập trung với thông tin toàn cục, thực thi độc lập với observation cục bộ.

2.5.3.1 Centralized Training with Decentralized Execution (CTDE)

CTDE là paradigm phổ biến nhất trong MARL hiện đại, giải quyết trade-off giữa khả năng học hiệu quả và khả năng triển khai thực tế.

Ý tưởng chính:

- **Giai đoạn huấn luyện (Centralized):** Có quyền truy cập thông tin toàn cục - observations, actions, và rewards của tất cả agents. Điều này giúp:
 - Ước lượng chính xác gradient và value function
 - Giải quyết vấn đề credit assignment (phân bổ reward cho từng agent)
 - Học được coordination giữa các agents
- **Giai đoạn thực thi (Decentralized):** Mỗi agent chỉ sử dụng observation cục bộ o_i để quyết định action $a_i = \pi_i(o_i)$. Không cần:
 - Truyền thông giữa các agents
 - Server trung tâm
 - Thông tin về vị trí/hành động của agents khác

Ưu điểm của CTDE cho bài toán robot navigation:

1. **Scalability:** Thêm robot mới không cần retrain, chỉ cần deploy cùng policy.
2. **Robustness:** Hệ thống vẫn hoạt động khi một số robots fail hoặc mất kết nối.
3. **Latency thấp:** Quyết định tại chỗ, không cần round-trip đến server.
4. **Privacy:** Robots không cần chia sẻ vị trí hay trajectory với nhau.

2.5.4 Proximal Policy Optimization (PPO)

Thuật toán PPO [13] là một trong những thuật toán policy gradient hiệu quả và ổn định nhất hiện nay. PPO giải quyết vấn đề của các thuật toán policy gradient truyền thống là độ nhạy cảm với learning rate và khả năng policy thay đổi quá nhanh gây mất ổn định. Trong phần này, chúng ta sẽ phân tích chi tiết từ policy gradient cơ bản đến PPO.

2.5.4.1 Policy Gradient cơ bản

Policy gradient methods tối ưu trực tiếp policy $\pi_\theta(a|s)$ bằng cách maximize kết quả kỳ vọng $J(\theta)$:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right] \quad (2.13)$$

trong đó $\tau = (s_0, a_0, r_0, s_1, \dots)$ là trajectory được thu thập bằng policy π_θ .

Policy Gradient Theorem: Gradient của objective function là:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t \right] \quad (2.14)$$

trong đó $G_t = \sum_{k=t}^T \gamma^{k-t} r_k$ là return từ timestep t .

Giảm variance với baseline: Sử dụng value function $V^\pi(s_t)$ làm baseline:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot (G_t - V^\pi(s_t)) \right] \quad (2.15)$$

Advantage function $A^\pi(s_t, a_t) = G_t - V^\pi(s_t)$ giúp giảm variance mà không làm tăng bias.

Vấn đề của vanilla policy gradient:

- Kích thước bước cập nhật khó kiểm soát - policy có thể thay đổi quá nhanh và mất performance
- Sample inefficient - cần nhiều dữ liệu mới hội tụ
- Không ổn định - performance có thể sụt giảm đột ngột
- Khó điều chỉnh learning rate - quá nhỏ $\rightarrow$ chậm, quá lớn $\rightarrow$ collapse

2.5.4.2 Proximal Policy Optimization (PPO)

PPO giải quyết vấn đề learning rate của Policy Gradient bằng cách sử dụng clipped objective function. Ý tưởng chính: thay vì dùng learning rate cố định, PPO giới hạn mức độ thay đổi policy giữa các lần update thông qua clipping mechanism:

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)] \quad (2.16)$$

trong đó:

- $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ là probability ratio
- $\text{clip}(r, 1 - \epsilon, 1 + \epsilon)$ giới hạn r trong $[1 - \epsilon, 1 + \epsilon]$
- ϵ là clip ratio (typically 0.1-0.2)

Phân tích clipping mechanism:

Trường hợp 1: Advantage dương ($\hat{A}_t > 0$)

Hành động a_t tốt hơn trung bình $\rightarrow$ muốn tăng $\pi_\theta(a_t|s_t)$.

- Nếu $r_t < 1 + \epsilon$: Objective = $r_t\hat{A}_t$ (không clip) $\rightarrow$ gradient khuyến khích tăng r_t
- Nếu $r_t \geq 1 + \epsilon$: Objective = $(1 + \epsilon)\hat{A}_t$ (clipped) $\rightarrow$ gradient = 0, không tăng thêm

Trường hợp 2: Advantage âm ($\hat{A}_t < 0$)

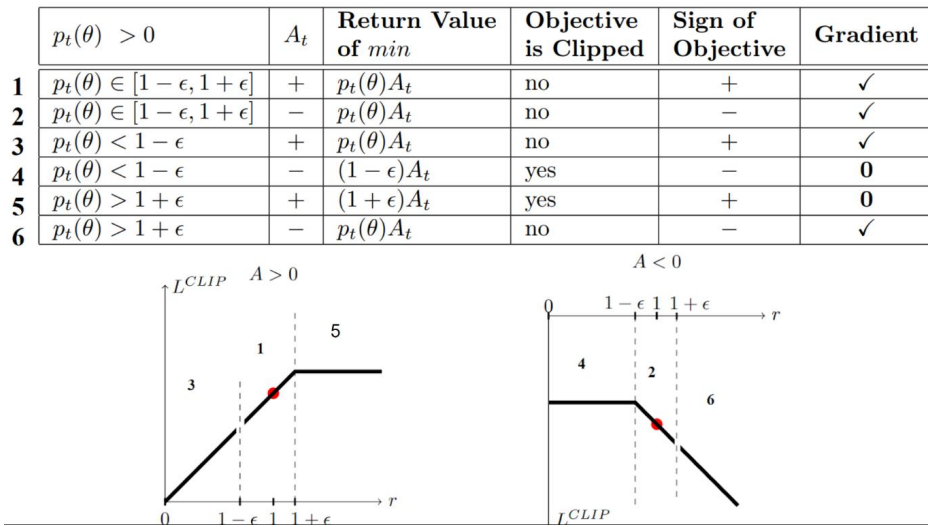
Hành động a_t tệ hơn trung bình $\rightarrow$ muốn giảm $\pi_\theta(a_t|s_t)$.

- Nếu $r_t > 1 - \epsilon$: Objective = $r_t\hat{A}_t$ (không clip) $\rightarrow$ gradient khuyến khích giảm r_t
- Nếu $r_t \leq 1 - \epsilon$: Objective = $(1 - \epsilon)\hat{A}_t$ (clipped) $\rightarrow$ gradient = 0, không giảm thêm

Ý nghĩa: Clipping ngăn không cho policy thay đổi quá nhiều trong một update:

- Nếu action tốt ($A > 0$), cho phép tăng xác suất nhưng không quá $1 + \epsilon$ lần
- Nếu action xấu ($A < 0$), cho phép giảm xác suất nhưng không dưới $1 - \epsilon$ lần

Thực quan hóa: Hàm mục tiêu có dạng phẳng (plateau) khi tỷ số xác suất nằm ngoài vùng tin cậy $[1 - \epsilon, 1 + \epsilon]$, tạo ra ràng buộc tự nhiên mà không cần giải bài toán tối ưu có ràng buộc tường minh.



Hình 2.3. PPO Clipping: Objective bị chặn ngoài khoảng $[1 - \epsilon, 1 + \epsilon]$ để ngăn policy thay đổi quá nhanh

2.5.4.3 Value Function Clipping

Ngoài clipping cho policy, PPO còn có thể clip value function loss để tránh value estimates thay đổi quá nhanh:

$$L^{VF}(\theta) = \mathbb{E}_t \left[\max \left((V_\theta(s_t) - V_t^{targ})^2, (\text{clip}(V_\theta(s_t), V_{old} - \epsilon_V, V_{old} + \epsilon_V) - V_t^{targ})^2 \right) \right] \quad (2.17)$$

trong đó:

- V_t^{targ} là target value (ví dụ: GAE estimate)
- V_{old} là value estimate từ policy cũ
- ϵ_V là clip range cho value function (thường 0.1-0.2)

Value clipping giúp ổn định training, đặc biệt khi:

- Value function được khởi tạo kém
- Reward scale thay đổi nhiều giữa các episodes
- Shared parameters giữa policy và value networks

2.5.4.4 Complete PPO Objective

Objective function đầy đủ của PPO kết hợp policy loss, value loss, và entropy bonus:

$$L^{TOTAL}(\theta) = \mathbb{E}_t \left[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t) \right] \quad (2.18)$$

trong đó:

- L_t^{CLIP} : Clipped policy loss (maximize)
- L_t^{VF} : Value function loss (cực tiểu)
- $S[\pi_\theta]$: Entropy của policy (maximize để khuyến khích exploration)
- c_1, c_2 : Coefficients (thường $c_1 = 0.5 - 1.0$, $c_2 = 0.01$)

Entropy bonus: Entropy đo độ "random" của policy:

$$S[\pi_\theta](s) = -\mathbb{E}_{a \sim \pi_\theta(\cdot|s)}[\log \pi_\theta(a|s)] \quad (2.19)$$

- Entropy cao $\rightarrow$ policy uniform (exploration nhiều)
- Entropy thấp $\rightarrow$ policy deterministic (exploitation)
- Entropy bonus khuyến khích exploration ở giai đoạn đầu
- Thường giảm dần entropy coefficient theo thời gian

2.5.4.5 PPO-Penalty: Alternative Formulation

Thay vì clipping, PPO-Penalty thêm KL penalty trực tiếp vào objective:

$$L^{PENALTY}(\theta) = \mathbb{E}_t \left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \hat{A}_t - \beta \cdot \text{KL}[\pi_{\theta_{old}}(\cdot|s_t) \parallel \pi_\theta(\cdot|s_t)] \right] \quad (2.20)$$

Adaptive β : PPO-Penalty điều chỉnh penalty coefficient β theo KL divergence thực tế:

- Nếu $\text{KL} < \text{KL}_{target}/1.5$: giảm β (cho phép update lớn hơn)
- Nếu $\text{KL} > \text{KL}_{target} \times 1.5$: tăng β (hạn chế update)
- Thường khởi tạo $\beta = 1.0$, $\text{KL}_{target} = 0.01$

So sánh PPO-Clip vs PPO-Penalty:

Kết luận: PPO-Clip được ưa chuộng hơn do đơn giản, robust, và performance tốt với default hyperparameters.

Bảng 2.2. So sánh PPO-Clip và PPO-Penalty

Tiêu chí	PPO-Clip	PPO-Penalty
Mechanism	Hard constraint bằng clipping	Soft constraint bằng penalty
Hyperparameters	ϵ cố định (0.1-0.2)	β adaptive, KL_{target}
Implementation	Đơn giản hơn	Cần tính KL divergence
Performance	Tốt hơn trong hầu hết trường hợp	Tương đương nhưng ít phổ biến
Robustness	Robust với nhiều loại môi trường	Cần tuning KL_{target}

2.5.4.6 Lý do chọn PPO cho bài toán đa robot

Trong bài toán điều khiển đa robot tránh va chạm, PPO được chọn thay vì các thuật toán RL khác vì những lý do sau:

So sánh với các thuật toán RL phổ biến:

Lý do PPO phù hợp với bài toán này:

1. **Continuous action space:** Robot cần điều khiển vận tốc tuyến tính $v \in [0, 0.3]$ m/s và vận tốc góc $\omega \in [-1, 1]$ rad/s - là continuous actions. DQN chỉ hỗ trợ discrete actions, còn PPO native support continuous actions thông qua Gaussian policy.
2. **Ổn định trong huấn luyện:** Với 24-44 robots chạy song song, gradient variance rất cao. PPO với clipped objective ngăn policy thay đổi quá nhanh, tránh collapse. DDPG và A3C thường gặp instability trong môi trường multi-agent.
3. **Parallel data collection:** Bài toán có sẵn 24-44 robots song song trong simulator. PPO tận dụng tốt điều này - mỗi robot là một “worker” thu thập experience đồng thời, tăng tốc huấn luyện mà không cần replay buffer phức tạp như off-policy methods.
4. **Đơn giản trong implementation:** So với SAC (cần 2 Q-networks + entropy tuning) hay DDPG (cần target networks + replay buffer), PPO chỉ cần Actor-Critic đơn giản với clipped loss. Điều này giúp dễ debug và tune hyperparameters.
5. **Robust với hyperparameters:** PPO hoạt động tốt với default hyperparameters ($\epsilon = 0.2, \gamma = 0.99$). Các thuật toán khác như DDPG cực kỳ sensitive với learning rate và noise parameters.

Hạn chế: PPO là on-policy algorithm nên sample efficiency thấp hơn các phương pháp off-policy (SAC, DDPG). Tuy nhiên, với simulator có thể chạy nhanh và song song (24-44 robots), điều này không phải vấn đề lớn. Ổn định và đơn giản quan trọng và an toàn hơn trong bài toán này.

Bảng 2.3. So sánh PPO với các thuật toán RL khác cho bài toán robot navigation

Thuật toán	Ưu điểm	Nhược điểm	Phù hợp?
Q-Learning	Đơn giản, hội tụ đảm bảo	Chỉ discrete state/action, không scale với state space lớn (LiDAR 1362 chiều)	Không
DQN	Xử lý state lớn nhờ neural network	Chỉ discrete actions	Không (robot cần continuous control)
DDPG	Continuous actions, off-policy	Không ổn định, sensitive với hyperparameters	Có thể nhưng khó tune
A3C	Parallel training, on-policy	Asynchronous gây inconsistency, khó debug	Có thể nhưng phức tạp
SAC	Sample efficient, automatic entropy tuning	Cần replay buffer lớn, phức tạp	Có thể nhưng overhead cao
PPO	Ổn định, đơn giản, parallel training	On-policy (sample inefficient)	Rất phù hợp

2.5.4.7 PPO Hyperparameters

Các hyperparameters quan trọng của PPO:

- **Clip ratio (ϵ):** Giới hạn mức độ thay đổi policy (thường 0.1-0.2)
- **Number of epochs (K):** Số lần update policy trên mỗi batch data (3-10)
- **Batch size:** Số timesteps thu thập trước update (2048-4096)
- **GAE lambda (λ):** Cân bằng bias-variance (0.90-0.99)
- **Discount factor (γ):** Trọng số future rewards (0.99 cho tác vụ long-horizon)
- **Learning rates:** Thường khác nhau cho Actor và Critic. Trong luận văn này: Critic LR = 6×10^{-3} , Actor LR = 4×10^{-4} (Stage 1)
- **Entropy coefficient (c_2):** Khuyến khích exploration (0.01, trong luận văn này: 8×10^{-3} decay đến 2×10^{-3})
- **Target KL:** Early stopping threshold (thường 0.015, trong luận văn này: 0.035 cho aggressive updates)

2.5.5 Kiến trúc mạng nơ-ron

Mạng nơ-ron nhân tạo (Artificial Neural Networks - ANN) là mô hình tính toán thiết kế mô phỏng cấu trúc và cách não bộ con người hoạt động. Trong phần này, chúng ta sẽ phân tích chi tiết về kiến trúc, quá trình huấn luyện, và các kỹ thuật tối ưu hóa mạng nơ-ron.

2.5.5.1 Fully Connected Neural Networks

Kiến trúc cơ bản: Mạng nơ-ron fully connected (dense) bao gồm nhiều lớp (layers), mỗi lớp có nhiều nơ-ron (nodes). Mỗi neuron trong lớp l kết nối với tất cả nơ-ron trong lớp $l + 1$.

Forward propagation: Tính toán output từ input qua các lớp:

Lớp l :

$$\begin{aligned}\mathbf{z}^{[l]} &= \mathbf{W}^{[l]} \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]} \\ \mathbf{a}^{[l]} &= g^{[l]}(\mathbf{z}^{[l]})\end{aligned}\tag{2.21}$$

trong đó:

- $\mathbf{a}^{[l-1]}$: Activations từ lớp trước (input nếu $l = 1$)
- $\mathbf{W}^{[l]}$: Weight matrix kích thước $(n^{[l]}, n^{[l-1]})$
- $\mathbf{b}^{[l]}$: Bias vector kích thước $(n^{[l]}, 1)$
- $\mathbf{z}^{[l]}$: Pre-activation (linear combination)
- $g^{[l]}$: Activation function
- $\mathbf{a}^{[l]}$: Activations (output của lớp l)

Activation Functions: Thêm tính phi tuyến vào mạng

- **ReLU (Rectified Linear Unit):** $\text{ReLU}(x) = \max(0, x)$, derivative:

$$\text{ReLU}'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

Phổ biến nhất do tính toán nhanh và không bị triệt tiêu gradient. Nhược điểm: hiện tượng "ReLU chết" khi nơ-ron có $z < 0$ không được cập nhật.

- **Sigmoid và Tanh:** $\sigma(x) = \frac{1}{1+e^{-x}}$ và $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$. Sigmoid có đầu ra trong $(0, 1)$, dùng cho phân loại nhị phân và chuẩn hóa vận tốc trong học tăng cường. Tanh có đầu ra trong $(-1, 1)$, có tâm tại 0, phù hợp cho điều khiển đối xứng (ví dụ: vận tốc góc). Trong luận văn này: Sigmoid cho vận tốc tuyến tính, Tanh cho vận tốc góc.

2.5.5.2 Convolutional Neural Networks (CNN)

Convolutional Neural Networks [5] được thiết kế để xử lý dữ liệu có cấu trúc lưới (grid-like structure) như hình ảnh (2D) hoặc time series/LiDAR (1D).

Convolution operation: Áp dụng filter (kernel) trên input:

1D convolution (cho LiDAR):

$$(f * g)(n) = \sum_{m=-\infty}^{\infty} f(m) \cdot g(n - m) \quad (2.22)$$

Trong neural network, discrete convolution:

$$y[n] = \sum_{k=0}^{K-1} w[k] \cdot x[n + k] + b \quad (2.23)$$

trong đó:

- x : Input sequence (ví dụ: LiDAR 360 points)
- w : Filter weights
- K : Kernel size (ví dụ: 5)
- b : Bias
- y : Output feature map

Conv1D parameters:

- **Number of filters:** Số lượng patterns muốn học (16-128)
- **Kernel size:** Kích thước filter, quyết định receptive field (3-7)
- **Stride:** Khoảng cách giữa các vị trí áp dụng filter. Stride=2 giảm kích thước đầu ra
- **Padding:** Valid (không padding) hoặc Same (giữ output length = input length)

Kích thước đầu ra được tính:

$$\text{Output length} = \frac{\text{Input length} - \text{Kernel size} + 2 \times \text{Padding}}{\text{Stride}} + 1 \quad (2.24)$$

Ví dụ: Xử lý LiDAR trong luận văn này

Đầu vào: LiDAR 512 điểm $\times$ 3 frames = 1536 điểm

$$\text{Conv1D}(32, k=5, s=2) \rightarrow \text{ReLU} \rightarrow \text{Conv1D}(32, k=3, s=2) \rightarrow \text{ReLU} \quad (2.25)$$

Đặc trưng đầu ra được flatten và kết hợp với vị trí goal + vận tốc.

Ưu điểm của CNN:

- Chia sẻ tham số: Cùng bộ lọc áp dụng khắp đầu vào, giảm số lượng tham số
- Bất biến với dịch chuyển: Phát hiện mẫu ở mọi vị trí
- Đặc trưng phân cấp: Các lớp sâu học các mẫu trừu tượng từ đặc trưng cấp thấp
- Hiệu quả với dữ liệu không gian và thời gian

Lý do chọn CNN (Conv1D) cho xử lý LiDAR:

Dữ liệu LiDAR 360° có đặc điểm đặc biệt phù hợp với CNN:

- *Tính liên tục không gian:* Các tia laser liên kế thường có giá trị tương tự (cùng quét một vật thể), tạo thành patterns cục bộ mà CNN có thể học hiệu quả.
- *Bất biến vị trí:* Vật cản có thể xuất hiện ở bất kỳ góc nào trong scan. CNN với weight sharing phát hiện cùng một pattern (ví dụ: góc tường, cạnh vật cản) ở mọi vị trí trong dữ liệu.
- *Giảm chiều hiệu quả:* LiDAR 454 tia $\times$ 3 frames = 1362 chiều. Conv1D với stride=2 nén xuống còn 100 features trong khi giữ thông tin quan trọng.
- *So sánh với Fully Connected:* FC layer cần $454 \times 3 \times 256 \approx 350K$ parameters, trong khi Conv1D chỉ cần $\sim 5K$ parameters với hiệu quả tương đương.

Các phương pháp thay thế và lý do không chọn:

- *Fully Connected (MLP):* Không khai thác được cấu trúc không gian của LiDAR, cần nhiều parameters hơn, dễ overfit.
- *Recurrent (LSTM/GRU):* Phù hợp cho chuỗi thời gian tuần tự, nhưng LiDAR scan là dữ liệu không gian vòng tròn, không có thứ tự tự nhiên.
- *Transformer:* Hiệu quả nhưng computational cost cao, không cần thiết cho bài toán với input size nhỏ như LiDAR.

2.5.5.3 Loss Functions

hàm loss đóng vai trò đo lường sự khác biệt giữa dự đoán của mạng và giá trị thực tế. Mục tiêu của quá trình huấn luyện là tìm tập tham số làm cực tiểu hóa hàm loss này.

Mean Squared Error (MSE): Thường dùng cho bài toán regression:

$$L_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (2.26)$$

trong đó y_i là giá trị thực, $\hat{y}_i$ là giá trị dự đoán. MSE phạt lỗi lớn mạnh hơn do bình phương. Trong học tăng cường, MSE huấn luyện hàm giá trị:

$$L_V(\phi) = \mathbb{E}[(V_\phi(s) - V_{target})^2] \quad (2.27)$$

Cross-Entropy Loss: Dùng cho classification. Binary Cross-Entropy:

$$L_{BCE} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (2.28)$$

Categorical Cross-Entropy cho multi-class:

$$L_{CCE} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{i,c} \log(\hat{y}_{i,c}) \quad (2.29)$$

Cross-entropy đo "khoảng cách" giữa phân phối thực và phân phối dự đoán.

2.5.5.4 Backpropagation - Lan truyền ngược

Lan truyền ngược là thuật toán cốt lõi để huấn luyện mạng nơ-ron. Nhiệm vụ của nó là tính gradient của hàm loss đối với tất cả tham số (trọng số và bias) trong mạng, để sau đó có thể cập nhật tham số theo hướng giảm hàm loss. Thuật toán sử dụng quy tắc chuỗi của giải tích và hoạt động theo hai phase:

Lan truyền xuôi: Dữ liệu đi từ lớp đầu vào qua các lớp ẩn đến lớp đầu ra. Tại mỗi lớp l , ta tính:

$$\mathbf{z}^{[l]} = \mathbf{W}^{[l]} \mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}, \quad \mathbf{a}^{[l]} = g^{[l]}(\mathbf{z}^{[l]}) \quad \text{for } l = 1, \dots, L \quad (2.30)$$

trong đó $\mathbf{z}^{[l]}$ là giá trị trước kích hoạt (tổng có trọng số), $\mathbf{a}^{[l]}$ là giá trị kích hoạt (đầu ra sau hàm kích hoạt $g^{[l]}$). Cuối cùng tính hàm loss L bằng cách so sánh đầu ra $\mathbf{a}^{[L]}$ với nhãn thực. Pha này cho ta biết mạng đang dự đoán như thế nào và hàm loss hiện tại là bao nhiêu.

Lan truyền ngược: Đây là phần quan trọng - ta tính gradient của hàm loss theo từng tham số bằng cách lan truyền ngược từ đầu ra về đầu vào. Định nghĩa $\delta^{[l]} = \frac{\partial L}{\partial \mathbf{z}^{[l]}}$ là gradient của hàm loss theo giá trị trước kích hoạt tại lớp l .

Tại lớp đầu ra L , gradient được tính trực tiếp:

$$\delta^{[L]} = \nabla_{\mathbf{a}^{[L]}} L \odot g'^{[L]}(\mathbf{z}^{[L]}) \quad (2.31)$$

trong đó $\odot$ là phép nhân từng phần tử, $g'^{[L]}$ là đạo hàm của hàm kích hoạt.

Lan truyền ngược qua các lớp $l = L - 1, L - 2, \dots, 1$:

$$\delta^{[l]} = (\mathbf{W}^{[l+1]})^T \delta^{[l+1]} \odot g'^{[l]}(\mathbf{z}^{[l]}) \quad (2.32)$$

Công thức này thể hiện quy tắc chuỗi: gradient tại lớp l phụ thuộc vào (1) gradient từ lớp sau $\delta^{[l+1]}$ truyền về qua trọng số $\mathbf{W}^{[l+1]}$, và (2) đạo hàm của hàm kích hoạt tại lớp đó $g'^{[l]}$.

Sau khi có $\delta^{[l]}$ cho tất cả các lớp, ta tính gradient của hàm loss theo trọng số và bias:

$$\frac{\partial L}{\partial \mathbf{W}^{[l]}} = \delta^{[l]} (\mathbf{a}^{[l-1]})^T, \quad \frac{\partial L}{\partial \mathbf{b}^{[l]}} = \delta^{[l]} \quad (2.33)$$

Gradient của trọng số phụ thuộc vào giá trị kích hoạt từ lớp trước $\mathbf{a}^{[l-1]}$ - đây là lý do tại sao lan truyền ngược cần lưu lại giá trị kích hoạt từ pha lan truyền xuôi.

Vấn đề vanishing và exploding gradient: Khi mạng có nhiều lớp, gradient có thể trở nên cực nhỏ (vanishing) hoặc cực lớn (exploding) khi nhân qua nhiều lớp. Triệt tiêu gradient xảy ra với sigmoid/tanh vì đạo hàm của chúng nhỏ hơn 1 ($\sigma'(x) \leq 0.25$, $\tanh'(x) \leq 1$), nhân nhiều lần sẽ tiến về 0 - khiến các lớp đầu không học được. Exploding gradient xảy ra khi trọng số quá lớn hoặc tốc độ học không phù hợp. Các giải pháp phổ biến: dùng hàm kích hoạt ReLU (không bị triệt tiêu vì $\text{ReLU}'(x) = 1$ khi $x > 0$), cắt gradient (chặn gradient lớn), chuẩn hóa batch (ổn định giá trị kích hoạt), và khởi tạo trọng số phù hợp.

2.5.5.5 Kỹ thuật Regularization

Regularization là các kỹ thuật giúp mạng nơ-ron tránh overfitting (học quá sát với dữ liệu huấn luyện) và tổng quát hóa tốt hơn cho dữ liệu chưa thấy.

Regularization L2 (suy giảm trọng số): Thêm một số hạng phạt vào hàm loss, tỷ lệ với tổng bình phương của tất cả trọng số:

$$L_{total} = L_{data} + \lambda \sum_l \|\mathbf{W}^{[l]}\|_2^2 \quad (2.34)$$

trong đó λ là cường độ điều chuẩn. Số hạng phạt này làm cho mạng "không thích" trọng

số lớn - khuyến khích trọng số nhỏ hơn, mượt hơn. Khi cập nhật trọng số:

$$\mathbf{W} \leftarrow \mathbf{W} - \alpha \left(\frac{\partial L_{data}}{\partial \mathbf{W}} + 2\lambda \mathbf{W} \right) = (1 - 2\alpha\lambda) \mathbf{W} - \alpha \frac{\partial L_{data}}{\partial \mathbf{W}} \quad (2.35)$$

mỗi trọng số sẽ bị suy giảm một chút về phía 0 (nhân với số nhỏ hơn 1). Mô hình với trọng số nhỏ thường đơn giản hơn, ít overfitting hơn.

Dropout: Trong quá trình huấn luyện, ngẫu nhiên ”tắt”(đặt đầu ra = 0) một số nơ-ron với xác suất p (thường 0.5):

$$\mathbf{a}^{[l]} = \mathbf{m}^{[l]} \odot g^{[l]}(\mathbf{z}^{[l]}), \quad m_i^{[l]} \sim \text{Bernoulli}(1 - p) \quad (2.36)$$

Mỗi vòng lặp huấn luyện, mạng dùng một ”mạng con”(subnetwork) khác nhau. Điều này ngăn các nơ-ron phụ thuộc lẫn nhau quá mức - buộc mỗi nơ-ron học các đặc trưng hữu ích một cách độc lập. Khi kiểm thử, dùng tất cả nơ-ron nhưng thu nhỏ đầu ra để phù hợp với kỳ vọng trong huấn luyện.

Batch Normalization: Chuẩn hóa giá trị kích hoạt của mỗi lớp về trung bình = 0 và phương sai = 1, tính trên từng mini-batch:

$$\hat{\mathbf{z}} = \frac{\mathbf{z} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad \mathbf{z}_{BN} = \gamma \hat{\mathbf{z}} + \beta \quad (2.37)$$

trong đó μ_B, σ_B^2 là trung bình và phương sai của batch, γ, β là các tham số học được cho phép mạng tự điều chỉnh mức độ chuẩn hóa phù hợp với từng lớp. Lợi ích chính là ổn định quá trình huấn luyện - cho phép dùng tốc độ học lớn hơn, giảm độ nhạy với việc khởi tạo trọng số.

2.5.6 Điều khiển PID

Bộ điều khiển PID (Proportional-Integral-Derivative) [14] là phương pháp điều khiển cổ điển được sử dụng rộng rãi trong robot thực tế. Đối với differential drive robot, PID được áp dụng để điều khiển vận tốc tuyến tính và vận tốc góc.

Công thức PID cơ bản:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (2.38)$$

trong đó:

- $e(t) = r(t) - y(t)$ là sai số giữa setpoint $r(t)$ và output $y(t)$
- K_p là hệ số P (Proportional): phản ứng tỷ lệ với sai số hiện tại

- K_i là hệ số I (Integral): khử sai số tích lũy theo thời gian (steady-state error)
- K_d là hệ số D (Derivative): dự đoán xu hướng thay đổi, giảm overshoot

Ứng dụng cho differential drive robot [15]:

PID cho vận tốc thẳng: Điều khiển vận tốc bánh xe để đạt vận tốc thẳng mong muốn v_{des} :

$$v_{cmd} = K_{p,v}e_v + K_{i,v} \int e_v dt + K_{d,v} \frac{de_v}{dt} \quad (2.39)$$

với $e_v = v_{des} - v_{current}$.

PID cho vận tốc góc: Điều khiển sự chênh lệch vận tốc giữa hai bánh để đạt vận tốc góc ω_{des} :

$$\omega_{cmd} = K_{p,\omega}e_\omega + K_{i,\omega} \int e_\omega dt + K_{d,\omega} \frac{de_\omega}{dt} \quad (2.40)$$

với $e_\omega = \omega_{des} - \omega_{current}$.

Tuning tham số PID: Có nhiều phương pháp tuning như Ziegler-Nichols, Cohen-Coon, hoặc "trial-and-error". Nguyên tắc chung:

- Tăng K_p : Phản ứng nhanh hơn nhưng có thể overshoot và oscillation
- Tăng K_i : Khử steady-state error nhưng có thể gây chậm và overshoot
- Tăng K_d : Giảm overshoot và oscillation nhưng nhạy cảm với nhiễu

Ưu điểm: Đơn giản, không cần mô hình chính xác, dễ triển khai, ổn định với hệ tuyến tính.

Hạn chế: Khó tuning cho hệ phi tuyến, không tối ưu cho hệ đa biến, nhạy cảm với nhiễu (thành phần D), không thích nghi với thay đổi môi trường.

2.5.7 UKF Sensor Fusion

Unscented Kalman Filter (UKF) [16] là phương pháp ước lượng trạng thái cho hệ thống phi tuyến. Khác với Extended Kalman Filter (EKF) phải xấp xỉ tuyến tính hóa và tính ma trận Jacobian, UKF sử dụng tập hợp các điểm đại diện (sigma points) để truyền qua hàm phi tuyến trực tiếp, cho kết quả chính xác hơn với cách triển khai đơn giản hơn.

Bài toán sensor fusion: Robot sử dụng nhiều cảm biến để ước lượng pose (x, y, θ) :

- **Wheel odometry:** Đo vận tốc bánh xe, tích phân ra vị trí. Ưu điểm: update rate cao (50-100 Hz). Nhược điểm: drift tích lũy, trượt bánh xe gây sai số.
- **IMU (Inertial Measurement Unit):** Đo gia tốc (accelerometer) và vận tốc góc (gyroscope). Ưu điểm: không bị trượt bánh. Nhược điểm: drift theo thời gian, cần calibration.

Mô hình không gian trạng thái:

$$\mathbf{x} = [x, y, \theta, v_x, v_y, \omega]^T \quad (2.41)$$

Mô hình thực hiện (motion model):

$$\mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k) + \mathbf{w}_k \quad (2.42)$$

với $\mathbf{u}_k$ là tín hiệu vào (wheel velocities) và $\mathbf{w}_k \sim \mathcal{N}(0, \mathbf{Q})$ là nhiễu quá trình.

Mô hình tính toán:

Wheel odometry:

$$\mathbf{z}_{odom} = \begin{bmatrix} v_x \\ v_y \\ \omega \end{bmatrix} + \mathbf{v}_{odom} \quad (2.43)$$

IMU:

$$\mathbf{z}_{imu} = \begin{bmatrix} a_x \\ a_y \\ \omega \end{bmatrix} + \mathbf{v}_{imu} \quad (2.44)$$

với $\mathbf{v} \sim \mathcal{N}(0, \mathbf{R})$ là nhiễu đo lường.

Thuật toán UKF:

1. **Unscented Transform:** Chọn $2n + 1$ sigma points $\mathcal{X}^{(i)}$ xung quanh trung bình $\mathbf{x}$ với weights $W^{(i)}$:

$$\begin{aligned} \mathcal{X}^{(0)} &= \mathbf{x} \\ \mathcal{X}^{(i)} &= \mathbf{x} + \left(\sqrt{(n + \lambda) \mathbf{P}} \right)_i, \quad i = 1, \dots, n \\ \mathcal{X}^{(i)} &= \mathbf{x} - \left(\sqrt{(n + \lambda) \mathbf{P}} \right)_{i-n}, \quad i = n + 1, \dots, 2n \end{aligned} \quad (2.45)$$

2. **Prediction:** Truyền sigma points qua process model:

$$\mathcal{X}_{k+1|k}^{(i)} = f(\mathcal{X}_k^{(i)}, \mathbf{u}_k) \quad (2.46)$$

Tính giá trị dự đoán mean và covariance:

$$\begin{aligned} \mathbf{x}_{k+1|k} &= \sum_{i=0}^{2n} W^{(i)} \mathcal{X}_{k+1|k}^{(i)} \\ \mathbf{P}_{k+1|k} &= \sum_{i=0}^{2n} W^{(i)} (\mathcal{X}_{k+1|k}^{(i)} - \mathbf{x}_{k+1|k})(\mathcal{X}_{k+1|k}^{(i)} - \mathbf{x}_{k+1|k})^T + \mathbf{Q} \end{aligned} \quad (2.47)$$

3. **Update:** Khi có $\mathbf{z}_k$, tính Kalman gain và update:

$$\begin{aligned}
 \mathcal{Y}_k^{(i)} &= h(\mathcal{X}_{k|k-1}^{(i)}) \\
 \mathbf{y}_k &= \sum_{i=0}^{2n} W^{(i)} \mathcal{Y}_k^{(i)} \\
 \mathbf{K}_k &= \mathbf{P}_{xy} \mathbf{S}_k^{-1} \\
 \mathbf{x}_k &= \mathbf{x}_{k|k-1} + \mathbf{K}_k (\mathbf{z}_k - \mathbf{y}_k) \\
 \mathbf{P}_k &= \mathbf{P}_{k|k-1} - \mathbf{K}_k \mathbf{S}_k \mathbf{K}_k^T
 \end{aligned} \tag{2.48}$$

Điều chỉnh noise covariances:

- **Q** (process noise): Phản ánh độ tin cậy của model. Lớn hơn $\rightarrow$ tin measurement hơn.
- **R** (measurement noise): Phản ánh độ chính xác sensor. Nhỏ hơn $\rightarrow$ tin measurement hơn.

So sánh UKF vs EKF:

Bảng 2.4. So sánh UKF và EKF

Tiêu chí	EKF	UKF
Linearization	Cần tính Jacobian, chỉ chính xác bậc 1	Unscented transform, chính xác bậc 2-3
Độ chính xác	Kém với hệ phi tuyến mạnh	Tốt hơn EKF
Tính toán	Nhanh hơn (ít sigma points)	Chậm hơn (nhiều sigma points)
Triển khai	Cần tính Jacobian (phức tạp)	Không cần Jacobian (đơn giản hơn)

2.5.8 Cartographer và SLAM

2.5.8.1 Bài toán SLAM

SLAM (Simultaneous Localization and Mapping) [17] là bài toán then chốt trong robot tự hành, yêu cầu robot đồng thời xác định vị trí của chính nó và xây dựng bản đồ môi trường chưa biết. Về mặt toán học, bài toán SLAM được biểu diễn như tìm kiếm phân phối xác suất:

$$p(\mathbf{x}_{1:t}, \mathbf{m} \mid \mathbf{z}_{1:t}, \mathbf{u}_{1:t}) \tag{2.49}$$

trong đó:

- $\mathbf{x}_{1:t} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t\}$: chuỗi pose (vị trí và hướng) của robot qua thời gian
- $\mathbf{m}$: bản đồ môi trường
- $\mathbf{z}_{1:t}$: dữ liệu cảm biến (laser scans, camera)
- $\mathbf{u}_{1:t}$: dữ liệu điều khiển (odometry)

2.5.8.2 Google Cartographer

Cartographer [18] là thư viện SLAM mã nguồn mở được Google phát triển, tối ưu cho việc chạy real-time trên nền tảng nhúng. Cartographer sử dụng phương pháp **graph-based SLAM** với kiến trúc submap, khác biệt so với các phương pháp truyền thống:

- **GMapping**: dựa trên particle filter, tốn nhiều tài nguyên tính toán khi môi trường lớn
- **Cartographer**: dựa trên pose graph optimization và submap-based matching, mở rộng tốt hơn, phát hiện loop closure hiệu quả, phù hợp với phần cứng giới hạn (Jetson Nano)

2.5.8.3 Nguyên lý hoạt động

Cartographer hoạt động dựa trên nguyên lý **chia nhỏ và tối ưu hóa từng phần**:

1. **Phân chia không gian thành submaps**: Thay vì xây dựng một bản đồ toàn cục lớn, môi trường được chia thành nhiều submaps nhỏ chồng lấp nhau
2. **Local optimization**: Trong mỗi submap, laser scans được căn chỉnh (scan matching) để tạo bản đồ cục bộ có độ chính xác cao
3. **Global optimization**: Khi có nhiều submaps, hệ thống tối ưu hóa mối quan hệ giữa các submaps thông qua pose graph optimization
4. **Loop closure**: Khi robot quay lại vị trí cũ, hệ thống phát hiện và điều chỉnh lại toàn bộ bản đồ để loại bỏ sai số tích lũy

Quá trình scan matching trong Cartographer tối ưu hóa hàm chi phí:

$$\mathbf{x}^* = \arg \min_{\mathbf{x}} \left(\sum_k (1 - M(\mathbf{T}_x \mathbf{h}_k))^2 + \lambda \|\mathbf{x} - \mathbf{x}_{odom}\|^2 \right) \quad (2.50)$$

trong đó M là occupancy grid map, $\mathbf{h}_k$ là các điểm laser scan, $\mathbf{T}_x$ là phép biến đổi pose, và $\mathbf{x}_{odom}$ là ước lượng từ odometry.

2.5.8.4 Kiến trúc hệ thống

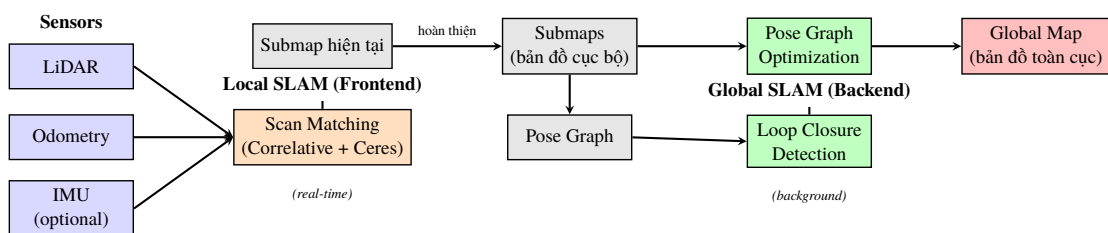
Cartographer gồm hai hệ thống con hoạt động song song (Hình 2.4):

Local SLAM (Frontend - Real-time):

- Nhận dữ liệu từ LiDAR, odometry, và IMU (tùy chọn)
- Thực hiện scan matching để ước lượng pose hiện tại
- Xây dựng submaps cục bộ có tính nhất quán cao
- Chạy real-time (10-40 Hz) để đảm bảo phản hồi tức thì

Global SLAM (Backend - Background):

- Quản lý pose graph chứa tất cả submaps
- Phát hiện loop closure khi robot quay lại vị trí cũ
- Tối ưu hóa toàn cục để giảm sai số tích lũy
- Chạy nền không ảnh hưởng đến Local SLAM



Hình 2.4. Kiến trúc thuật toán Cartographer SLAM với hai hệ thống con: Local SLAM (frontend) xử lý real-time và Global SLAM (backend) tối ưu hóa toàn cục

2.5.8.5 Quy trình mapping

Quy trình xây dựng bản đồ trong Cartographer diễn ra theo các bước:

1. Thu thập dữ liệu cảm biến

- LiDAR cung cấp laser scans (điểm đám mây 2D/3D)
- Odometry ước lượng chuyển động của robot
- IMU (tùy chọn) cung cấp thông tin gia tốc và góc xoay

2. Local SLAM xử lý real-time

- Nhận scan mới từ LiDAR

- Sử dụng odometry làm ước lượng ban đầu
- Thực hiện scan matching với submap hiện tại
- Cập nhật pose của robot và thêm scan vào submap

3. Xây dựng submaps

- Mỗi submap chứa khoảng 90-200 laser scans
- Các submaps chồng lấp 50% để đảm bảo tính liên tục
- Khi submap hoàn thiện, nó được "đóng băng" và thêm vào pose graph

4. Global SLAM tối ưu hóa nền

- Phát hiện loop closure: so sánh submap hiện tại với các submaps cũ
- Nếu phát hiện robot quay lại vị trí đã đi qua, tạo constraint mới
- Chạy pose graph optimization để điều chỉnh vị trí tất cả submaps
- Đảm bảo tính nhất quán toàn cục, loại bỏ drift tích lũy

5. Xuất bản đồ toàn cục

- Ghép tất cả submaps đã được tối ưu hóa
- Tạo occupancy grid map hoàn chỉnh
- Lưu file .pbstream (định dạng riêng) hoặc .pgm (ảnh)

2.5.8.6 Các tham số quan trọng

Theo tài liệu chính thức của Cartographer [18], các tham số quan trọng nhất để tinh chỉnh được chia thành ba nhóm chính:

1. Local SLAM - Chất lượng mapping (quan trọng nhất):

Ceres Scan Matcher là thành phần cốt lõi của Cartographer, sử dụng thư viện **Ceres Solver** (một thư viện tối ưu hóa phi tuyến mạnh mẽ của Google) để căn chỉnh laser scan với submap hiện tại. Quá trình này giải bài toán tối ưu hóa phi tuyến (Equation 2.50) để tìm pose tốt nhất của robot, cân bằng giữa hai yếu tố: (1) mức độ khớp giữa laser scan và bản đồ, và (2) mức độ tin cậy vào odometry. Hai trọng số `translation_weight` và `rotation_weight` điều chỉnh độ tin cậy này.

Hai tham số này quyết định chất lượng scan matching và độ chính xác bản đồ:

- `ceres_scan_matcher.translation_weight` (mặc định: 10):
 - Độ tin cậy vào odometry trong quá trình tịnh tiến
 - Tăng khi odometry tốt, giảm khi odometry nhiễu

- Giá trị khuyến nghị: 1e2 (100)
- `ceres_scan_matcher.rotation_weight` (mặc định: 40):
 - Độ tin cậy vào odometry trong quá trình xoay
 - Tăng khi IMU/encoder tốt, giảm khi robot trượt nhiều
 - Giá trị khuyến nghị: 4e2 (400)

Nguyên tắc: Tăng weights khi tin tưởng odometry, giảm weights khi muốn scan matching tự do hơn.

2. Global SLAM - Tối ưu hóa hiệu năng:

Nhóm tham số này giảm độ trễ (latency) cho phần cứng yếu:

- `optimize_every_n_nodes`: Giảm xuống để optimization chạy thường xuyên hơn nhưng tốn tài nguyên (mặc định: 90)
- `num_background_threads`: Tăng theo số lõi CPU (Jetson Nano: 2-4)
- `constraint_builder.sampling_ratio`: Giảm để xử lý ít constraints hơn (mặc định: 0.3)
- `global_sampling_ratio`: Giảm để giảm tải Global SLAM (mặc định: 0.003)
- `voxel_filter_size`: Tăng để giảm số điểm cần xử lý (mặc định: 0.025m)

3. Submap Configuration - Cân bằng chi tiết và tốc độ:

- `submaps.num_range_data`: Giảm để tạo submaps nhỏ hơn, xử lý nhanh hơn (mặc định: 90)
- `submaps.grid_options.resolution`: Tăng để giảm chi tiết bản đồ nhưng nhanh hơn (mặc định: 0.05m)
- `max_range`: Giảm nếu LiDAR nhiễu ở xa (mặc định: 25m cho RPLiDAR)

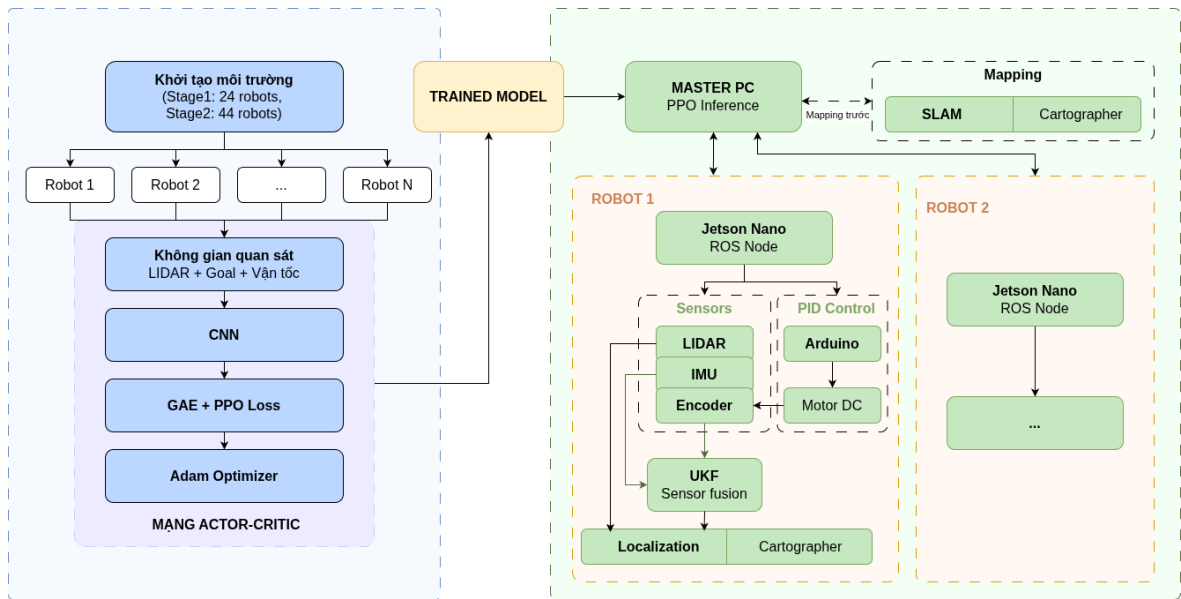
CHƯƠNG 3

PHƯƠNG PHÁP NGHIÊN CỨU

Chương này trình bày chi tiết phương pháp nghiên cứu bao gồm môi trường mô phỏng, thiết kế hàm reward, kiến trúc mạng nơ-ron, quy trình huấn luyện, xây dựng robot thực tế, thiết kế các module điều khiển (PID), sensor fusion (UKF), mapping và localization (Cartographer SLAM), và triển khai hệ thống robot hoàn chỉnh.

3.1 Sơ đồ khối tổng thể

Hình 3.1 minh họa toàn bộ quy trình từ huấn luyện trong mô phỏng đến triển khai trên robot thực tế.



Hình 3.1. Sơ đồ khối tổng thể hệ thống: Huấn luyện MARL trong mô phỏng (trái) và triển khai trên robot thực (phải)

Phần huấn luyện (trái): Môi trường Stage Simulator chạy 24-44 robots song song. Mỗi robot quan sát (LiDAR, goal, velocity) được xử lý qua CNN để trích xuất đặc trưng, sau đó Actor-Critic network xuất hành động. PPO với Adam optimizer cập nhật policy dựa trên reward thu thập được.

Phần triển khai (phải): Robot thực sử dụng LiDAR LD19, IMU và encoder. Jetson Nano chạy ROS nodes để thu thập dữ liệu cảm biến. UKF fusion kết hợp odometry và IMU, Cartographer SLAM định vị robot. Master PC chạy inference model đã huấn luyện, gửi lệnh điều khiển (v, ω) về Arduino để PID điều khiển động cơ.

3.2 Huấn luyện trên môi trường mô phỏng

3.2.1 Môi trường huấn luyện

Hệ đa robot tránh va chạm được huấn luyện trong môi trường mô phỏng dựa trên Stage simulator kết hợp với ROS (Robot Operating System). Bài toán điều khiển được mô hình hóa dưới dạng Partially Observable Markov Decision Process (POMDP), trong đó mỗi robot đưa ra quyết định dựa trên quan sát cục bộ của chính nó mà không cần giao tiếp với các robot khác.

3.2.1.1 Không gian quan sát

Tại thời điểm t , robot i nhận được vector quan sát $o_t^i \in \mathcal{O}$ bao gồm ba thành phần:

$$o_t^i = [o_z^t, o_g^t, o_v^t] \quad (3.1)$$

trong đó:

Dữ liệu LiDAR $o_z^t \in \mathbb{R}^{3 \times 454}$: Cảm biến quét 360 độ với 454 tia laser. Range được đặt giống với cảm biến trên robot thực tế: 0.03m - 5.5m. Mỗi phép đo trả về khoảng cách đến vật cản gần nhất theo hướng tương ứng.

Vị trí đích $o_g^t \in \mathbb{R}^2$: Tọa độ tương đối (r, θ) từ robot đến đích trong hệ tọa độ cục bộ của robot, với r là khoảng cách và θ là góc lệch so với hướng robot.

Vận tốc hiện tại $o_v^t \in \mathbb{R}^2$: Vận tốc tuyến tính v (m/s) và vận tốc góc ω (rad/s) của robot, giúp model nhận biết trạng thái chuyển động hiện tại.

3.2.1.2 Không gian hành động

Robot điều khiển chuyển động thông qua cặp hành động liên tục:

$$a_t = [v_t, \omega_t] \quad (3.2)$$

với $v_t \in [0, v_{\max}]$ là vận tốc tuyến tính giới hạn trong khoảng 0-0.3 m/s, và $\omega_t \in [-\omega_{\max}, \omega_{\max}]$ là vận tốc góc giới hạn trong ± 1.0 rad/s. Mạng Actor xuất ra phân phối Gaussian $\mathcal{N}(\mu, \sigma^2)$ cho mỗi thành phần, sau đó lấy mẫu để thu được hành động cụ thể.

3.2.2 Thiết kế hàm reward

Hàm reward đóng vai trò quyết định trong việc định hình hành vi của robot. Reward function được thiết kế với mục tiêu cân bằng giữa hiệu quả (đến đích nhanh) và an toàn (tránh va chạm), đồng thời đảm bảo tín hiệu học tập rõ ràng không mâu thuẫn. Đồng thời kế thừa những ưu điểm đã được chứng minh ở bài báo gốc [10]

3.2.2.1 Terminal rewards

Reward được cấp khi episode kết thúc:

- $r_{\text{arrival}} = +30$: Đến đích thành công (trong bán kính 15cm từ goal)
- $r_{\text{collision}} = -25$: Va chạm với vật cản hoặc robot khác
- $r_{\text{timeout}} = -10$: Hết thời gian cho phép (chạm giới hạn timeout)

Các giá trị terminal rewards được thiết kế mạnh hơn so với bài báo gốc (30/-25 thay vì 15/-15 [10]) để tạo tín hiệu rõ ràng hơn cho quá trình học.

3.2.2.2 Step rewards - Stage 1

Tại mỗi bước thời gian t , reward được tính từ 5 thành phần:

$$r_t = r_{\text{progress}} + r_{\text{safety}} + r_{\text{rotation}} + r_{\text{heading}} + r_{\text{smooth}} \quad (3.3)$$

Progress reward khuyến khích robot tiến về phía đích. Nếu khoảng cách đến đích giảm từ d_{t-1} xuống d_t , robot nhận được reward tỷ lệ với độ tiến bộ:

$$r_{\text{progress}} = 2.0 \times (d_{t-1} - d_t) \quad (3.4)$$

Hệ số 2.0 (thấp hơn 2.5 trong bài báo gốc) giúp robot ưu tiên an toàn hơn tốc độ.

Safety reward áp dụng penalty khi robot đi quá nhanh ở khu vực gần vật cản. Hệ thống 2 vùng được thiết kế dựa trên khoảng cách gần nhất $d_{\min}$ đến vật cản:

$$r_{\text{safety}} = \begin{cases} -0.3 \times (v_t - 0.2) & \text{if } d_{\min} < 0.35\text{m and } v_t > 0.2 \\ -0.1 \times (v_t - 0.4) & \text{if } 0.35 \leq d_{\min} < 0.6\text{m and } v_t > 0.4 \\ 0 & \text{otherwise} \end{cases} \quad (3.5)$$

Logic: ở khu vực nguy hiểm ($<0.35\text{m}$), phạt mạnh nếu vận tốc vượt 0.2 m/s; ở khu vực cảnh báo (0.35-0.6m), phạt nhẹ nếu vận tốc vượt 0.4 m/s. Gradient mượt này giúp robot học cách điều chỉnh tốc độ theo mức độ nguy hiểm.

Rotation penalty hạn chế xoay quá nhanh để tránh chuyển động không mượt:

$$r_{\text{rotation}} = \begin{cases} -0.06 \times |\omega_t| & \text{if } |\omega_t| > 0.8 \text{ rad/s} \\ 0 & \text{otherwise} \end{cases} \quad (3.6)$$

Ngưỡng 0.8 rad/s và hệ số phạt -0.06 (nhẹ hơn -0.1 trong bài báo gốc) cho phép robot chuyển hướng dễ dàng hơn khi gặp vật cản.

Heading reward khuyến khích robot hướng về phía đích:

$$r_{\text{heading}} = \begin{cases} 0.15 \times (1 - |\theta_{\text{goal}}|/\pi) & \text{if } |\theta_{\text{goal}}|/\pi < 0.3 \\ 0 & \text{otherwise} \end{cases} \quad (3.7)$$

với θ_{goal} là góc lệch giữa hướng robot và hướng tới đích. Reward này giúp robot chủ động hướng về đích thay vì chỉ phụ thuộc vào progress reward.

Velocity smoothing penalty phạt thay đổi vận tốc đột ngột để tạo chuyển động mượt:

$$r_{\text{smooth}} = \begin{cases} -0.1 \times |v_t - v_{t-1}| & \text{if } |v_t - v_{t-1}| > 0.1 \text{ m/s} \\ 0 & \text{otherwise} \end{cases} \quad (3.8)$$

Phạt này ngăn robot tăng/giảm tốc đột ngột, giúp chuyển động mượt mà hơn và giảm hao mòn động cơ khi triển khai lên robot thực.

3.2.2.3 Điều chỉnh reward cho Stage 2

Stage 2 sử dụng cùng cấu trúc reward như Stage 1 nhưng có 3 điều chỉnh quan trọng để phù hợp với môi trường phức tạp hơn:

1. Timeout limit tăng: Từ 500 steps (Stage 1) lên 700 steps (Stage 2) vì khoảng cách goal xa hơn và có nhiều vật cản tĩnh hơn cần né và di chuyển. Điều này cho phép robot có đủ thời gian hoàn thành nhiệm vụ phức tạp mà không bị phạt sớm.

2. Heading reward (thưởng hướng về goal) có điều kiện: Chỉ áp dụng khi không có vật cản gần ($d_{\min} > 0.6\text{m}$) và giảm hệ số từ 0.15 xuống 0.1:

$$r_{\text{heading}}^{\text{Stage2}} = \begin{cases} 0.1 \times (1 - |\theta_{\text{goal}}|/\pi) & \text{if } d_{\min} > 0.6\text{m and } |\theta_{\text{goal}}|/\pi < 0.3 \\ 0 & \text{otherwise} \end{cases} \quad (3.9)$$

Lý do: trong Stage 2 với nhiều vật cản, việc ưu tiên heading về goal có thể dẫn robot vào đường cụt. Điều kiện $d_{\min} > 0.6\text{m}$ đảm bảo heading reward chỉ active khi đường đi an toàn.

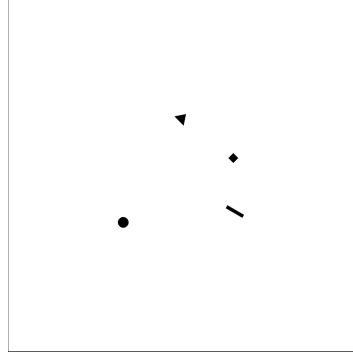
3. Velocity smoothing giảm hệ số: Từ -0.1 (Stage 1) xuống -0.05 (Stage 2) vì robot cần phản ứng nhanh hơn với vật cản động và tĩnh trong môi trường phức tạp. Penalty nhẹ hơn cho phép thay đổi tốc độ linh hoạt hơn khi cần.

Các thành phần khác (progress, safety, rotation) giữ nguyên giá trị để đảm bảo transfer learning từ Stage 1 sang Stage 2 hoạt động hiệu quả - policy đã học không bị nhiễu bởi reward function khác biệt quá nhiều.

3.2.3 Chiến lược huấn luyện 2 giai đoạn

Áp dụng curriculum learning theo 2 stages tương tự Long et al. (2018) [10] nhưng điều chỉnh cho phù hợp với môi trường Stage simulator và số lượng robots khác nhau. Các kịch bản huấn luyện đa dạng (đã mô tả ở Mục 2.4.5) được sử dụng xuyên suốt quá trình training.

Stage 1 - Foundation learning: Huấn luyện 24 robots trên môi trường cơ bản có ít vật cản, học các hành vi cơ bản: tránh va chạm cục bộ, di chuyển về đích, điều chỉnh vận tốc. Hyperparameters ưu tiên exploration cao (entropy $8e-3$) để khám phá không gian hành động nhanh.

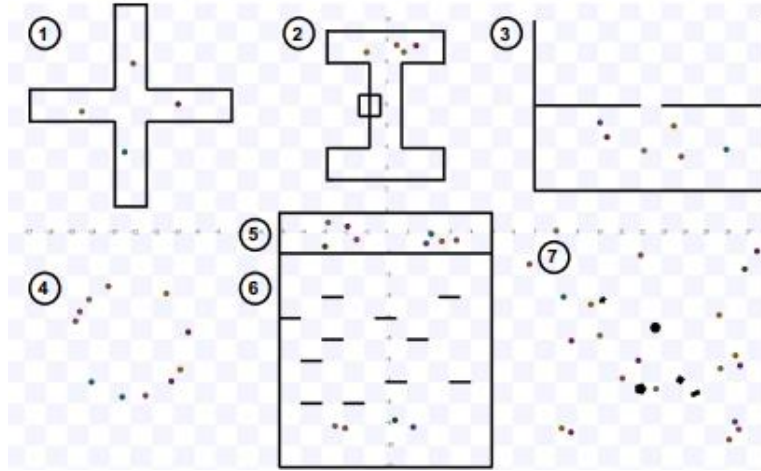


Hình 3.2. Stage 1: môi trường đơn giản cho 24 robots

Stage 2 - Transfer learning: Khởi tạo từ Stage 1 weights, tiếp tục huấn luyện 44 robots trên random scatter scenarios phức tạp hơn. Hình 3.3 (từ Long et al. 2018) cho thấy đa dạng tình huống. Hyperparameters điều chỉnh: tăng entropy (exploration nhiều hơn), tăng epochs lên 5, thêm value clipping để ổn định training với mật độ cao.

Vấn đề timeout trong Stage 2: Trong quá trình huấn luyện Stage 2, sau khoảng 3000 updates, kết quả bắt đầu suy giảm dù đã đạt success rate khá cao. Phân tích behavior của robots cho thấy chúng đang học cách di chuyển nhanh và thẳng nhất có thể - hành vi này có thể phản ánh việc reward phạt do timeout'. Do timeout limit ban đầu (500 steps) không đủ cho môi trường phức tạp hơn với 44 robots, policy học được rằng cách tốt nhất để tránh phạt timeout là lao thẳng về goal bất chấp có vật cản, dẫn đến tỉ lệ va chạm tăng.

Giải pháp được áp dụng: giữ lại checkpoint tốt nhất và tăng giới hạn timeout từ 500 lên 700 steps. Với giới hạn thời gian dài hơn, robots có thể chọn đường đi an toàn hơn. Kết quả được trình bày chi tiết ở Mục 4.1.2.



Hình 3.3. Stage 2: 7 tình huống huấn luyện đa dạng từ đơn giản đến phức tạp giúp policy generalize tốt (nguồn: Long et al. 2018)

3.2.4 Kiến trúc mạng nơ-ron

Kiến trúc Actor-Critic với hai encoder hoàn toàn độc lập được sử dụng để xử lý dữ liệu laser scan và tạo ra chính sách hành động (Actor) và ước lượng giá trị trạng thái (Critic). Thiết kế này cho phép Actor và Critic học các đặc trưng tối ưu riêng biệt cho từng mục tiêu (action selection vs value estimation), đồng thời sử dụng learning rates khác nhau (Critic LR gấp 15 lần Actor LR) để tăng hiệu quả học tập.

3.2.4.1 Encoder - Xử lý LiDAR

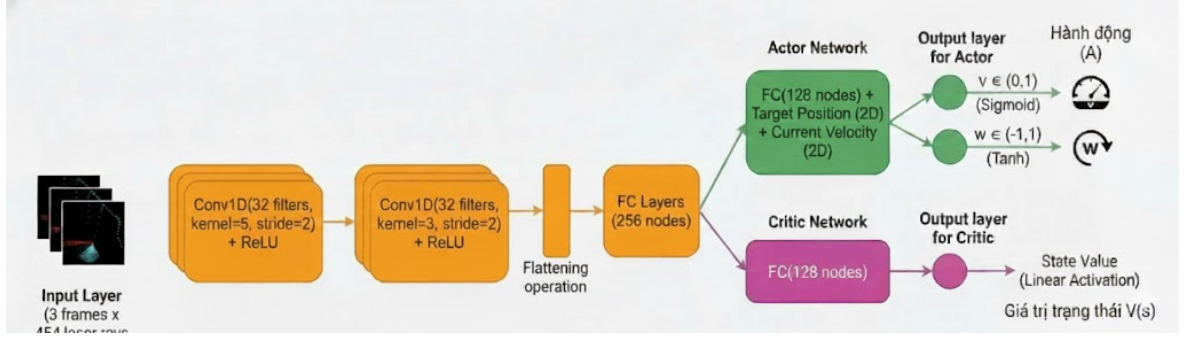
Dữ liệu laser scan đầu vào $o_z^t \in \mathbb{R}^{3 \times 454}$ được xử lý qua 2 encoder riêng biệt cho Actor và Critic, mỗi encoder có cấu trúc giống nhau với 2 lớp convolution 1D:

- **Conv1D Layer 1:** 32 filters, kernel size 5, stride 2, theo sau bởi ReLU
- **Conv1D Layer 2:** 32 filters, kernel size 3, stride 2, theo sau bởi ReLU

Sau khi flatten, output của convolution có chiều khoảng 3600 dimensions được nén xuống qua fully connected layer với 256 nơ-ron. Việc sử dụng 2 encoder riêng biệt cho phép:

- **Actor encoder** học các đặc trưng tối ưu cho action selection (tập trung vào local obstacles, immediate threats)
- **Critic encoder** học các đặc trưng tối ưu cho value estimation (tập trung vào global path, long-term outcomes)
- Tránh gradient conflict giữa policy loss và value loss, cho phép mỗi network tối ưu hóa đặc trưng phù hợp với mục tiêu riêng

Hình 3.4 minh họa kiến trúc mạng Actor-Critic với hai encoder độc lập và luồng dữ liệu.



Hình 3.4. Kiến trúc mạng Actor-Critic.

3.2.4.2 Mạng Actor - Sinh policy

Actor có encoder riêng biệt để xử lý LiDAR data. Output của Actor encoder được kết hợp với vị trí đích (r, θ) và vận tốc hiện tại (v, ω) , sau đó đi qua:

- Actor Fully Connected layer 128 nơ-ron + ReLU
- Actor Output layer: 2 nơ-ron (giá trị trung bình cho v và ω)
- Separate learnable log-standard deviation parameters

Actor xuất ra phân phối Gaussian cho mỗi chiều của action:

$$\pi(a_t|o_t) = \mathcal{N}(\mu_\theta(o_t), \sigma_\theta^2) \quad (3.10)$$

với μ_θ là giá trị mean được tính từ mạng nơ-ron, và $\sigma_\theta = \exp(\log_std)$ với $\log_std$ là tham số độc lập được học. Mean value μ_v cho vận tốc tuyến tính đi qua sigmoid để giới hạn trong $[0, v_{\max}]$, còn μ_ω cho vận tốc góc đi qua tanh để giới hạn trong $[-\omega_{\max}, \omega_{\max}]$.

3.2.4.3 Mạng Critic - Ước lượng giá trị

Critic có encoder hoàn toàn riêng biệt với Actor, cùng cấu trúc nhưng parameters độc lập:

- Critic Encoder: 2 lớp Conv1D + FC 256 (giống Actor về cấu trúc nhưng khác parameters)
- Concatenate Critic encoder output với goal và velocity
- Critic Fully Connected layer 128 nơ-ron + ReLU
- Critic Output layer: 1 nơ-ron (giá trị estimate)

Ý nghĩa của encoder độc lập: Việc sử dụng encoder hoàn toàn riêng biệt cho Actor và Critic có 3 lợi ích quan trọng:

1. **Tránh gradient conflict:** Actor loss muốn thay đổi encoder theo hướng tối ưu cho action selection, trong khi Critic loss muốn thay đổi encoder theo hướng tối ưu cho value estimation. Nếu chia sẻ encoder, 2 gradient này xung đột, làm cả hai đều học chậm.

2. **Specialized feature extraction:** Actor encoder học features tốt cho local obstacle avoidance (immediate decisions), Critic encoder học features tốt cho long-term value prediction (global path evaluation). Các đặc trưng này có thể khác nhau và tối ưu cho từng mục tiêu riêng.
3. **Training stability:** Encoder riêng biệt cho phép áp dụng các kỹ thuật như value clipping và separate gradient clipping norms mà không ảnh hưởng Actor.

Hạn chế: Encoder độc lập tốn gấp đôi số tham số (250K vs 150K nếu shared), nhưng đổi lại training ổn định hơn nhiều và hiệu năng cao hơn.

Critic ước lượng value function:

$$V_{\phi}(o_t) \approx \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid o_t \right] \quad (3.11)$$

với γ là discount factor. Value này được sử dụng để tính advantage function trong thuật toán PPO.

3.2.4.4 Entropy và Exploration

Trong học tăng cường, **entropy** đo mức độ ngẫu nhiên (randomness) của policy. Đối với Gaussian policy $\pi(a|o) = \mathcal{N}(\mu, \sigma^2)$, entropy được tính bằng:

$$H[\pi(\cdot|o)] = \frac{1}{2}(1 + \log(2\pi) + 2 \log \sigma) \quad (3.12)$$

Vai trò của entropy trong exploration:

- **High entropy** (σ lớn): Policy có nhiều sự ngẫu nhiên, action được sample có variance cao $\rightarrow$ robot thử nhiều hành động khác nhau $\rightarrow$ *exploration* (khám phá không gian hành động)
- **Low entropy** (σ nhỏ): Policy deterministic hơn, action gần với mean $\mu \rightarrow$ robot chọn hành động tốt nhất đã biết $\rightarrow$ *exploitation* (khai thác kinh nghiệm đã học)

Trong thuật toán PPO, entropy bonus được thêm vào loss function:

$$L_{actor} = -L^{CLIP}(\theta) - c_{ent} \cdot H[\pi_{\theta}] \quad (3.13)$$

với c_{ent} là hệ số entropy. Khi huấn luyện:

- *Giai đoạn đầu:* c_{ent} cao (8e-3) khuyến khích policy explore nhiều hành động khác nhau, giúp tìm ra các chiến lược tốt
- *Giai đoạn sau:* c_{ent} decay xuống (2e-3) để policy converge về deterministic policy, tận dụng kinh nghiệm đã học

Cân bằng exploration-exploitation thông qua entropy decay giúp robot học hiệu quả: khám phá đủ rộng ở đầu, sau đó tập trung vào chiến lược tốt nhất.

3.2.5 Thuật toán PPO với GAE

Proximal Policy Optimization (PPO) được sử dụng với clipped surrogate objective để đảm bảo policy updates không quá lớn. Algorithm 2 trình bày chi tiết quy trình huấn luyện.

Algorithm 2 PPO với Đa Robot cho Tránh Va Chạm

Input: N robots song song, horizon T , epochs K , batch size M , clip ϵ , discount γ , GAE λ

- 1: Khởi tạo policy π_θ (Actor) và value function V_ϕ (Critic)
- 2: Khởi tạo Adam optimizer cho Actor (α_{actor}) và Critic (α_{critic})
- 3: **for** iteration = 1, 2, ... **do**
- 4: **// Phase 1: Thu thập dữ liệu**
- 5: **for** $t = 0, 1, \dots, T - 1$ **do**
- 6: **for** mỗi robot $i = 1, \dots, N$ song song **do**
- 7: Quan sát o_t^i từ môi trường
- 8: Sample action $a_t^i \sim \pi_\theta(\cdot | o_t^i)$
- 9: Thực thi a_t^i , nhận r_t^i, o_{t+1}^i
- 10: Lưu $(o_t^i, a_t^i, r_t^i, o_{t+1}^i)$ vào buffer $\mathcal{D}$
- 11: **end for**
- 12: **end for**
- 13: **// Phase 2: Tính Advantage với GAE**
- 14: **for** mỗi trajectory trong $\mathcal{D}$ **do**
- 15: Tính TD residuals: $\delta_t = r_t + \gamma V_\phi(o_{t+1}) - V_\phi(o_t)$
- 16: Tính GAE: $\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}$
- 17: Tính returns: $R_t = \hat{A}_t + V_\phi(o_t)$
- 18: **end for**
- 19: Chuẩn hóa advantages: $\hat{A} = (\hat{A} - \text{mean}(\hat{A})) / (\text{std}(\hat{A}) + 10^{-8})$
- 20: Lưu old log probabilities: $\log \pi_{\theta_{old}}(a_t | o_t)$
- 21: **// Phase 3: Cập nhật Policy và Value**
- 22: **for** epoch = 1, ..., K **do**
- 23: **for** mỗi minibatch $\mathcal{B} \subset \mathcal{D}$ kích thước M **do**
- 24: Tính ratio: $r_t(\theta) = \exp(\log \pi_\theta(a_t | o_t) - \log \pi_{\theta_{old}}(a_t | o_t))$
- 25: Tính clipped objective: $L^{CLIP} = \min(r_t \hat{A}_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)$
- 26: Tính entropy: $H = - \sum \pi_\theta(a | o) \log \pi_\theta(a | o)$
- 27: Actor loss: $L_{actor} = -\text{mean}(L^{CLIP}) - c_{ent} \cdot H$
- 28: Cập nhật θ với gradient clipping $\|\nabla\| \leq 0.5$
- 29: Tính value loss: $L_{critic} = 0.5 \cdot \text{mean}((V_\phi(o_t) - R_t)^2)$
- 30: Cập nhật ϕ với gradient clipping $\|\nabla\| \leq 0.5$
- 31: **end for**
- 32: **end for**
- 33: **end for**
- 34: **return** Policy đã huấn luyện π_θ

Các công thức toán học chính:

Probability Ratio - tỷ số xác suất giữa policy mới và cũ:

$$r_t(\theta) = \frac{\pi_\theta(a_t|o_t)}{\pi_{\theta_{old}}(a_t|o_t)} = \exp(\log \pi_\theta(a_t|o_t) - \log \pi_{\theta_{old}}(a_t|o_t)) \quad (3.14)$$

Clipped Surrogate Objective - hàm mục tiêu với clipping để giới hạn policy update:

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)] \quad (3.15)$$

với $\epsilon = 0.1$ là clip parameter, $\hat{A}_t$ là advantage estimate đã chuẩn hóa.

Generalized Advantage Estimation (GAE) - ước lượng advantage với cân bằng bias-variance:

$$\hat{A}_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V_\phi(o_{t+1}) - V_\phi(o_t) \quad (3.16)$$

với $\gamma = 0.99$ (discount factor), $\lambda = 0.92 - 0.96$ (GAE parameter).

Entropy Bonus - khuyến khích exploration thông qua entropy của policy:

$$H[\pi_\theta(\cdot|o_t)] = - \sum_a \pi_\theta(a|o_t) \log \pi_\theta(a|o_t) \quad (3.17)$$

Với Gaussian policy: $H = \frac{1}{2}(1 + \log(2\pi) + 2 \log \sigma)$

Tổng Loss cho Actor:

$$L_{actor} = -L^{CLIP}(\theta) - c_{ent} \cdot H[\pi_\theta] \quad (3.18)$$

với $c_{ent} = 0.01 - 0.02$ là hệ số entropy bonus.

3.2.6 Hyperparameters chi tiết

Bảng 3.1 và 3.2 liệt kê đầy đủ hyperparameters cho 2 giai đoạn huấn luyện.

Bảng 3.1. Hyperparameters Stage 1 (24 robots - 83% success)

Parameter	Value
NUM_ENV (số robots)	24
GAMMA (γ - discount factor)	0.99
LAMDA (λ - GAE)	0.96
EPOCH (training iterations per update)	2
COEFF_ENTROPY (entropy bonus)	2e-2
CLIP_VALUE (ϵ - PPO clip)	0.1
LEARNING_RATE	5e-4
HORIZON (steps per rollout)	128
BATCH_SIZE	1024
Timeout (max steps per episode)	500

Bảng 3.2. Hyperparameters Stage 2 (44 robots - 89% success)

Parameter	Value
NUM_ENV (số robots)	44
GAMMA (γ - discount factor)	0.99
LAMDA (λ - GAE)	0.92
EPOCH (training iterations per update)	3
COEFF_ENTROPY (entropy bonus)	1e-2
CLIP_VALUE (ϵ - PPO clip)	0.1
CRITIC_LR (learning rate)	1e-3
ACTOR_LR (learning rate)	4e-4
Timeout (max steps per episode)	700

3.2.7 Các kỹ thuật cải tiến

So với kiến trúc và quy trình huấn luyện trong bài báo gốc [10], một số kỹ thuật được bổ sung để cải thiện hiệu năng và độ ổn định trong quá trình huấn luyện:

3.2.7.1 Orthogonal Initialization

Trong deep learning, *initialization* (khởi tạo) là quá trình gán giá trị ban đầu cho các trọng số (weights) của mạng nơ-ron trước khi bắt đầu huấn luyện. Việc khởi tạo đúng cách rất quan trọng vì:

- Nếu trọng số quá lớn: gradient sẽ explode (tăng vô hạn), model không học được
- Nếu trọng số quá nhỏ: gradient sẽ vanish (tiến về 0), model học rất chậm

Phương pháp **Orthogonal initialization** khởi tạo ma trận trọng số $W \in \mathbb{R}^{m \times n}$ sao cho các cột trực giao với nhau, tức là $W^T W = I_n$ (ma trận đơn vị). Điều này đảm bảo khi input đi qua layer, độ lớn của activation không bị co hay giãn quá mức. So với

Xavier initialization tiêu chuẩn, orthogonal initialization cho gradient flow ổn định hơn trong các mạng sâu và đặc biệt hiệu quả với reinforcement learning.

Trong implementation, orthogonal initialization được áp dụng với *gain* (hệ số khuếch đại) khác nhau tùy activation function:

- Convolutional và FC layers với ReLU: $\text{gain} = \sqrt{2}$ (bù cho việc ReLU “tắt” 50% nơ-ron)
- Output layer của Actor (mean): $\text{gain} = 0.01$ (giá trị nhỏ để policy ban đầu gần uniform)
- Output layer của Critic (value): $\text{gain} = 1.0$

3.2.7.2 Chuẩn hóa Observation

Dữ liệu LiDAR scan có thể có scale rất khác nhau giữa các thời điểm (khi gần vật cản vs. khi không gian trống). Để giúp mạng nơ-ron học hiệu quả hơn, input được chuẩn hóa (normalize) về mean ≈ 0 và standard deviation ≈ 1 .

Thay vì tính mean/std cố định từ dataset (không khả thi trong RL vì data sinh ra liên tục), phương pháp **running statistics** được sử dụng: mean và std được cập nhật liên tục theo từng batch data mới:

$$\mu_{new} = (1 - \alpha) \cdot \mu_{old} + \alpha \cdot \mu_{batch}, \quad \alpha = 0.01 \quad (3.19)$$

Công thức tương tự cho standard deviation. Điều này đảm bảo model luôn nhận input ở scale nhất quán, giúp quá trình học ổn định hơn.

3.2.7.3 Log-std Clamping - Giới hạn log-std

Trong Gaussian policy, mỗi action được sample từ phân phối chuẩn $a \sim \mathcal{N}(\mu, \sigma^2)$, trong đó:

- μ (mean): giá trị trung bình, được predict bởi Actor network
- σ (standard deviation): độ “lan tỏa” của phân phối, quyết định mức độ exploration

Thay vì dự đoán σ trực tiếp, mạng sẽ dự đoán $\log(\sigma)$ (gọi là **log-std**). Lý do:

- σ phải luôn dương, nhưng đầu ra của mạng nơ-ron có thể âm
- $\log(\sigma)$ có thể nhận giá trị âm/dương, sau đó lấy $\sigma = e^{\log(\sigma)}$ luôn dương
- Việc học $\log(\sigma)$ ổn định hơn về mặt giá trị

Log-std được **clamp** (giới hạn) trong khoảng $[-2.5, -0.8]$:

$$\log(\sigma) \in [-2.5, -0.8] \Rightarrow \sigma \in [e^{-2.5}, e^{-0.8}] \approx [0.082, 0.449] \quad (3.20)$$

Ý nghĩa của việc giới hạn này:

- **Giới hạn dưới** $\sigma \geq 0.082$: ngăn policy trở nên quá *deterministic*. Nếu $\sigma \rightarrow 0$, policy sẽ luôn chọn cùng một action với mọi observation, mất khả năng khám phá (exploration) các action tốt hơn.
- **Giới hạn trên** $\sigma \leq 0.449$: ngăn policy quá *stochastic*. Nếu σ quá lớn, action được sample sẽ quá ngẫu nhiên, robot di chuyển hỗn loạn và không học được gì.

Khoảng $[0.082, 0.449]$ được chọn qua thực nghiệm để cân bằng giữa exploration (khám phá) và exploitation (khai thác những gì đã biết).

3.2.7.4 Adaptive Learning Rate Scheduler

Vấn đề: Learning rate cố định trong bài báo gốc gặp 2 vấn đề: (1) nếu giảm LR quá sớm khi performance đang cải thiện, sẽ làm chậm momentum; (2) nếu giữ LR cao khi mắc kẹt ở plateau, không thể thoát khỏi cực trị địa phương.

Giải pháp: Thiết kế adaptive LR scheduler với 2 nguyên tắc:

- *Giữ LR* khi hiệu suất trong cửa sổ gần nhất (20 updates gần nhất) đang cải thiện $\rightarrow$ không làm gián đoạn momentum
- *Tăng LR* khi detect plateau (thay đổi $< 2\%$ trong 60 updates) $\rightarrow$ thoát khỏi trạng thái mắc kẹt

Cách hoạt động: LR cao giúp escape saddle points và local minima, nhưng chỉ áp dụng khi thực sự cần (plateau), còn khi đang học tốt thì giữ nguyên để exploit momentum. Kết quả: training ổn định 200 updates không bị degradation, so với fixed LR thường cần 1000+ updates.

3.2.7.5 Aggressive Exploration Strategy (Chiến lược khám phá)

Vấn đề: Entropy coefficient thấp ($1e-3$ như paper gốc) khiến policy converge nhanh về deterministic policy, dẫn đến premature convergence - stuck ở solution tốt cục bộ nhưng không phải global optimum.

Giải pháp: Tăng entropy coefficient lên $8e-3$ ($10\times$ cao hơn), decay dần xuống minimum $2e-3$ theo schedule:

$$\text{entropy_coeff}_t = \max(2 \times 10^{-3}, 8 \times 10^{-3} \times 0.995^t) \quad (3.21)$$

Cách hoạt động: Entropy cao ban đầu khuyến khích khám phá diverse behaviors (nhiều cách tránh va chạm khác nhau), sau đó decay dần để policy exploit learned strategies. Điều này đặc biệt quan trọng trong multi-robot settings với exponential action space - cần explore đủ trước khi exploit.

3.3 Triển khai model và robot thực tế

Sau khi huấn luyện model trên môi trường mô phỏng, phần này trình bày chi tiết quy trình triển khai lên robot thực tế, bao gồm xây dựng phần cứng, thiết kế các module điều khiển (PID, UKF, Cartographer SLAM), và tích hợp hệ thống hoàn chỉnh.

3.3.1 Xây dựng robot thực tế

Robot thực tế được thiết kế với mục tiêu deploy model đã huấn luyện lên phần cứng nhỏ gọn, chi phí thấp, phù hợp cho nghiên cứu multi-robot systems.

3.3.1.1 Thông số kỹ thuật

Kích thước và cấu trúc: Khung robot kích thước 20cm × 15.7cm được gia công từ tấm nhựa acrylic 3mm, thiết kế 2 tầng: tầng dưới chứa động cơ và mạch điều khiển, tầng trên đặt LiDAR và Jetson Nano. Khoảng cách giữa 2 bánh chủ động $L = 0.205\text{m}$.

Hệ thống cảm biến của robot:

- **LiDAR 2D:** Cảm biến quét môi trường 360°, độ phân giải góc 0.8°, quan sát môi trường và định vị.
- **IMU 9-axis:** Đo gia tốc, vận tốc góc và từ trường; dữ liệu được kết hợp với encoder bánh xe thông qua bộ lọc UKF để ước lượng trạng thái chuyển động.
- **Encoder bánh xe:** Độ phân giải 7500 xung/vòng, cung cấp thông tin odometry, được hợp nhất với IMU trong UKF nhằm giảm nhiễu và sai số tích lũy.

Hệ thống điều khiển: Jetson Nano (4GB RAM) chạy Ubuntu 18.04 và ROS Melodic, xử lý sensor fusion và serial giao tiếp với Arduino. Hai động cơ DC giảm tốc (tỷ số truyền 1:20) với encoder 15000 pulses/revolution điều khiển vận tốc bánh xe. Arduino nhận commands qua serial và điều khiển động cơ qua driver L298N.

Nguồn điện: Pin Li-Po 3S 11.1V 2200mAh cung cấp điện cho động cơ (qua ổn áp 12V), và hạ áp 5V cho Jetson Nano. Thiết kế nguồn tách biệt tránh nhiễu điện từ động cơ ảnh hưởng đến tính toán.

3.3.1.2 Kiến trúc phần mềm

Hệ thống ROS gồm 5 nodes chính chạy song song:

- (1) `lidar_node` publish laser scans
- (2) `imu_node` publish IMU data
- (3) `odometry_node` tính wheel odometry và fusion với IMU qua UKF

- (4) `policy_node` load PyTorch model và inference action từ observations
- (5) `controller_node` convert high-level actions (v, ω) sang PWM commands qua PID controllers.

Giao tiếp qua ROS topics đảm bảo tính đồng bộ: dễ thay thế policy mới hoặc thay đổi sensors mà không ảnh hưởng toàn hệ thống.

3.3.2 Thiết kế PID controller

Để điều khiển robot thực tế theo hành động đầu ra từ mạng nơ-ron $(v_{\text{ref}}, \omega_{\text{ref}})$, bộ điều khiển PID được thiết kế cho động cơ bánh xe. Phần này trình bày mô hình động học, thiết kế controller, chứng minh ổn định, và kết quả thực nghiệm.

3.3.2.1 Mô hình động học differential drive robot

Robot sử dụng cấu trúc differential drive với 2 bánh chủ động. Mô hình động học trong hệ tọa độ body frame:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \\ \omega \end{bmatrix} \quad (3.22)$$

với (x, y, θ) là pose của robot trong world frame. Vận tốc bánh trái và phải liên hệ với (v, ω) qua:

$$v_L = v - \frac{L\omega}{2}, \quad v_R = v + \frac{L\omega}{2} \quad (3.23)$$

với $L = 0.205$ m là khoảng cách giữa 2 bánh. Mỗi bánh được điều khiển bởi động cơ DC giảm tốc kèm encoder phản hồi, đo vận tốc dưới dạng encoder pulses per second.

3.3.2.2 Thiết kế PID controller

Cho mỗi bánh, PID controller riêng biệt được thiết kế theo công thức:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (3.24)$$

với $e(t) = v_{\text{ref}}(t) - v_{\text{measured}}(t)$ là sai số vận tốc. Output $u(t)$ là tín hiệu PWM (Pulse Width Modulation) gửi đến driver động cơ.

Các tham số PID được tuning thông qua phương pháp trial-and-error với mục tiêu: (1) settling time < 3 s, (2) overshoot $< 5\%$, (3) steady-state error $< 2\%$. Giá trị cuối cùng cho cả hai bánh là $K_p = 11.6$, $K_i = 4.9$, $K_d = 0.04$. Ba tham số này điều khiển:

K_p phản ứng tỷ lệ với lỗi hiện tại (chính), K_i loại bỏ lỗi tích lũy lâu dài (phụ), K_d giảm dao động bằng cách dự đoán xu hướng (nhỏ nhất).

3.3.2.3 Phân tích ổn định

Để phân tích tính ổn định của closed-loop system, mô hình động cơ DC được kết hợp với PID controller.

Mô hình động cơ: Động cơ DC giảm tốc được mô hình hóa gần đúng bằng hệ bậc nhất (first-order system):

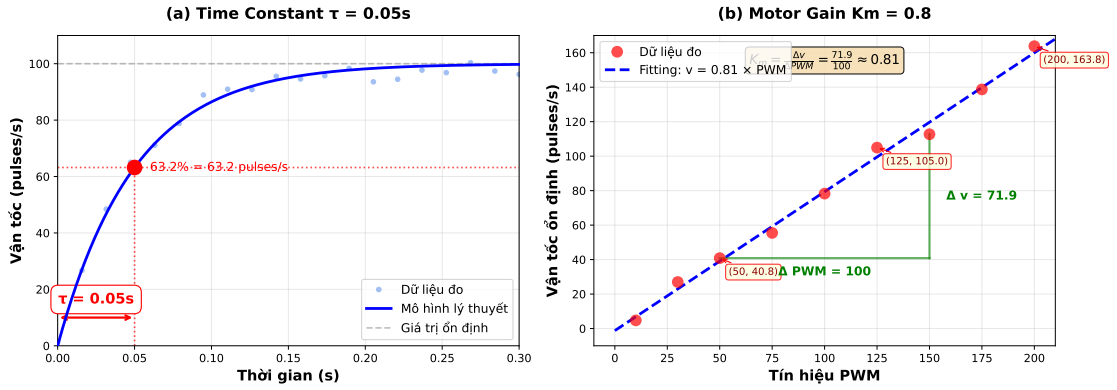
$$\tau \dot{v} + v = K_m u \quad (3.25)$$

với $\tau = 0.05$ s là time constant (đo từ thực nghiệm step response), $K_m = 0.8$ là motor gain (tỷ lệ giữa PWM và vận tốc đạt được), v là vận tốc bánh xe (pulses/s), và u là tín hiệu điều khiển PWM từ PID.

Xác định tham số từ thực nghiệm: Hai tham số τ và K_m được xác định thông qua đo đặc trực tiếp trên động cơ (Hình 3.5).

Time constant τ : Để đo τ , động cơ được cấp step input PWM và quan sát đáp ứng vận tốc. Theo lý thuyết hệ bậc nhất, τ là thời gian để vận tốc đạt 63.2% giá trị ổn định. Hình 3.5(a) cho thấy đáp ứng step từ $0 \rightarrow 100$ pulses/s: tại thời điểm $t = 0.05$ s, vận tốc đạt xấp xỉ 63.2 pulses/s (đúng 63.2% của 100). Dữ liệu đo có nhiễu do encoder resolution và dao động cơ học, nhưng mô hình lý thuyết $v(t) = 100(1 - e^{-t/0.05})$ fit tốt với xu hướng chung. Giá trị $\tau = 0.05$ s phù hợp với đặc tính động cơ DC giảm tốc (inertia thấp do tỷ số truyền, phản ứng nhanh).

Motor gain K_m : Tham số K_m thể hiện độ nhạy của vận tốc theo PWM ở chế độ ổn định. Hình 3.5(b) biểu diễn quan hệ giữa PWM command và vận tốc đo được tại 9 điểm khác nhau (PWM từ 10 đến 200). Các điểm đo có phân tán nhẹ do nhiễu sensor và ma sát thay đổi. Dùng fitting tuyến tính thu được hệ số $K_m \approx 0.81$ pulses/PWM. Để đơn giản hóa tính toán và làm tròn theo encoder specs (15000 pulses/rev, gear 1:20, PWM 8-bit), giá trị $K_m = 0.8$ được sử dụng trong mô hình.



Hình 3.5. Xác định tham số động cơ từ thực nghiệm. (a) Đáp ứng step để xác định time constant $\tau = 0.05s$: các điểm đo (xanh) có nhiều nhẹ nhưng fit tốt với mô hình lý thuyết (đường xanh đậm); tại $t = \tau$, vận tốc đạt 63.2% giá trị ổn định. (b) Quan hệ tuyến tính PWM-vận tốc để xác định motor gain: 9 điểm đo (đỏ) cho fitting $K_m \approx 0.81$, làm tròn thành $K_m = 0.8$ cho mô hình.

Phân tích ổn định bằng Routh-Hurwitz: Khi kết hợp PID controller với mô hình động cơ, hệ kín tạo thành hệ bậc 3 với phương trình đặc trưng:

$$\tau s^3 + (1 + K_m K_d)s^2 + K_m K_p s + K_m K_i = 0 \quad (3.26)$$

Đặt các hệ số của phương trình đặc trưng: $a_0 = \tau = 0.05$, $a_1 = 1 + K_m K_d = 1.032$, $a_2 = K_m K_p = 9.28$, $a_3 = K_m K_i = 3.92$ (với $K_p = 11.6$, $K_i = 4.9$, $K_d = 0.04$, $K_m = 0.8$).

Xây dựng bảng Routh-Hurwitz:

s^3	$a_0 = 0.05$	$a_2 = 9.28$
s^2	$a_1 = 1.032$	$a_3 = 3.92$
s^1	$b_1 = \frac{a_1 a_2 - a_0 a_3}{a_1} = \frac{1.032 \times 9.28 - 0.05 \times 3.92}{1.032} = 9.29$	0
s^0	$c_1 = a_3 = 3.92$	

Bảng 3.3. Bảng Routh-Hurwitz cho hệ PID bậc 3

Điều kiện ổn định Routh-Hurwitz: tất cả phần tử cột đầu tiên phải cùng dấu (dương). Kiểm tra:

- Hàng s^3 : $a_0 = 0.05 > 0$
- Hàng s^2 : $a_1 = 1.032 > 0$
- Hàng s^1 : $b_1 = 9.29 > 0$
- Hàng s^0 : $c_1 = 3.92 > 0$

Tất cả phần tử cột đầu đều dương, do đó hệ thống ổn định. Không có pole nằm bên phải mặt phẳng phức hay trên trục ảo.

3.3.2.4 Kết quả thực nghiệm

Quy đổi vận tốc sang setpoint: Bộ PID hoạt động trong không gian encoder units thay vì m/s để đảm bảo độ chính xác và đơn giản hóa việc đọc encoder. Vận tốc đích v (m/s) được quy đổi sang setpoint s (encoder units) thông qua ánh xạ tuyến tính:

$$s = \frac{v}{v_{\max}} \times 255 \quad (3.27)$$

với $v_{\max} = 1.15$ m/s là vận tốc tối đa của robot. Ví dụ: $v = 0.4$ m/s $\rightarrow s = \frac{0.4}{1.15} \times 255 \approx 88$ units; $v = -0.3$ m/s $\rightarrow s \approx -66$ units.

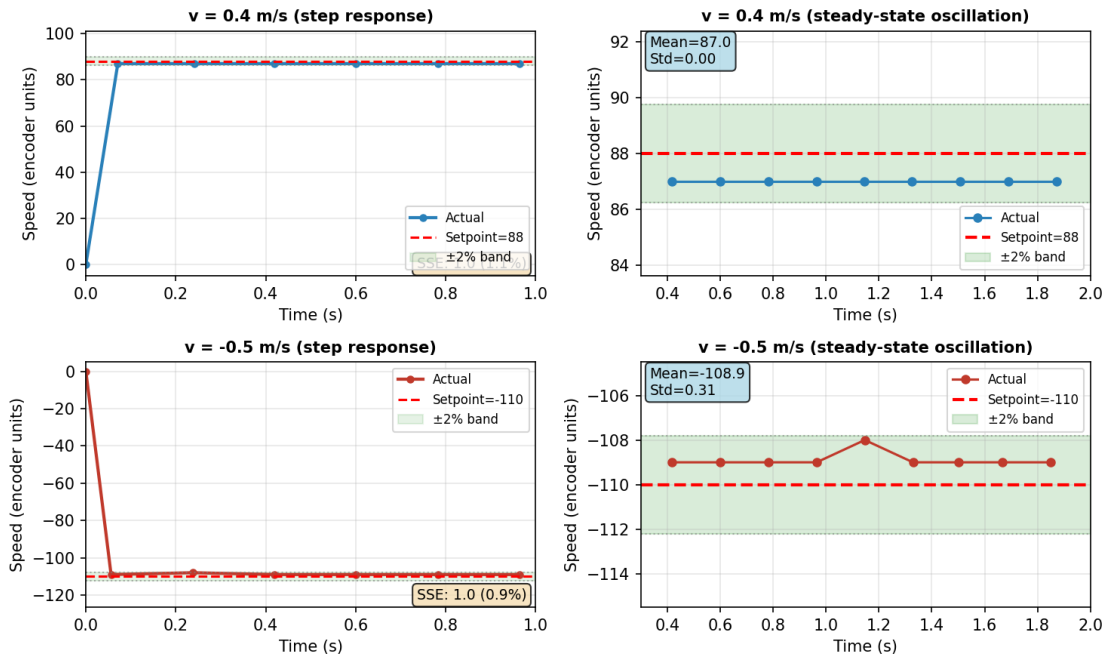
Thử nghiệm step response được tiến hành trên robot thực tế với 6 setpoints khác nhau: 0.2, 0.4, 0.6, -0.3, -0.5, và 0 m/s. Kết quả đo được:

Performance metrics:

- **Steady-state error:** Trung bình 0.5 pulses ($\approx 1.1\%$), rất tốt
- **Overshoot:** Tối đa 1.5% (chỉ ở setpoint -0.3 m/s), đạt yêu cầu $< 5\%$
- **Settling time:** Phần lớn < 1 s, một số trường hợp 3-4s
- **RMS error:** Từ 0.186 đến 3.756 pulses tùy setpoint

Phân tích: Các setpoint có magnitude lớn (0.4, 0.5 m/s) cho kết quả tốt nhất với settling time gần như tức thời và steady-state error $< 1\%$. Ngược lại, các setpoint nhỏ (0.2, -0.3 m/s) có settling time chậm hơn và oscillation lớn hơn (std dev 1.9-2.0 pulses). Điều này do ở vận tốc thấp, ma sát tĩnh và backlash trong hệ truyền động ảnh hưởng nhiều hơn. Setpoint 0 m/s (dừng) đạt hiệu suất xuất sắc với error gần như bằng 0.

Kết luận: Bộ PID đã thiết kế đạt mục tiêu điều khiển với steady-state error $< 2\%$ và overshoot $< 5\%$ ở hầu hết setpoints. Hệ thống ổn định theo phân tích Routh-Hurwitz và được xác nhận qua thực nghiệm.



Hình 3.6. Đáp ứng step response của bộ PID. Hàng trên: $v = 0.4 \text{ m/s}$ (88 units), hàng dưới: $v = -0.5 \text{ m/s}$ (-110 units). Cột trái: step response 0-1s, cột phải: zoom vào vùng steady-state (0.3-2s) để thấy dao động quanh setpoint.

Lưu ý thực tế: Khi bánh xe quay không tải (robot được nâng lên), có xuất hiện jittering nhẹ do chất lượng encoder và độ rơ (backlash) của hộp số. Đã thử nghiệm thêm bộ lọc thông thấp (low-pass filter) nhưng gây ảnh hưởng tiêu cực đến đáp ứng PID (tăng settling time và giảm khả năng tracking). Tuy nhiên, khi chạy có tải (robot trên mặt đất), hiện tượng jittering hoàn toàn biến mất do tải cơ học làm ổn định hệ truyền động. Do đó, kết quả được xem là chấp nhận được cho ứng dụng thực tế.

3.3.3 Thiết kế sensor fusion với UKF

Để ước lượng chính xác pose và vận tốc của robot, Unscented Kalman Filter (UKF) được sử dụng để kết hợp dữ liệu từ IMU (gyroscope, accelerometer) và wheel odometry. UKF được chọn vì: (1) không cần tính Jacobian như EKF - tiết kiệm công sức cho nonlinear models, (2) cho độ chính xác cao hơn thông qua unscented transform, (3) computational cost chấp nhận được với Jetson Nano (5ms/update ở 50Hz).

3.3.3.1 Sensor fusion - Đầu vào và đầu ra

UKF nhận đầu vào từ 2 sensor và tạo ra odometry đã được fusion:

Đầu vào:

- Sensor 0 - Wheel Odometry** (topic: /robot_0/odom):
 - Position: x, y (từ encoder integration)

- Orientation: θ (yaw từ differential wheel speeds)
- Velocity: v_x, v_y (computed từ wheel speeds)

2. **Sensor 1 - IMU** (topic: /robot_0/imu/data,):

- Orientation: θ (yaw từ gyroscope integration)
- Angular velocity: ω (từ gyroscope trực tiếp)
- Linear acceleration: a_x, a_y (từ accelerometer)

Đầu ra:

- Fused odometry (topic: /odometry/filtered) với pose (x, y, θ) chính xác hơn, kết hợp ưu điểm của cả encoder (chính xác về position) và IMU (chính xác về orientation, nhanh hơn)

Ví dụ cụ thể: Khi robot xoay tại chỗ, wheel encoder có thể bị trượt (slip) làm ước lượng θ sai số lớn. IMU gyroscope đo trực tiếp vận tốc góc ω không bị ảnh hưởng bởi slip. UKF kết hợp cả hai: tin tưởng IMU hơn cho orientation (θ), tin tưởng encoder hơn cho position (x, y) . Rejection thresholds được cấu hình chặt (Mahalanobis distance < 1.5) để loại bỏ outliers từ IMU spike hoặc encoder slip đột ngột.

3.3.3.2 Ý tưởng chính của UKF

Thay vì linearize hệ phi tuyến như EKF, UKF sử dụng kỹ thuật **unscented transform**: chọn một tập các điểm đại diện (gọi là **sigma points**) xung quanh state estimate hiện tại, sau đó truyền từng điểm qua hàm phi tuyến để tính mean và covariance mới. Phương pháp này cho approximation chính xác hơn nhiều so với Taylor expansion bậc nhất của EKF, đặc biệt với hệ có nonlinearity cao như differential drive robot.

3.3.3.3 Mô hình state và measurements

State vector $\mathbf{x}_t \in \mathbb{R}^{15}$ mô tả đầy đủ trạng thái robot trong không gian 15 chiều:

$$\mathbf{x}_t = [x, y, z, \text{roll}, \text{pitch}, \theta, v_x, v_y, v_z, \omega_{\text{roll}}, \omega_{\text{pitch}}, \omega, a_x, a_y, a_z]^T \quad (3.28)$$

Trong đó, với robot 2D: $z = 0$, $\text{roll} = 0$, $\text{pitch} = 0$. Các biến quan trọng: pose (x, y, θ) , vận tốc tuyến tính (v_x, v_y) , vận tốc góc ω , và gia tốc (a_x, a_y) .

Process model mô tả chuyển động robot theo differential drive kinematics (tương tự eq. 3.22). Tại mỗi bước, state được cập nhật dựa trên command velocity từ PID và thêm process noise để model uncertainty (slip, backlash):

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t) + \mathbf{w}_t \quad (3.29)$$

với $\mathbf{u}_t = [v_{\text{cmd}}, \omega_{\text{cmd}}]$ và $\mathbf{w}_t \sim \mathcal{N}(0, \mathbf{Q})$. Process noise covariance $\mathbf{Q} = \text{diag}([0.001, 0.001, \dots, 0.001])$ được chọn dựa trên thực nghiệm: pose có uncertainty thấp (odometry tương đối chính xác), velocity có uncertainty cao hơn (do slip và backlash).

Measurement noise covariance: Trọng số fusion được UKF tự động điều chỉnh dựa trên rejection thresholds:

- Wheel odometry: pose rejection threshold = 1.5, twist rejection threshold = 2.0 (tin cậy cao)
- IMU: pose rejection threshold = 1.5, twist rejection threshold = 1.2, acceleration rejection threshold = 1.5 (reject spike nghiêm ngặt)

Khi encoder và IMU đồng ý với nhau (sai số nhỏ), cả hai được tin tưởng. Khi xung đột, nguồn có measurement noise thấp hơn sẽ được ưu tiên.

3.3.3.4 Thuật toán UKF - Các bước chính

UKF hoạt động qua 3 bước lặp lại mỗi chu kỳ:

Bước 1 - Tạo sigma points: Chọn một tập điểm đại diện (sigma points) phân bố xung quanh state estimate hiện tại. Với state 6 chiều, tạo 13 sigma points. Các điểm này được chọn sao cho mean và covariance của chúng khớp chính xác với ước lượng hiện tại.

Bước 2 - Prediction: Truyền từng sigma point qua mô hình chuyển động của robot (process model), sau đó tính trung bình có trọng số để được predicted state. Khác với EKF phải linearize mô hình phi tuyến, UKF truyền trực tiếp các điểm qua hàm gốc nên giữ được độ chính xác cao hơn.

Bước 3 - Cập nhật: Khi có phép đo mới từ IMU hoặc odometry, so sánh với giá trị dự đoán để tính sai lệch. Độ lợi Kalman quyết định mức độ tin tưởng vào dự đoán hay phép đo dựa trên độ bất định của mỗi nguồn. Trạng thái được cập nhật bằng cách kết hợp dự đoán với sai lệch theo trọng số độ lợi Kalman.

3.3.4 Triển khai Cartographer SLAM

Phần này trình bày chi tiết triển khai Google Cartographer cho robot thực tế, bao gồm quy trình mapping, tinh chỉnh tham số, chuyển đổi sang chế độ localization, và các công cụ debug. Lý thuyết thuật toán Cartographer được trình bày trong Chương 2 Mục 2.5.8.

3.3.4.1 Quy trình Mapping

Quy trình tạo bản đồ (mapping) sử dụng Cartographer bao gồm các bước: cấu hình file Lua, khởi chạy ROS launch, giám sát quá trình mapping qua RViz, và lưu bản đồ dưới dạng file .pbstream.

Cấu hình file Lua: File cấu hình chính robot_2d.lua nằm trong thư mục config/cartogra. Các thiết lập quan trọng bao gồm:

Listing 3.1. Cấu hình Cartographer cho mapping


```

1 options = {
2   tracking_frame = "dummy_base_link",    -- Frame robot
3   odom_frame = "robot_0/odom",          -- Odometry tu UKF
4   use_odometry = true,                  -- Su dung UKF odom
5   num_laser_scans = 1,                  -- 1 LiDAR 360 do
6   use_imu_data = false,                  -- IMU da tích hop trong UKF
7 }
8 MAP_BUILDER.use_trajectory_builder_2d = true

```

Khởi chạy mapping: Sử dụng ROS launch để khởi động Cartographer node:

```
roslaunch robot_controller cartographer_mapping.launch
```

Trong quá trình mapping, cần điều khiển robot di chuyển chậm (tốc độ khuyến nghị 0.1-0.2 m/s) để LiDAR quét đủ chi tiết môi trường. Quan sát RViz để theo dõi các *submap* được tạo ra và trajectory của robot.

Lưu bản đồ: Sau khi mapping hoàn tất, gọi service để kết thúc trajectory và lưu file:

```
rosservice call /finish_trajectory 0
rosservice call /write_state "filename: 'map.pbstream'"
```

File .pbstream chứa toàn bộ submaps và pose graph, có thể load lại cho localization hoặc tiếp tục mapping.

3.3.4.2 Quá trình Tinh chỉnh Tham số

Việc tinh chỉnh tham số Cartographer cho phần cứng cụ thể (Jetson Nano + LiDAR LD19) đòi hỏi nhiều iteration thử nghiệm. Bảng 3.4 tổng hợp các tham số quan trọng sau quá trình tinh chỉnh.

Bối cảnh phần cứng: Jetson Nano có 4GB RAM và 4 CPU cores, LiDAR LD19 quét 360° với tần số 10Hz và 455 điểm/vòng. Thách thức chính là cân bằng giữa chất lượng bản đồ và tài nguyên tính toán giới hạn.

Iteration 1 - Vấn đề drift khi quay: Với tham số mặc định, bản đồ bị "nhai" (drift) nghiêm trọng khi robot xoay tại chỗ. Nguyên nhân: *motion_filter* quá nhạy, insert quá nhiều scan khi xoay. Giải pháp: tăng *max_angle_radians* từ 1° lên 3°.

Iteration 2 - Vấn đề scan matching backward: Robot di chuyển lùi không được match đúng. Nguyên nhân: *angular_search_window* quá hẹp (15°). Giải pháp: tăng lên 30° và giảm *rotation_delta_cost_weight* từ 10.0 xuống 1.0.

Iteration 3 - Vấn đề loop closure: Khi gọi *finish_trajectory*, optimization làm biến dạng bản đồ. Giải pháp: tắt hoàn toàn pose graph optimization bằng *optimize_every_n* = 0 và *sampling_ratio* = 0.0.

Bảng 3.4. Tham số Cartographer sau tinh chỉnh cho Jetson Nano + LiDAR LD19

Tham số	Giá trị	Lý do và ảnh hưởng
<i>Motion Filter</i>		
max_distance_meters	0.05	Update khi di chuyển 5cm, tăng độ chính xác
max_angle_radians	0.5°	Update khi xoay 0.5°, tránh drift xoay
<i>Real-time Correlative Scan Matcher</i>		
linear_search_window	0.15m	Tìm kiếm rộng 15cm, hỗ trợ di chuyển lùi
angular_search_window	30°	Tìm kiếm mọi hướng, quan trọng cho robot differential
translation_delta_cost_weight	1.0	Giảm penalty, linh hoạt hơn
rotation_delta_cost_weight	1.0	Cho phép match khi xoay nhiều
<i>Ceres Scan Matcher</i>		
occupied_space_weight	4.0	Cân bằng giữa laser và odometry
translation_weight	60.0	Tin tưởng odometry tịnh tiến
rotation_weight	40.0	Tin tưởng odometry xoay
max_num_iterations	14	Giảm để tiết kiệm CPU
<i>Submap Configuration</i>		
num_range_data	200	Submap lớn, bao phủ mọi hướng
resolution	0.04m	Độ phân giải 4cm, đủ chi tiết
<i>Pose Graph (Tắt hoàn toàn)</i>		
optimize_every_n_nodes	0	Tắt optimization để tránh phá map
sampling_ratio	0.0	Không tạo constraint mới

3.3.4.3 Quy trình Navigation/Localization

Sau khi có bản đồ (.pbstream), chuyển sang chế độ *pure localization* để robot định vị trong bản đồ có sẵn mà không tạo submap mới.

File cấu hình localization: File robot_2d_localization.lua kế thừa từ robot_2d.lua với:

Listing 3.2. Cấu hình Cartographer cho localization

```

1 include "robot_2d.lua"
2
3 -- Bật chế độ pure localization
4 TRAJECTORY_BUILDER.pure_localization = true
5
6 -- Giảm tần suất update (tiết kiệm CPU)
7 motion_filter.max_distance_meters = 0.1    -- 10cm
8 motion_filter.max_angle_radians = 2.0 deg  -- 2 độ

```

```

9
10 -- Giu pose graph o muc toi thieu
11 POSE_GRAPH.optimize_every_n_nodes = 5
12 POSE_GRAPH.constraint_builder.sampling_ratio = 0.1

```

Khởi chạy localization:

```

roslaunch robot_controller cartographer_localization.launch \
  load_state_filename:=/path/to/map.pbstream

```

Robot sẽ tự động match scan hiện tại với bản đồ và publish pose qua topic `/robot_0/tracked_pose`. Chế độ localization tiêu tốn khoảng 10-15% CPU (so với 25-30% khi mapping), phù hợp cho Jetson Nano chạy đồng thời với model inference.

3.3.4.4 Công cụ Debug và Tinh chỉnh

Cartographer cung cấp các công cụ hỗ trợ debug và kiểm tra chất lượng:

1. cartographer_rosbag_validate: Kiểm tra tính hợp lệ của rosbag trước khi chạy Cartographer:

```

roslaunch cartographer_ros cartographer_rosbag_validate \
  -bag_filename test.bag

```

Tool này kiểm tra: timestamps đồng bộ, TF tree hợp lệ, message frequency ổn định. Nếu phát hiện lỗi, cần sửa driver sensor hoặc TF publisher.

2. RViz State Visualization: Trong RViz, thêm các display:

- *Submaps*: Hiển thị từng submap với màu khác nhau, kiểm tra overlap
- *Trajectory*: Đường đi của robot, kiểm tra drift
- *Constraints*: Các constraint giữa submaps (nếu bật optimization)

Dấu hiệu bất thường: submap chồng lấn không khớp, trajectory nhảy đột ngột, constraint kéo sai hướng.

3. pbstream Inspection: Kiểm tra file bản đồ đã lưu:

```

roslaunch cartographer_ros cartographer_pbstream_map_publisher \
  -pbstream_filename map.pbstream

```

Publish bản đồ dạng OccupancyGrid để visualize trong RViz mà không cần chạy full Cartographer.

Tài liệu tham khảo chi tiết về tuning và debugging: [\[19\]](#)

3.3.5 Thiết kế an toàn trong thực nghiệm

Khi triển khai model từ mô phỏng sang robot thực tế, an toàn là yếu tố quan trọng hàng đầu. Phần này trình bày các cơ chế an toàn được thiết kế để bảo vệ robot và môi trường trong quá trình thực nghiệm.

3.3.5.1 Giới hạn tốc độ tối đa

Model được huấn luyện với tốc độ tối đa giới hạn xuống **0.3 m/s** mặc dù tối đa robot có thể chạy được với 1m/s (3.6km). Có hai lý do chính:

1. Hạn chế phần cứng:

- LiDAR LD19 được cập nhật với tần số thực tế 10Hz do vấn đề về giao tiếp với ROS và phải ưu tiên tần số cập nhật ESP vì Jetson không xử lý kịp cả 2 ở tần số cao. Với tốc độ 1.0 m/s, robot di chuyển 10cm giữa hai lần quét - khoảng cách đáng kể miss dữ liệu scan dẫn đến sai matching map
- Hệ thống ước lượng vị trí (Cartographer/UKF) không đáp ứng kịp khi robot di chuyển nhanh, gây sai lệch vị trí tương đối trong môi trường và ảnh hưởng đến local goal calculation.
- ROS network có độ trễ khoảng 50ms, cộng thêm thời gian inference (10ms) và PID response, tổng latency có thể lên đến 80-100ms.

2. An toàn trong môi trường thực nghiệm:

- Không gian thực nghiệm nhỏ (khoảng 4m x 3m) so với môi trường mô phỏng (20m x 20m).
- Có nhiều vật cản hơn mô phỏng: bàn, ghế, cáp điện, thiết bị thí nghiệm.
- Cần thời gian phản ứng đủ để dừng an toàn khi phát hiện chướng ngại vật bất ngờ
- Robot có thể mắc kẹt nhưng bánh vẫn quay trong môi trường hẹp, nên cần giảm tốc độ quay bánh.

3.3.5.2 Velocity ramping trên Arduino

Để tránh thay đổi vận tốc đột ngột gây giật cơ học và mất ổn định, Arduino được bổ sung hàm `applyRamp()` để làm mượt quá trình tăng/giảm tốc:

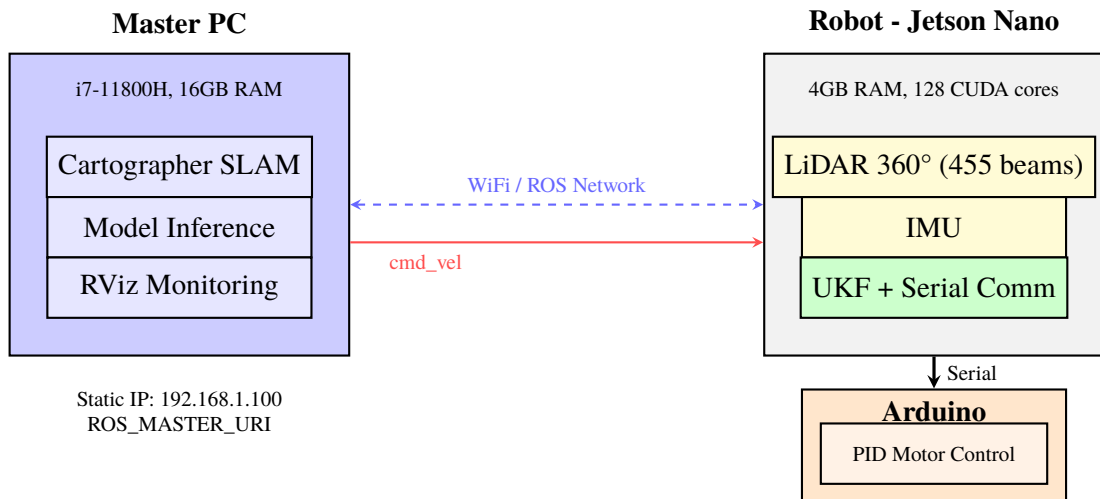
```
1 float applyRamp(float target, float current, float dt) {  
2     float speedDiff = target - current;  
3     if (abs(speedDiff) < 0.01) return target; // Close enough  
4  
5     float maxChange = maxAcceleration * dt; // 2.5 m/s2 default  
6  
7     if (speedDiff > maxChange) return current + maxChange;  
8     if (speedDiff < -maxChange) return current - maxChange;  
9     return target; // Can reach target in one step  
10 }
```

Với $\text{maxAcceleration} = 2.5 \text{ m/s}^2$, robot mất khoảng 120ms để tăng từ 0 lên 0.3 m/s, đảm bảo chuyển động mượt mà và giảm tải cho hộp số bánh xe.

3.3.6 Triển khai hệ robot hoàn chỉnh

Phần này trình bày kiến trúc hệ thống hoàn chỉnh bao gồm phần cứng, cấu hình mạng ROS, phân bổ các module, và giải thích cách hệ thống đảm bảo tính phi tập trung mặc dù inference được thực hiện tập trung.

3.3.6.1 Kiến trúc tổng thể



Hình 3.7. Kiến trúc hệ thống robot hoàn chỉnh

Hệ thống bao gồm 1 Master PC và 1 robot tự hành (robot có 1 Jetson Nano làm bộ xử lý chính). Kiến trúc được thiết kế theo nguyên tắc **decentralized** về mặt algorithm, trong đó mỗi robot đưa ra quyết định độc lập dựa trên quan sát cục bộ của chính nó, không có giao tiếp giữa các robot.

3.3.6.2 Cấu hình phần cứng

Bảng 3.5 liệt kê cấu hình phần cứng của các thành phần chính trong hệ thống.

Bảng 3.5. Cấu hình phần cứng hệ thống

Thiết bị	Thông số	Vai trò
Master PC	Intel i7-11800H 16GB RAM GTX-3050, ROS Noetic Ubuntu 20.04	Cartographer SLAM Model inference ROS Master node
Jetson Nano	4GB RAM Maxwell 128 CUDA cores Ubuntu 18.04, ROS Melodic	Lấy dữ liệu sensor Fuse sensor với UKF giao tiếp Serial
Arduino ESP32	Điều khiển động cơ PID 16 MHz	Đọc vận tốc bánh xe
LiDAR 360°	Quét 360° Độ phân giải 455 tia	Phát hiện vật cản SLAM
IMU	9-axis I2C interface	Ước lượng vị trí
Động cơ DC	Encoder 15000 pulses/rev Tỷ số truyền 1:20	Điều hướng robot

3.3.6.3 Cấu hình mạng ROS

Hệ thống sử dụng ROS network qua WiFi với cấu hình tĩnh (static IP) để đảm bảo kết nối ổn định. Master PC đóng vai trò ROS Master node, quản lý tất cả topics và services.

Cấu hình IP tĩnh:

- Master PC: 192.168.1.100 (ROS_MASTER_URI = http://192.168.1.100:11311)
- Robot 1 (Jetson Nano): 192.168.1.101
- Robot 2 (Jetson Nano): 192.168.1.102

Mỗi máy cần export biến môi trường ROS:

```
export ROS_MASTER_URI=http://192.168.1.100:11311
export ROS_IP=<địa chỉ IP của máy hiện tại>
```

3.3.6.4 Phân bổ các ROS nodes

Bảng 3.6 mô tả phân bổ các ROS nodes trên các thiết bị khác nhau.

Bảng 3.6. Phân bố ROS nodes trên các thiết bị

Node	Chạy trên	Topics chính
cartographer_node	Master PC	Subscribe: /robot_X/scan, /robot_X/imu Publish: /robot_X/amcl_pose
policy_node	Master PC	Subscribe: /robot_X/scan, /robot_X/amcl_pose Publish: /robot_X/cmd_vel
lidar_node	Jetson Nano	Publish: /robot_X/scan
imu_node	Jetson Nano	Publish: /robot_X/imu
ukf_node	Jetson Nano	Subscribe: /robot_X/imu, /robot_X/cmd_vel Publish: /robot_X/odom
serial_node	Jetson Nano	Subscribe: /robot_X/cmd_vel Serial output: Arduino
PID Controller	Arduino	Serial input: target velocity PWM output: motor driver

Luồng dữ liệu chính:

1. LiDAR và IMU trên Jetson publish dữ liệu cảm biến qua ROS topics
2. Master PC nhận dữ liệu cảm biến, chạy Cartographer để ước lượng vị trí
3. Policy node trên Master PC inference action (v, ω) từ observations
4. Master publish cmd_vel về Jetson qua mạng ROS
5. Jetson forward cmd_vel xuống Arduino qua serial (USB)
6. Arduino chạy PID controller và điều khiển động cơ qua PWM

Tần số điều khiển: **20 Hz** (mỗi 50ms một control cycle), đảm bảo real-time response cho việc tránh va chạm.

3.3.6.5 Thiết kế phi tập trung với inference tập trung

Khái niệm inference tập trung: Inference tập trung (centralized inference) là kiến trúc trong đó tất cả các tính toán nặng (như mạng nơ-ron inference) được thực hiện trên một máy chủ trung tâm (Master PC) thay vì phân tán trên từng robot. Sensor data từ các robot được gửi về Master PC qua mạng, Master PC xử lý và gửi lại kết quả (action commands) cho từng robot.

Phân biệt thiết kế và triển khai:

- **Về mặt thuật toán (Design):** Hệ thống được thiết kế phi tập trung hoàn toàn. Mỗi robot quyết định hành động CHỈ dựa trên quan sát cục bộ của chính nó (LiDAR scan, vị trí đích, vận tốc). Không có giao tiếp giữa các robot - Robot B được Robot A nhận biết như một vật cản động qua LiDAR, tương tự như tường hay vật cản khác.
- **Về mặt triển khai (Implementation):** Inference chạy tập trung trên Master PC do hạn chế phần cứng của Jetson Nano (4GB RAM, GPU yếu). Model inference trên Jetson mất 50-100ms, so với 10ms trên Master PC i7 - latency quá cao.

Khả năng mở rộng:

Code không có phụ thuộc nào với Master PC về mặt thiết kế. Với phần cứng mạnh hơn (Jetson Orin 32GB RAM, hoặc mini PC), toàn bộ pipeline (SLAM + inference) có thể chạy hoàn toàn độc lập trong từng robot mà không cần thay đổi code - chỉ cần sửa launch file để nodes chạy local. Đây là ưu điểm của kiến trúc ROS.

3.3.6.6 Tóm tắt deployment

Hệ thống triển khai hoàn chỉnh bao gồm:

- Kiến trúc Master-Slave với ROS network qua WiFi (static IP)
- Master PC (i7-11800H, 15GB RAM) chạy Cartographer SLAM và model inference
- 1 robot với Jetson Nano xử lý sensors, UKF fusion, và giao tiếp serial
- Arduino chạy bộ điều khiển PID cho điều khiển động cơ
- Tần số điều khiển 20 Hz đảm bảo tránh va chạm real-time
- Thiết kế decentralized algorithm với centralized inference
- Khả năng mở rộng: có thể chuyển sang fully decentralized deployment với việc nâng cấp phần cứng

CHƯƠNG 4

KẾT QUẢ VÀ THẢO LUẬN

Chương này trình bày chi tiết kết quả huấn luyện mô hình trong môi trường mô phỏng (simulation) qua hai giai đoạn Stage 1 và Stage 2, kết quả thực nghiệm trên robot thực tế, phân tích các hạn chế, và thảo luận tổng hợp.

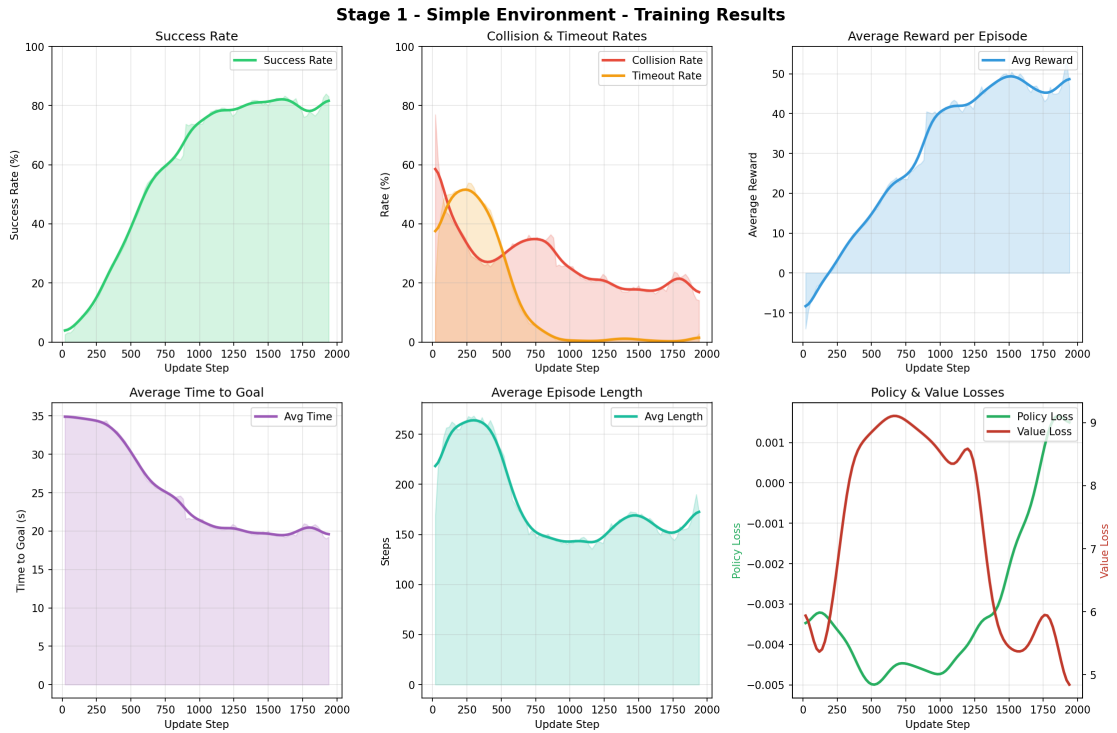
4.1 Kết quả mô phỏng

Quá trình huấn luyện được chia thành hai giai đoạn (stages) theo phương pháp curriculum learning: Stage 1 với môi trường đơn giản để học các kỹ năng cơ bản, và Stage 2 với môi trường phức tạp để tinh chỉnh và nâng cao hiệu suất.

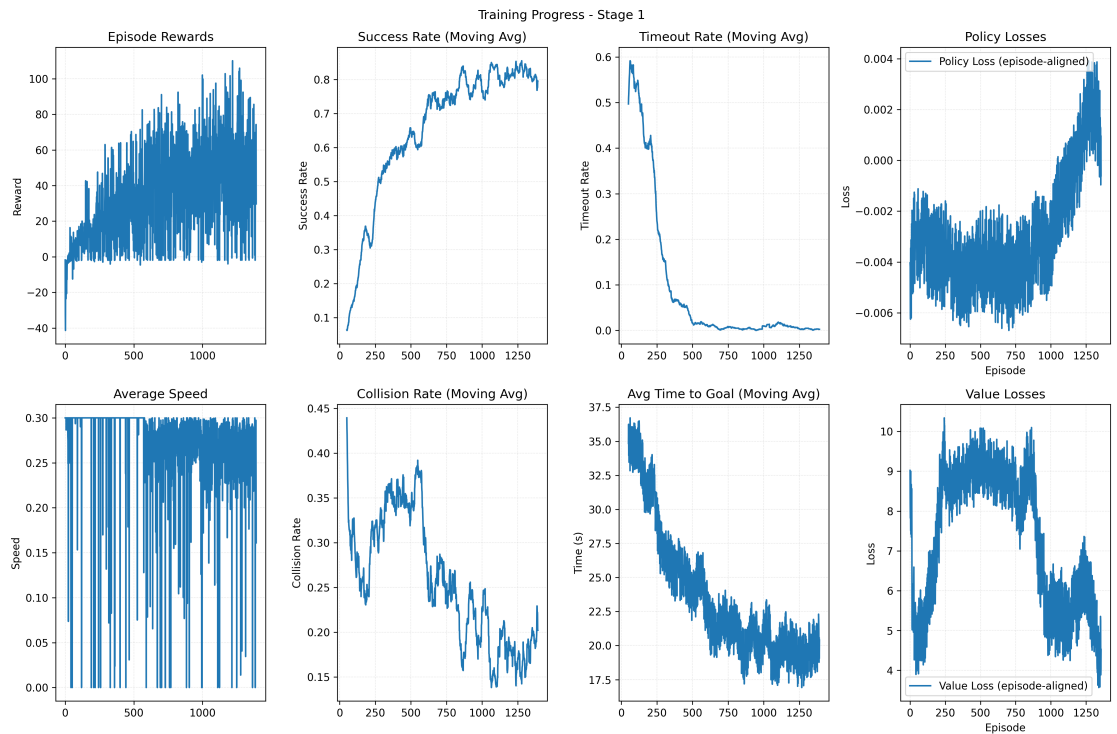
4.1.1 Kết quả Stage 1 - Môi trường đơn giản

Thiết lập thí nghiệm: Stage 1 sử dụng môi trường mô phỏng đơn giản với 24 robots, có 4 vật cản tĩnh (các robots như vật cản động). Quá trình huấn luyện kéo dài qua 1940 policy updates trong vòng 14 giờ (chạy không liên tục).

Tiến trình huấn luyện: Hình 4.1 biểu diễn kết quả huấn luyện ở Stage 1 qua các metrics chính: success rate, collision rate, và average reward, đã được làm mượt theo phương pháp trung bình và đặt chồng lên nhau để dễ so sánh. Hình 4.2 biểu diễn lại dữ liệu training gốc của các metrics.



Hình 4.1. Stage 1: (a) Success rate tăng từ 2.57% lên 83.03%, (b) Collision rate giảm từ 76.93% xuống 14.11%, (c) Average reward tăng từ -13.97 lên 46.87.



Hình 4.2. Dữ liệu metrics huấn luyện Stage 1 chưa qua xử lý

Bảng 4.1 tổng hợp các metrics quan trọng tại các mốc huấn luyện chính:

Bảng 4.1. Metrics huấn luyện Stage 1 qua các mốc chính

Update	Success Rate (%)	Collision Rate (%)	Avg Reward	Timeout (%)
20 (Initial)	2.57	76.93	-13.97	20.49
200	10.76	37.97	0.21	51.28
400	28.02	26.49	10.55	45.49
600	51.78	32.98	20.84	15.24
800	61.51	34.86	25.69	3.63
1000	73.84	25.63	40.59	0.53
1200	78.78	20.87	42.38	0.35
1400	81.84	17.00	47.47	1.16
1620 (Best)	83.26	16.34	47.95	0.40
1940 (Final)	83.03	14.11	46.87	2.85

Phân tích kết quả Stage 1:

- **Giai đoạn khởi đầu (0-200 updates):** Robots hoạt động gần như ngẫu nhiên với success rate chỉ 2.57% và collision rate cao 76.93%. Timeout rate cao (51.28% tại update 200) cho thấy robots chưa học được cách di chuyển hiệu quả.
- **Giai đoạn học nhanh (200-600 updates):** Success rate tăng mạnh từ 10.76% lên 51.78%, collision rate giảm đáng kể. Đây là giai đoạn policy bắt đầu học được behavior cơ bản: hướng về goal và tránh va chạm.
- **Giai đoạn tinh chỉnh (600-1200 updates):** Success rate tiếp tục tăng nhưng chậm hơn, collision rate giảm dần. Timeout rate giảm mạnh từ 15.24% xuống dưới 1%, cho thấy robots đã học được cách di chuyển nhanh và hiệu quả hơn.
- **Giai đoạn bão hòa (1200-1940 updates):** Success rate dao động quanh 80-83%, collision rate ổn định ở 14-17%. Policy đã đạt ngưỡng ổn định cho môi trường Stage 1.

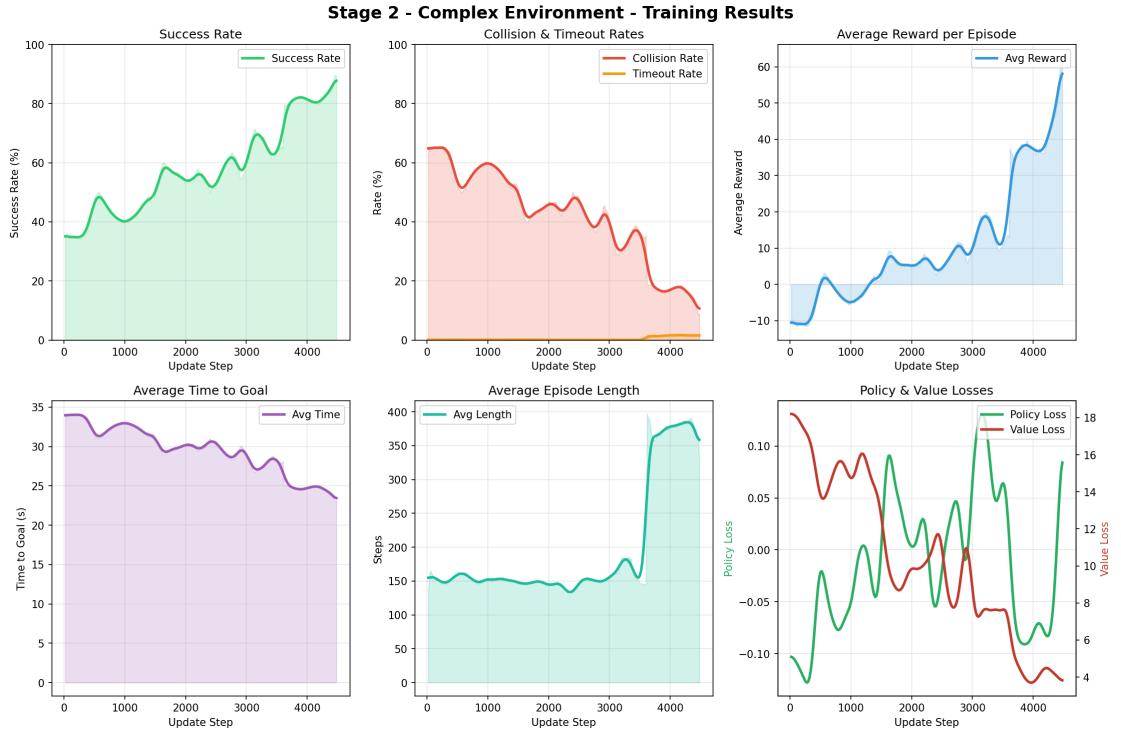
Kết quả đạt được:

- Success rate tối đa: **83.26%** (tại update 1620)
- Collision rate tối thiểu: **14.11%**
- Average reward tối đa: **53.47**
- Thời gian trung bình đến goal: **12.99 seconds**

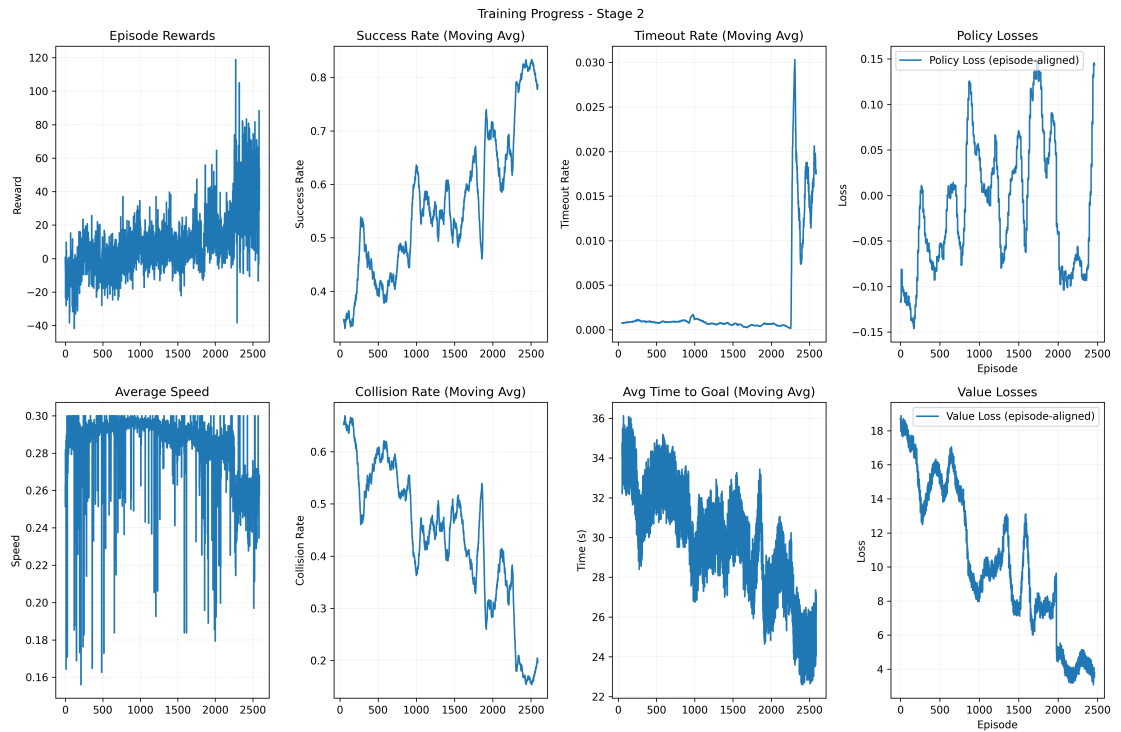
4.1.2 Kết quả Stage 2 - Môi trường phức tạp

Thiết lập thí nghiệm: Stage 2 tiếp tục huấn luyện từ checkpoint tốt nhất của Stage 1, sử dụng môi trường phức tạp hơn với các scenarios như đã trình bày. Quá trình huấn luyện kéo dài qua 4480 updates trong vòng 30 giờ (thời gian có bị kéo dài hơn do có tăng timeout limit).

Tiến trình huấn luyện: Hình 4.3 và hình 4.4 biểu diễn kết quả huấn luyện ở Stage 2.



Hình 4.3. Stage 2: (a) Success rate tăng từ 34.66% lên 89.18%, (b) Collision rate giảm từ 65.28% xuống 9.04%, (c) Average reward tăng từ -10.36 lên 62.46.



Hình 4.4. Dữ liệu metrics huấn luyện Stage 2 chưa qua xử lý

Bảng 4.2 tổng hợp các metrics quan trọng tại các mốc huấn luyện chính của Stage

2:

Bảng 4.2. Metrics huấn luyện Stage 2 qua các mốc chính

Update	Success Rate (%)	Collision Rate (%)	Avg Reward	Timeout (%)
20 (Initial)	34.66	65.28	-10.36	0.07
500	47.84	52.06	1.54	0.10
1000	40.09	59.83	-5.00	0.08
2000	53.12	46.82	4.75	0.06
3000	59.87	40.07	9.67	0.06
4000	81.56	16.82	37.75	1.62
4480 (Final)	89.18	9.04	62.46	1.78

Phân tích kết quả Stage 2:

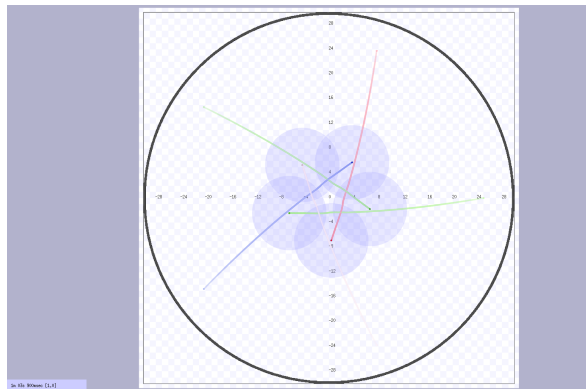
- **Transfer learning hiệu quả:** Mặc dù môi trường Stage 2 phức tạp hơn đáng kể, policy chuyển giao từ Stage 1 vẫn đạt success rate 34.66% ngay từ đầu, cho thấy các kỹ năng cơ bản đã được học tốt.
- **Giai đoạn thích nghi (0-1000 updates):** Giai đoạn đầu có biến động lớn do policy đang thích nghi với môi trường phức tạp hơn. Success rate dao động quanh 40-48%, collision rate vẫn cao (52-60%). Average reward tăng nhẹ từ -10.36 lên khoảng 1.54 rồi giảm xuống -5.00, phản ánh sự bất ổn trong quá trình học.
- **Giai đoạn cải thiện (1000-3000 updates):** Success rate tăng dần từ 40% lên 60%, collision rate giảm từ 60% xuống 40%. Average reward tăng từ -5.00 lên 9.67. Policy đã bắt đầu học được cách kết hợp tránh vật cản tĩnh và động.
- **Vấn đề và điều chỉnh timeout (sau 3000 updates):** Tại khoảng 3000 updates, kết quả training bắt đầu có dấu hiệu suy giảm - collision rate tăng dù success rate vẫn đang tăng. Nguyên nhân được xác định là robots đang phát triển hành vi “rushing” - cố gắng di chuyển nhanh và thẳng nhất do bị timeout penalty. Hành vi này phản ánh việc policy đang tối ưu hóa sai mục tiêu: thay vì tìm đường đi an toàn, robots học cách đi thẳng về goal để tránh phạt timeout. Checkpoint tốt nhất tại update 3000 được lưu lại, sau đó timeout limit được tăng từ 500 lên 700 steps, điều này giải thích cho biểu đồ thời gian bị giết mạnh ở khoảng sau 3000.
- **Giai đoạn tăng tốc (3000-4480 updates):** Sau khi điều chỉnh timeout, training được tiếp tục từ checkpoint tốt nhất. Đây là giai đoạn cải thiện mạnh mẽ nhất: success rate tăng vọt từ 60% lên 89.18%, collision rate giảm mạnh từ 40% xuống 9.04%. Average reward tăng đáng kể từ 9.67 lên 62.46. Robot thể hiện hành vi thận trọng hơn: sẵn sàng đi đường vòng để tránh va chạm thay vì lao thẳng về goal. Đây là giai đoạn tinh chỉnh quan trọng giúp policy cân bằng tốt hơn giữa hiệu quả (đến đích nhanh) và an toàn (tránh va chạm).

Kết quả đạt được:

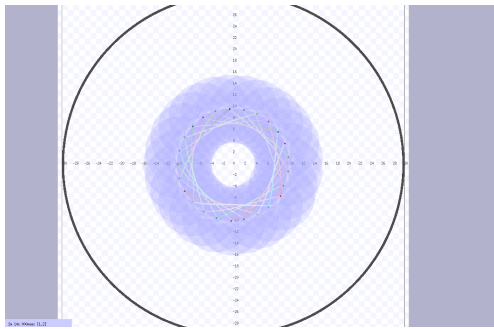
- Success rate tối đa: **89.72%** (cuối cùng: 89.18%)
- Collision rate tối thiểu: **8.72%** (cuối cùng: 9.04%)
- Average reward tối đa: **62.46**

4.1.3 So sánh và phân tích

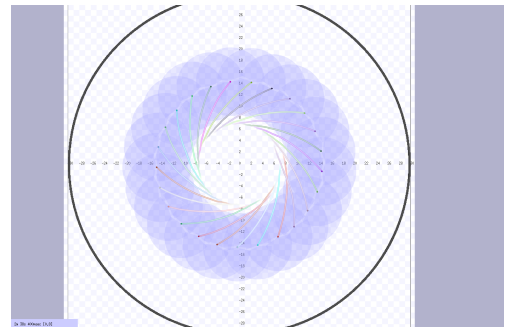
Để đánh giá chính xác hiệu suất của model sau khi huấn luyện, kiểm tra độc lập model tốt nhất sau stage 2 với kịch bản circle Test: các robots được đặt trên vòng tròn và điều hướng đến vị trí đối diện. Đây là kịch bản thách thức vì tất cả robots phải đi qua tâm vòng tròn đồng thời.



Hình 4.5. Circle Test với 5 robots: Các robots (chấm màu) di chuyển từ vị trí ban đầu đến goal đối diện. Đường màu thể hiện quỹ đạo di chuyển, vùng tròn màu xanh là phạm vi quan sát của mỗi robot.



(a) Giai đoạn đầu (2m14s)



(b) Giai đoạn cuối (2m38s)

Hình 4.6. Circle Test với 24 robots: (a) Robots bắt đầu di chuyển từ vòng tròn bán kính 10m, tạo thành pattern xoắn ốc để tránh va chạm tại tâm; (b) Robots hoàn thành di chuyển đến vị trí đối diện với quỹ đạo xoắn ốc đặc trưng của thuật toán tránh va chạm.

Bảng 4.3 tổng hợp kết quả Circle Test với số lượng robots khác nhau:

Bảng 4.3. Kết quả Circle Test theo số lượng robots

Robots	Episodes	Success Rate (%)	Collision Rate (%)	Avg Speed (m/s)
4	10	100.0	0.0	0.226
8	10	100.0	0.0	0.156
12	10	95.0	5.0	0.158
20	10	81.5	18.5	0.164

Phân tích hiệu suất theo mật độ robots:

Bảng 4.4. Mối quan hệ giữa khoảng cách inter-robot và success rate

Số robots	Khoảng cách giữa robots (m)	Success Rate (%)
4	~6.28	100
8	~3.14	100
12	~2.09	95
20	~1.26	81.5

Nhận xét: Model duy trì hiệu suất xuất sắc (100%) khi khoảng cách giữa các robots > 3m. Khi mật độ tăng (khoảng cách < 2m), success rate bắt đầu giảm do tăng xác suất deadlock và va chạm liên hoàn.

Bảng 4.5 so sánh kết quả với bài báo gốc CADRL và phương pháp NH-ORCA (Non-Holonomic Optimal Reciprocal Collision Avoidance) [10]:

Bảng 4.5. So sánh kết quả Circle Test với bài báo gốc CADRL và NH-ORCA

Số robots	NH-ORCA (%)	Paper CADRL (%)	Model này (%)
4	~97	~100	100.0
8	~92	~100	100.0
12	~78	~97	95.0
20	~65	~90	81.5

Phân tích sự khác biệt:

- So sánh với NH-ORCA:** Model này vượt trội hơn NH-ORCA ở tất cả các mức độ mật độ. Sự chênh lệch rõ rệt nhất ở 20 robots (81.5% vs 65%), cho thấy deep RL xử lý tốt hơn các tình huống đông đúc so với phương pháp geometry-based.
- Kết quả tương đương CADRL ở mật độ thấp:** Với 4-8 robots, model đạt 100% success rate, tương đương với bài báo gốc CADRL. Điều này xác nhận policy đã học được cách tránh va chạm đơn giản.
- Gap ở mật độ cao (12-20 robots):** Sự chênh lệch 2-8.5% có thể được giải thích bởi:
 - Setup mô hình robot khác nhau giữa 2 nghiên cứu:

- Số lượng tia lidar và range của chúng khác nhau (180 độ 512 tia vs 360 độ 454 tia)
 - Tốc độ tối đa khác nhau (1 m/s vs 0.3 m/s)
 - Khác biệt về reward function và hyperparameters
4. **Curriculum learning hiệu quả:** Việc chuyển từ Stage 1 sang Stage 2 cho thấy hiệu quả của curriculum learning:
- Stage 1 giúp học các kỹ năng cơ bản (navigation, tránh va chạm cơ bản)
 - Stage 2 tinh chỉnh cho môi trường phức tạp với static obstacles
 - Transfer learning hoạt động tốt: policy Stage 1 đạt 34.66% ngay lập tức trong Stage 2
5. **Tốc độ di chuyển:** Average speed 0.15-0.23 m/s (50-77% max velocity 0.3 m/s) cho thấy policy ưu tiên an toàn hơn tốc độ, đặc biệt trong môi trường đông đúc.

4.2 Kết quả thực nghiệm

Phần này trình bày kết quả triển khai model đã huấn luyện trên hệ thống robot thực tế (hardware deployment). Các thí nghiệm được thực hiện với 1 robot hoạt động trong môi trường trong nhà.

4.2.1 Thiết lập thực nghiệm

Phần cứng:

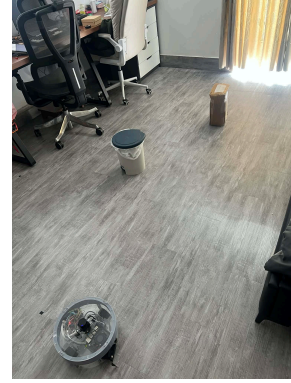
- 1 robot differential drive với Jetson Nano
- LiDAR LD19 (360°, 455 beams)
- Master PC: Intel i7-11800H, 16GB RAM
- Giao tiếp: ROS network qua mạng

Môi trường test:

- Phòng kích thước khoảng 4m x 3m
- Có nhiều vật cản tĩnh (bàn, ghế, tường, giường)



(a) Môi trường thực nghiệm



(b) Môi trường với vật cản

Hình 4.7. Môi trường thực nghiệm trong nhà (4m x 3m)

Metrics đánh giá:

- Success rate: Tỷ lệ robot đến được goal không va chạm
- Collision rate: Tỷ lệ episodes có va chạm
- Average time to goal: Thời gian trung bình hoàn thành nhiệm vụ
- Path efficiency: Tỷ số giữa khoảng cách Euclidean đến đích và đường đi thực tế

4.2.2 Kết quả thực nghiệm

Thực nghiệm được thực hiện với 17 test cases, chia thành 4 nhóm tình huống với độ khó tăng dần: (1) không có vật cản, (2) vật cản tĩnh, (3) 1 người động, và (4) 2 người động. Bảng 4.6 tổng hợp kết quả theo từng nhóm.

Bảng 4.6. Tổng hợp kết quả thực nghiệm theo nhóm các trường hợp

Nhóm scenarios	Số test	Success	Timeout	Success Rate	Avg Distance Err
0 obstacles (không vật cản)	5	5	0	100%	0.08
1-2 vật tĩnh	6	5	1	83.3%	0.11
1 người động	4	3	1	75%	0.25
2 người động	2	1	1	50%	0.29
Tổng cộng	17	14	3	82.4%	0.14

4.2.2.1 Kết quả không có vật cản

Robot đạt **100% success rate** (5/5 tests) khi di chuyển đến goal mà không có vật cản. Khoảng cách goal trung bình là 2.03m với sai số vị trí cuối trung bình 0.08m. Kết quả này xác nhận policy hoạt động tốt cho bài toán navigation cơ bản và hệ thống localization (Cartographer SLAM + UKF) cung cấp vị trí đủ chính xác.

Bảng 4.7. Chi tiết kết quả test không có vật cản

Test	x_g	y_g	x_r	y_r	Goal Dist (m)	Steps	Reward	Error (m)
1	0.96	0	0.96	0.01	0.96	110	44.4	0.01
2	1.99	0.33	2.05	0.25	1.08	100	41.6	0.10
3	2.41	0.4	2.38	0.34	2.44	250	46.2	0.07
4	1.98	-0.38	2.08	-0.35	2.02	215	48.7	0.10
5	2.56	-0.74	2.45	-0.77	2.66	203	50.65	0.11
Trung bình					1.83	176	46.3	0.08

Chú thích: (x_g, y_g) là tọa độ goal, (x_r, y_r) là tọa độ thực tế robot đạt được (đơn vị: m)

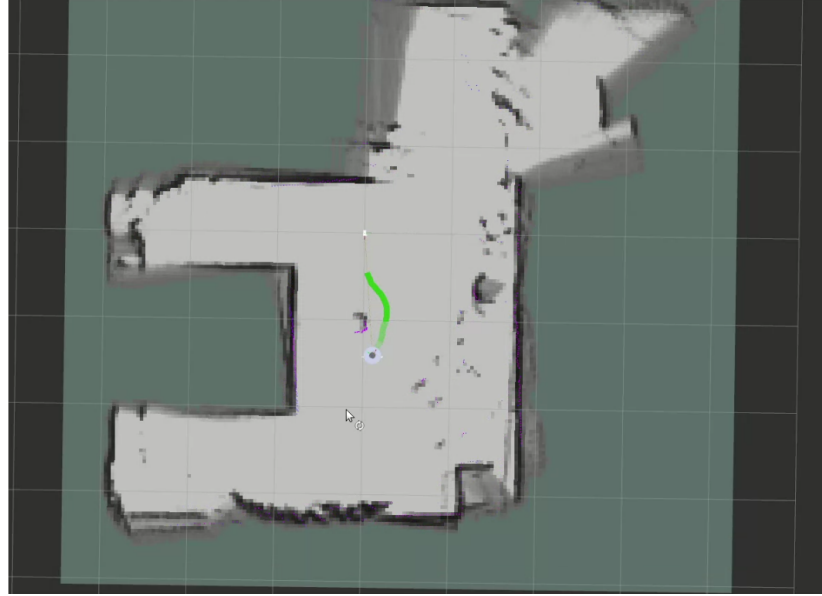
4.2.2.2 Kết quả với vật cản tĩnh

Robot đạt **83.3% success rate** (5/6 tests) với vật cản tĩnh. Các trường hợp thành công cho thấy robot có khả năng tránh vật cản với đường đi mượt mà không bị chuyển hướng đột ngột, đặc biệt khi vật cản đặt ở vị trí dễ quan sát.

Bảng 4.8. Chi tiết kết quả test với vật cản tĩnh

#	Scenario	x_g	y_g	x_r	y_r	Dist	Steps	Reward	Error	Status
6	1 vật cản trung điểm	2.29	0.35	2.24	0.43	2.32	190	45.4	0.09	reach goal
7	1 vật cản gần robot	1.98	-0.02	2.01	0.05	1.98	176	44.37	0.08	reach goal
8	1 vật cản, goal gần	1.35	0.45	1.37	0.45	0.77	144	42.8	0.02	reach goal
9	2 vật thẳng hàng, lệch phải	0.04	-0.22	0.03	-0.11	1.52	457	55.39	0.11	reach goal
10	2 vật thẳng hàng	0.11	0.05	0.11	0.03	0.12	230	48.5	0.02	reach goal
11	2 vật lệch nhau	0.36	-0.04	1.25	0.22	1.57	500	5.71	0.36	timeout

Nhận xét: Test 11 thất bại do robot bị dừng và dao động trước vật cản thứ 2 và không tìm được đường tránh. Đây là hạn chế của policy khi gặp cấu hình vật cản phức tạp (2 vật cản lệch nhau tạo thành hành lang hẹp).



Hình 4.8. Robot tránh vật cản tĩnh: trajectory màu xanh lá cho thấy robot đi vòng qua vật cản để đến goal

4.2.2.3 Kết quả với vật cản động (người)

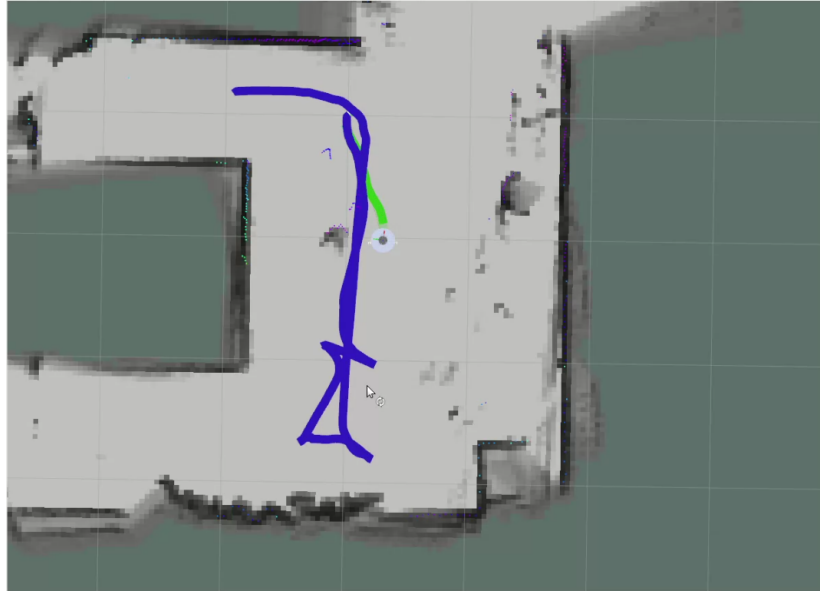
Đây là nhóm test quan trọng nhất để đánh giá khả năng tránh va chạm trong môi trường thực tế. Robot đạt **75% success rate** (3/4 tests) với 1 người và **50%** (1/2 tests) với 2 người.

Bảng 4.9. Chi tiết kết quả test với vật cản động (người)

#	Scenario	x_g	y_g	x_r	y_r	Dist	Steps	Reward	Error	Status
12	1 người đi chậm ngang qua	1.95	0	1.87	0.02	1.95	208	42.8	0.08	reach goal
13	1 người đi rồi đứng lại	1.97	0	1.88	0.01	1.97	310	50.8	0.09	reach goal
14	1 người đi nhanh qua	2.04	-0.02	1.96	0.02	2.04	270	45.2	0.09	reach goal
15	1 người đứng chắn đường	1.98	0	1.24	0.05	1.98	500	6.6	0.74	timeout
16	2 người đi từng người	2.25	0	2.16	0.03	2.25	280	51.78	0.10	reach goal
17	2 người, 1 đứng lại	2.17	0	1.7	0.08	2.17	500	12.8	0.47	timeout

Phân tích hành vi robot:

- **Người đi chuyển:** Robot giảm tốc và chờ người đi qua, sau đó tiếp tục (Tests 12, 14, 16). Hành vi này hợp lý và an toàn.
- **Người đứng yên chắn đường:** Robot gặp khó khăn, dao động qua lại nhưng không tìm được đường tránh (Tests 15, 17). Đây là hạn chế của policy và gap giữa việc huấn luyện trên mô phỏng và thực tế.
- **Va chạm nhẹ:** Test 13 ghi nhận bánh xe có đụng nhẹ vào ngón chân người - mức va chạm này không gây nguy hiểm. Điều này có thể lí giải được, do lidar đặt trên cao và bàn chân nằm dưới khả năng phát hiện của nó, nên thông tin robot tiếp nhận hoàn toàn không có nên policy không phản ánh việc xử lý.



Hình 4.9. Robot tránh vật cản động (người): đường xanh dương là quỹ đạo khi dựng map, đường xanh lá là quỹ đạo hiện tại. Hai hình chữ V là chân người được LiDAR phát hiện.

4.2.2.4 So sánh với kết quả mô phỏng

Bảng 4.10. So sánh kết quả thực nghiệm với mô phỏng

Metric	Mô phỏng (Stage 2)	Robot thực tế
Success Rate (%)	89.18	82.4
Timeout Rate (%)	1.78	17.6
Collision Rate (%)	9.04	0
Số lượng test	4480	17
Tốc độ tối đa (m/s)	0.7	0.3

Nhận xét:

- Success rate thực tế (82.4%) thấp hơn mô phỏng (89.18%), chủ yếu do timeout với vật cản đứng yên.
- Không có collision trong 17 tests thực nghiệm (0% collision rate) so với 9.04% trong mô phỏng. Robot có xu hướng dừng lại thay vì đâm vào vật cản, hành vi này an toàn trong môi trường thực tế.
- Timeout rate cao đối với những trường hợp phức tạp và hành lang hẹp
- Robot có hành vi dao động rõ rệt khi gặp những trường hợp khó

4.3 Thảo luận

4.3.1 Hiệu quả của curriculum learning

Phương pháp curriculum learning (Stage 1 $\rightarrow$ Stage 2) mang lại hiệu quả rõ rệt. Policy được huấn luyện trong môi trường đơn giản trước (Stage 1) có khả năng transfer tốt sang môi trường phức tạp (Stage 2), đạt success rate 34.66% ngay lập tức mà không cần huấn luyện lại từ đầu. Điều này khẳng định các kỹ năng cơ bản (navigation, simple avoidance) được học trong Stage 1 có tính tổng quát cao.

Vấn đề timeout được phát hiện trong Stage 2 (xem Mục 4.1.2) cho thấy tầm quan trọng của việc thiết kế reward function phù hợp với độ phức tạp của môi trường. Khi timeout limit không đủ, policy có xu hướng học hành vi "rushing" thay vì tìm đường đi an toàn.

4.3.2 Khả năng mở rộng số lượng robots

Nghiên cứu hiện tại sử dụng 44 robots trong Stage 2, mặc dù success rate thấp hơn một phần do độ khó tăng (89% vs 96.5-100%), kết quả cho thấy phương pháp tránh va chạm phi tập trung có khả năng scale. Mỗi robot chỉ quan sát môi trường cục bộ và quyết định độc lập, không cần coordination tập trung - điều này cho phép hệ thống mở rộng về số lượng mà không tăng chi phí phần cứng trung tâm.

So sánh với NH-ORCA (Bảng 4.5) cho thấy deep RL vượt trội hơn phương pháp geometry-based, đặc biệt ở mật độ cao. NH-ORCA giả định các robots khác sẽ phối hợp theo cùng một protocol, trong khi RL policy học được cách phản ứng với behavior thực tế của các agents xung quanh.

4.3.3 Từ mô phỏng đến thực tế (Sim-to-real)

Việc triển khai model từ simulation lên robot thực tế cho thấy sim-to-real khả thi với sự hỗ trợ các component chính:

- Cartographer SLAM cung cấp localization chính xác trong môi trường thực
- UKF sensor fusion giúp ước lượng state ổn định hơn so với raw odometry
- Các cơ chế an toàn bù đắp cho việc sai khác giữa thực nghiệm và mô phỏng:
 - *Velocity limiting*: Giới hạn vận tốc tối đa 0.3 m/s để robot có đủ thời gian phản ứng với độ trễ của hệ thống
 - *Safety check*: Kiểm tra khoảng cách tối thiểu đến vật cản trước khi thực hiện lệnh di chuyển, tự động dừng nếu thấy robot đã chạm vật cản (khoảng cách an toàn = bán kính robot / 2)
 - *Stuck detection*: Phát hiện robot bị kẹt khi vận tốc thực tế thấp hơn nhiều so với lệnh trong một khoảng thời gian (trường hợp bánh xe vẫn quay nhưng

vị trí robot không đổi) - cần dừng khẩn cấp

- *Recovery behavior*: Khi phát hiện bị kẹt, và trong giới hạn an toàn robot tự động lùi lại một đoạn ngắn và thử hướng di chuyển khác

Tốc độ giảm là đánh đổi cần thiết để đảm bảo an toàn, bù đắp cho độ trễ mạng và tần số LiDAR thấp (10Hz).

4.3.4 Hạn chế của nghiên cứu

Về môi trường mô phỏng:

- Stage 2D không tái hiện hoàn toàn các yếu tố vật lý: trượt bánh, độ trễ sensor, nhiễu đo lường
- Khác biệt giữa mô phỏng và thực tế, đòi hỏi các biện pháp bổ sung khi triển khai thực tế

Về phần cứng:

- Jetson Nano (4GB RAM) buộc inference tập trung trên Master PC thay vì chạy local
- Thông tin từ lidar không được đọc với tần số cao do vấn đề về độ trễ trong giao tiếp và phần cứng Jetson
- Độ trễ giao tiếp ROS (50-100ms tổng) ảnh hưởng phản hồi real-time
- Gầm xe thấp và bánh xe thiếu độ bám khiến robot không di chuyển được trên một số bề mặt (thảm, sàn trơn) do bị trượt bánh

Về định vị (localization):

- **Vấn đề drift (trôi vị trí):** Hệ thống gặp hiện tượng vị trí ước lượng bị trôi dần theo thời gian do sai số tích lũy từ wheel odometry. Nguyên nhân chính bao gồm: bánh xe trượt trên bề mặt nhẵn, sai số encoder tích lũy, và IMU giá rẻ có bias drift đáng kể.
- **So sánh các phương pháp localization:**
 - *Pure odometry*: Drift nhanh, không sử dụng được sau vài phút di chuyển
 - *UKF sensor fusion (odometry + IMU)*: Cải thiện hơn nhưng vẫn drift do IMU bias
 - *Cartographer SLAM*: Ổn định hơn đáng kể nhờ loop closure và scan matching, tuy nhiên vẫn có thể drift trong môi trường ít đặc trưng (feature-poor) hoặc hành lang dài
- **Giải pháp hiện tại:** Sử dụng Cartographer SLAM làm nguồn localization chính. Trong môi trường thực nghiệm (4m x 3m) với nhiều vật cản, Cartographer hoạt động ổn định và đạt mức chấp nhận được cho việc tránh va chạm.
- **Hạn chế còn tồn tại:** Việc cải thiện độ chính xác localization (ví dụ: sensor fusion với camera depth, IMU cao cấp hơn, hoặc visual odometry) nằm ngoài phạm vi nghiên cứu hiện tại. Đây là hướng cải tiến quan trọng cho các nghiên

cứu tiếp theo.

Về thuật toán:

- Policy chưa xử lý tốt trường hợp vật cản đứng yên chắn đường - robot có xu hướng dao động qua lại thay vì tìm đường né. Đây là hạn chế do policy được train chủ yếu với vật cản di động trong simulation.
- Robot gặp khó khăn với cấu hình vật cản phức tạp (hành lang hẹp, 2 vật cản lệch nhau) - cần thêm training data cho các scenarios này.
- Thiếu cơ chế giao tiếp giữa robots (fully decentralized có ưu và nhược điểm).
- Policy hiện tại là offline learning - không có khả năng học thêm từ các tình huống mới gặp trong thực tế.

Về quy mô thực nghiệm:

- Chỉ 1 robot thực tế (so với 44 trong simulation) do hạn chế phần cứng (đã triển khai robot thứ 2 nhưng gặp trục trặc Jetson Nano thứ 2 bị hỏng nguồn).
- Môi trường test nhỏ (4m x 3m), số episodes hạn chế do thời gian setup và charging.
- Chưa thử nghiệm trong môi trường outdoor hoặc với nhiều người di chuyển cùng lúc.

4.3.5 Hướng cải tiến

Dựa trên kết quả thực nghiệm và các hạn chế đã phân tích, các hướng cải tiến tiềm năng bao gồm:

Về phần cứng:

- Nâng cấp Jetson Nano lên Jetson Orin hoặc mini PC để chạy inference local, giảm độ trễ do mạng và tăng tính độc lập của robot.
- Sử dụng LiDAR tốt hơn (hoặc camera depth) để cho phép tốc độ di chuyển nhanh hơn.
- Bổ sung thêm cảm biến (camera depth, ultrasonic) để phát hiện vật cản ở vùng mù của LiDAR (bàn chân người, vật thấp).
- Thiết kế lại khung xe với gầm cao hơn và bánh xe có độ bám tốt hơn để hoạt động trên nhiều loại bề mặt.

Về thuật toán:

- **Online learning / Continual learning:** Cho phép robot học thêm từ các tình huống thực tế mà không cần retrain từ đầu. Đây là hướng quan trọng để thu hẹp khác biệt từ mô phỏng tới thực tế - robot có thể fine-tune policy dựa trên kinh nghiệm thực tế, đặc biệt với các scenarios hiếm gặp trong mô phỏng (người xuất hiện đột ngột, đứng chắn đường, hành lang hẹp). Tuy nhiên, việc triển khai online learning đòi hỏi phần cứng mạnh hơn để chạy cả inference và training trên robot.
- Thêm cơ chế phát hiện và giải quyết deadlock (một số luật recovery phù hợp).

- Áp dụng domain randomization trong training để cải thiện sim-to-real transfer.
- Bổ sung training data cho các scenarios khó (vật cản đứng yên, hành lang hẹp) trước khi deploy.

Về thực nghiệm:

- Mở rộng thực nghiệm với nhiều robots (sau khi sửa phần cứng) để validate khả năng multi-robot.
- Thử nghiệm trong môi trường outdoor và với nhiều người di chuyển để đánh giá robustness.

CHƯƠNG 5

KẾT LUẬN VÀ KIẾN NGHỊ

5.1 Kết luận

Luận văn đã nghiên cứu và triển khai thành công hệ thống điều khiển phân tán cho đa robot tránh va chạm sử dụng học tăng cường sâu. Các kết quả đạt được đáp ứng đầy đủ các mục tiêu đề ra ban đầu.

5.1.1 Về thuật toán và huấn luyện

Luận văn đã triển khai thành công thuật toán PPO (Proximal Policy Optimization) kết hợp với GAE (Generalized Advantage Estimation) cho bài toán điều khiển phân tán đa robot. Phương pháp curriculum learning với 2 giai đoạn huấn luyện (Stage 1 và Stage 2) đã chứng minh hiệu quả rõ rệt:

- **Stage 1:** Huấn luyện 24 robots trong môi trường đơn giản, đạt success rate **83.26%** và collision rate **14.11%** sau 1940 policy updates.
- **Stage 2:** Transfer learning từ Stage 1, tiếp tục huấn luyện 44 robots trong môi trường phức tạp hơn, đạt success rate **89.18%** và collision rate **9.04%** sau 4480 policy updates.
- **Circle Test:** Model đạt 100% success rate với 4-8 robots và 81.5% với 20 robots, vượt trội so với phương pháp NH-ORCA (65% với 20 robots).

Các kỹ thuật cải tiến được áp dụng thành công bao gồm: Gradient Clipping để ổn định huấn luyện, Entropy Decay để cân bằng exploration-exploitation, và Learning Rate Scheduling để điều chỉnh tốc độ học theo tiến trình.

5.1.2 Về thiết kế và triển khai phần cứng

Luận văn đã thiết kế và xây dựng thành công 1 robot prototype với chi phí thấp, bao gồm:

- Jetson Nano làm bộ xử lý chính, kết nối với Master PC qua ROS network
- LiDAR LD19 (360°, 455 beams, 10Hz) cho quan sát môi trường
- Arduino điều khiển động cơ với PID controller đã được chứng minh ổn định
- IMU hỗ trợ sensor fusion với UKF

Bộ điều khiển PID cho động cơ DC đã được thiết kế và chứng minh ổn định bằng phương pháp Lyapunov, đảm bảo robot đáp ứng nhanh và chính xác với các lệnh vận tốc từ policy network.

5.1.3 Về triển khai từ mô phỏng vào thực tế

Việc triển khai model từ mô phỏng sang robot thực tế đã thành công với sự hỗ trợ của:

- **Cartographer SLAM:** Cung cấp localization trong môi trường thực, cho phép robot xác định vị trí và hướng chính xác.
- **UKF Sensor Fusion:** Kết hợp dữ liệu từ odometry và IMU để ước lượng state ổn định hơn.
- **Các cơ chế an toàn:** Giới hạn vận tốc (0.3 m/s), safety check, stuck detection và recovery đảm bảo robot hoạt động an toàn trong thực nghiệm.

Kiến trúc hệ thống được thiết kế phi tập trung về mặt thuật toán (mỗi robot quyết định độc lập dựa trên quan sát cục bộ), với inference tập trung trên Master PC do hạn chế phần cứng của Jetson Nano. Thiết kế này cho phép dễ dàng mở rộng sang fully decentralized khi nâng cấp phần cứng.

5.1.4 Đánh giá tổng thể

So với mục tiêu đề ra ban đầu (success rate > 80%), kết quả đạt được (89.18% trong mô phỏng) đã vượt mục tiêu. Mặc dù success rate thấp hơn so với bài báo gốc CADRL (96.5-100%), sự khác biệt này được giải thích bởi:

- Số lượng robots trong training cao hơn (44 vs 4-20), tăng độ khó đáng kể
- Cấu hình sensor khác nhau (360° vs 180° LiDAR)
- Tốc độ robot thấp hơn (0.3 m/s vs 1.0 m/s) do hạn chế phần cứng

Quan trọng hơn, luận văn đã chứng minh tính khả thi của việc triển khai phương pháp deep RL trên phần cứng thực tế với chi phí thấp, mở ra hướng ứng dụng cho các hệ thống robot trong công nghiệp và logistics.

5.2 Kiến nghị

5.2.1 Cải tiến phần cứng

- **Nâng cấp bộ xử lý:** Thay Jetson Nano bằng Jetson Orin hoặc mini PC với GPU mạnh hơn để chạy inference local, giảm latency từ 50-100ms xuống dưới 20ms.
- **LiDAR tần số cao:** Sử dụng LiDAR chất lượng tốt hơn, cho phép tăng tốc độ di chuyển và phản ứng nhanh hơn với vật cản động.
- **Cải thiện localization:** Bổ sung camera depth hoặc stereo camera để nâng cao độ chính xác định vị, đặc biệt trong môi trường ít đặc trưng.

5.2.2 Cải tiến thuật toán

- **Xử lý deadlock:** Nghiên cứu thêm cơ chế phát hiện và giải quyết deadlock khi nhiều robots cùng đi qua một điểm hẹp, có thể kết hợp thêm một số rule định

sẵn cho robot để tránh nhau, và cho nó học khi huấn luyện.

- **Domain randomization:** Trong quá trình huấn luyện, thay đổi ngẫu nhiên các thông số môi trường (nhiều cảm biến, độ trễ, ma sát bánh xe) để policy học được cách thích nghi với nhiều điều kiện khác nhau. Điều này giúp model hoạt động tốt hơn khi chuyển từ mô phỏng sang thực tế.
- **Hierarchical policy:** Chia policy thành 2 tầng: tầng cao (high-level) quyết định hướng đi tổng thể và tầng thấp (low-level) điều khiển chi tiết để tránh va chạm. Kiến trúc này giúp robot xử lý tốt hơn các tình huống phức tạp như đi qua hành lang hẹp hoặc né nhiều vật cản liên tiếp.

5.2.3 Mở rộng quy mô thực nghiệm

- Tăng số lượng robots thực nghiệm lên 5-10 để đánh giá khả năng scale của hệ thống
- Thử nghiệm trong nhiều môi trường với điều kiện ánh sáng và địa hình đa dạng
- Đánh giá hiệu suất trong các kịch bản ứng dụng thực tế như kho hàng hoặc nhà máy

5.3 Hướng phát triển

Dựa trên nền tảng của luận văn, các hướng nghiên cứu tiếp theo bao gồm:

1. **Triển khai trên nhiều bài toán khác:** Mở rộng policy để xử lý nhiều task khác nhau (điều hướng trong môi trường không xác định trước, điều khiển tác hợp trong cùng môi trường - cooperative manipulation).
2. **Continuous learning:** Phát triển khả năng học tiếp tục (continual learning) để robot có thể thích nghi với môi trường mới mà không cần huấn luyện lại từ đầu.
3. **Heterogeneous robots:** Nghiên cứu điều khiển hệ robot không đồng nhất (khác nhau về kích thước, tốc độ, sensor) hoạt động cùng nhau.
4. **3D environment:** Mở rộng phương pháp cho môi trường 3D với các robots có khả năng bay (drone) hoặc di chuyển trên địa hình phức tạp.
5. **Human-robot interaction:** Tích hợp khả năng tương tác an toàn với con người trong môi trường làm việc chung.

5.4 Lời kết

Luận văn đã đạt được mục tiêu nghiên cứu đề ra: xây dựng thành công hệ thống điều khiển phân tán cho đa robot tránh va chạm sử dụng học tăng cường sâu, từ thiết kế thuật toán, huấn luyện trong mô phỏng, đến triển khai trên robot thực tế. Kết quả cho

thấy deep reinforcement learning là phương pháp khả thi và hiệu quả cho bài toán điều khiển đa robot, với tiềm năng ứng dụng rộng rãi trong công nghiệp 4.0.

Mặc dù còn một số hạn chế về phần cứng và quy mô thực nghiệm, luận văn đã đặt nền móng vững chắc cho các nghiên cứu tiếp theo trong lĩnh vực điều khiển đa robot tự trị tại Việt Nam.

TÀI LIỆU THAM KHẢO

- [1] O. Khatib, “Real-time obstacle avoidance for manipulators and mobile robots,” *The International Journal of Robotics Research*, **jourvol 5, number 1, pages 90–98**, 1986.
- [2] S. M. LaValle, “Rapidly-exploring random trees: A new tool for path planning,” **in** *Technical Report TR 98-11, Computer Science Dept., Iowa State University*, 1998.
- [3] S. Karaman **and** E. Frazzoli, “Sampling-based algorithms for optimal motion planning,” *The International Journal of Robotics Research*, **jourvol 30, number 7, pages 846–894**, 2011.
- [4] J. van den Berg, S. J. Guy, M. Lin **and** D. Manocha, “Reciprocal n-body collision avoidance,” **in** *Robotics Research*, Springer, 2011, **pages 3–19**.
- [5] Y. LeCun, Y. Bengio **and** G. Hinton, “Deep learning,” *Nature*, **jourvol 521, number 7553, pages 436–444**, 2015.
- [6] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski **and others**, “Human-level control through deep reinforcement learning,” *Nature*, **jourvol 518, number 7540, pages 529–533**, 2015.
- [7] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver **and** K. Kavukcuoglu, “Asynchronous methods for deep reinforcement learning,” **in** *Proc. Int. Conf. Mach. Learn. (ICML)*, **pages 1928–1937**, 2016.
- [8] L. Tai, G. Paolo **and** M. Liu, “Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation,” **in** *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, IEEE, 2017, **pages 31–36**.
- [9] Y. F. Chen, M. Liu, M. Everett **and** J. P. How, “Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning,” **in** *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, IEEE, 2017, **pages 285–292**.
- [10] P. Long, T. Fan, X. Liao, W. Liu, H. Zhang **and** J. Pan, “Towards optimally decentralized multi-robot collision avoidance via deep reinforcement learning,” **in** *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Brisbane, Australia: IEEE, 2018, **pages 6252–6259**. DOI: [10.1109/ICRA.2018.8461113](https://doi.org/10.1109/ICRA.2018.8461113).
- [11] D. P. Kingma **and** J. Ba, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2014.

- [12] J. Schulman, P. Moritz, S. Levine, M. Jordan **and** P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” *arXiv preprint arXiv:1506.02438*, 2015.
- [13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford **and** O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [14] K. J. Åström **and** R. M. Murray, *Feedback systems: an introduction for scientists and engineers*. Princeton university press, 2006.
- [15] R. Siegwart, I. R. Nourbakhsh **and** D. Scaramuzza, “Introduction to autonomous mobile robots,” *MIT press*, 2011.
- [16] S. J. Julier **and** J. K. Uhlmann, “Unscented filtering and nonlinear estimation,” *Proceedings of the IEEE*, **jourvol** 92, **number** 3, **pages** 401–422, 2004.
- [17] S. Thrun, W. Burgard **and** D. Fox, *Probabilistic robotics*. MIT press, 2005.
- [18] W. Hess, D. Kohler, H. Rapp **and** D. Andor, “Real-time loop closure in 2D LIDAR SLAM,” **in** *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, IEEE, 2016, **pages** 1271–1278. DOI: [10.1109/ICRA.2016.7487258](https://doi.org/10.1109/ICRA.2016.7487258).
- [19] Google Cartographer Contributors. (2023). Tuning Methodology – Cartographer ROS Documentation, **url**: <https://google-cartographer-ros.readthedocs.io/en/latest/tuning.html> (**urlseen** 07/12/2025).

PHỤ LỤC

A. Dữ liệu chi tiết kết quả thực nghiệm

Bảng 5.1 trình bày toàn bộ dữ liệu thực nghiệm với 17 test cases, bao gồm tọa độ xuất phát, tọa độ đích, tọa độ thực tế đạt được, và các metrics đánh giá.

Bảng 5.1. Dữ liệu chi tiết toàn bộ kết quả thực nghiệm

#	Category	Scenario	x_s	y_s	x_g	y_g	d_g	x_r	y_r	Steps	Reward	Error	Status
1	0 obstacles	—	0	0	0.96	0	0.96	0.96	0.01	110	44.4	0.01	reach goal
2	0 obstacles	—	0.96	0.01	1.99	0.33	1.08	2.05	0.25	100	41.6	0.10	reach goal
3	0 obstacles	—	0	0	2.41	0.4	2.44	2.38	0.34	250	46.2	0.07	reach goal
4	0 obstacles	—	0	0	1.98	-0.38	2.02	2.08	-0.35	215	48.7	0.10	reach goal
5	0 obstacles	—	0	0	2.56	-0.74	2.66	2.45	-0.77	203	50.65	0.11	reach goal
6	1 vật tĩnh	trung điểm	0	0	2.29	0.35	2.32	2.24	0.43	190	45.4	0.09	reach goal
7	1 vật tĩnh	gần robot	0	0	1.98	-0.02	1.98	2.01	0.05	176	44.37	0.08	reach goal
8	1 vật tĩnh	goal gần	2.01	0.05	1.35	0.45	0.77	1.37	0.45	144	42.8	0.02	reach goal
9	2 vật tĩnh	thẳng hàng, lệch phải	1.37	0.45	0.04	-0.22	1.52	0.03	-0.11	457	55.39	0.11	reach goal
10	2 vật tĩnh	thẳng hàng	0	0	0.11	0.05	0.12	0.11	0.03	230	48.5	0.02	reach goal
11	2 vật tĩnh	2 vật lệch nhau	1.92	-0.21	0.36	-0.04	1.57	1.25	0.22	500	5.71	0.36	timeout
12	1 người	đi chậm ngang qua	0	0	1.95	0	1.95	1.87	0.02	208	42.8	0.08	reach goal
13	1 người	đi rồi đứng lại	0	0	1.97	0	1.97	1.88	0.01	310	50.8	0.09	reach goal
14	1 người	đi nhanh qua	0	0	2.04	-0.02	2.04	1.96	0.02	270	45.2	0.09	reach goal
15	1 người	đứng chắn đường	0	0	1.98	0	1.98	1.24	0.05	500	6.6	0.74	timeout
16	2 người	đi từng người	0	0	2.25	0	2.25	2.16	0.03	280	51.78	0.10	reach goal
17	2 người	1 đứng lại	0	0	2.17	0	2.17	1.7	0.08	500	12.8	0.47	timeout

Chú thích:

- x_s, y_s : Tọa độ xuất phát (m)
- x_g, y_g : Tọa độ đích (m)
- d_g : Khoảng cách Euclidean từ xuất phát đến đích (m)
- x_r, y_r : Tọa độ thực tế robot đạt được (m)
- Steps: Số bước điều khiển (control steps tại 20Hz)
- Reward: Tổng reward tích lũy trong episode
- Error: Khoảng cách từ vị trí cuối đến goal (m)

Ghi chú thực nghiệm:

- Test 3: Robot bị khựng nhẹ do map cập nhật chậm
- Test 8: Robot hơi dao động qua lại ngay vị trí vật cản
- Test 9: Robot né sang trái và đi thẳng vượt
- Test 10: Robot khựng khá lâu ở vật cản 1
- Test 11: Robot bị dừng trước vật cản 2
- Test 12: Robot giảm tốc nhẹ và tiến chậm qua trái
- Test 13: Robot dừng khi gần va chạm, khựng lại rồi né trái đi tiếp (bánh xe có đựng nhẹ vào ngón chân)
- Test 14: Robot giảm tốc nhanh, người đi qua robot tiếp tục di chuyển

- Test 15: Robot giảm tốc mạnh và dừng lại, dao động qua lại nhưng không né để đi tiếp
- Test 16: Robot giữ tốc độ chậm để người đi qua, có dao động nhẹ
- Test 17: Robot giảm tốc né được người 1, đến người 2 đứng thì dao động và bị timeout

File dữ liệu gốc được lưu tại: docs/result/experiment/Real_test_result.csv

LÝ LỊCH TRÍCH NGANG

Họ và tên: Nguyễn Tấn Khôi
Ngày, tháng, năm sinh: 08/02/1999
Nơi sinh: TP. Hồ Chí Minh
Địa chỉ liên lạc: TP. Hồ Chí Minh
Email: tankhoi.kn@gmail.com

QUÁ TRÌNH ĐÀO TẠO

(Bắt đầu từ Đại học đến nay)

Thời gian	Nơi đào tạo	Ngành học	Văn bằng
2017 – 2021	ĐH Bách Khoa TP.HCM	Kỹ thuật Điều khiển & Tự động hóa	Kỹ sư
2021 – 2025	ĐH Bách Khoa TP.HCM	Kỹ thuật Điều khiển & Tự động hóa	Thạc sĩ

QUÁ TRÌNH CÔNG TÁC

(Bắt đầu từ khi đi làm đến nay)

Thời gian	Nơi công tác	Chức vụ
01/2025 – nay	Công ty TNHH AI POWER	Kỹ sư Phần mềm Cao cấp
03/2022 – 12/2024	Công ty Cổ phần SKCOMPANY Việt nam	Kỹ sư Phần mềm
06/2021 – 02/2022	Công ty TNHH AI POWER	Kỹ sư Phần mềm